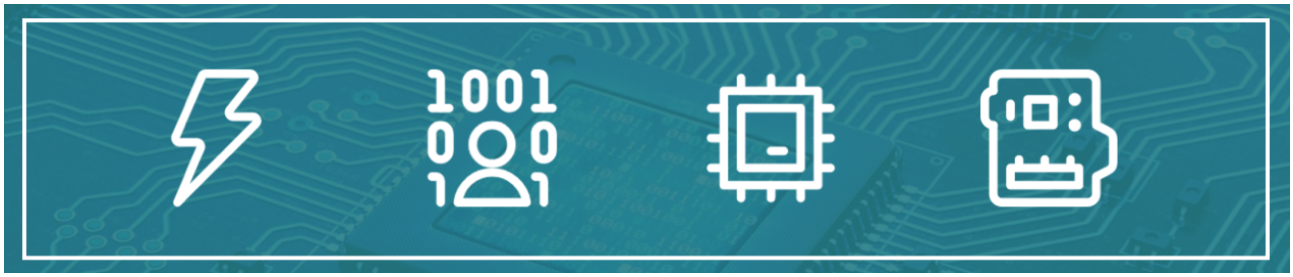
ING3
Semestre 5

LIVRET DE TRAVAUX PRATIQUES

PROTOTYPAGE ÉLECTRONIQUE



Département Électronique
ECE – Programme Grande École d'Ingénieurs

Version du 14 septembre 2026

Programme des séances de TP


TP 1 - Initiation à la NeuroBoard S3 . . . . . p. 4


TP 2 - L'écran OLED . . . . . p. 13


TP 3 - La résistance . . . . . p. 20


TP 4 - Le condensateur . . . . . p. 29


TP 5 - L'amplificateur opérationnel . . . . . p. 36


TP 6 - Le filtrage analogique . . . . . p. 42


TP 7 - La diode à jonction PN . . . . . p. 51


TP 8 - La chaîne d'acquisition . . . . . p. 58


TP 9 - Le transistor bipolaire en régime statique . . . . . p. 63


TP 10 - Analyse spectrale . . . . . p. 67


TP 11 - Démodulation AM . . . . . p. 70


TP 12 - L'additionneur numérique et le multiplexeur . . . . . p. 77


TP 13 - Le diviseur d'horloge . . . . . p. 81



TP 14 - Interruptions externes . . . . . p. 85



TP 15 - Timers et GPIO de l'ESP32-S3 . . . . . p. 91

TP n° 1

Initiation à la NeuroBoard S3

Motivation

La carte NeuroBoard S3 est une carte électronique conçue par l'ECE, construite autour du **microcontrôleur ESP32-S3** développé par Espressif Systems. La programmation de cette carte permet de lire et d'analyser des signaux électriques issus de divers capteurs via ses **entrées** et de piloter des composants électriques (LEDs, moteurs, etc.) via ses **sorties**. Grâce à sa puissance de calcul accrue et à ses fonctionnalités avancées (Wi-Fi, Bluetooth, vision), elle permet de réaliser des tâches très diverses en robotique, informatique embarquée, systèmes connectés (IoT) et intelligence artificielle embarquée.

La carte comporte de nombreuses broches d'entrées/sorties numériques (GPIO) opérant à un niveau logique de 3,3 V, toutes capables de générer des signaux PWM (Pulse Width Modulation, modulation de largeur d'impulsion). De plus, le microcontrôleur intègre deux convertisseurs analogique-numérique (CAN / ADC) de 12 bits (codage de 0 à 4095) : **ADC1** (associé aux GPIO 1 à 10) et **ADC2** (associé aux GPIO 11 à 20). Les autres GPIOs fonctionnent uniquement comme entrées/sorties numériques, comme illustré sur la Figure 1.1.

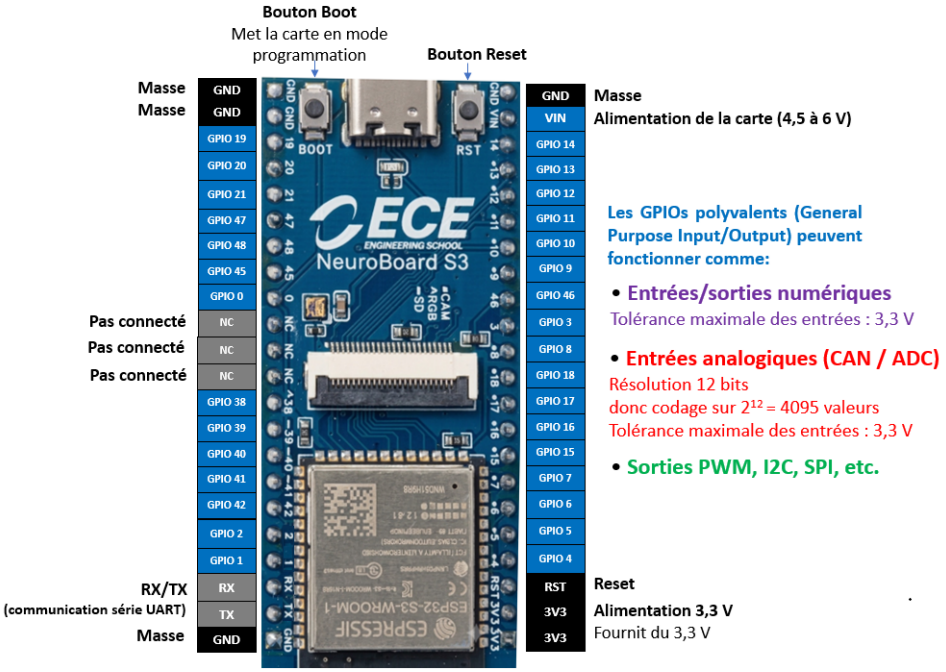


Figure 1.1 – Principaux ports de la carte Neuroboard S3

Dans ce TP nous allons prendre en main la carte NeuroBoard S3 au travers d'exemples simples que nous vous invitons à réaliser chez vous (partie théorique) puis un cas plus concret au travers d'un exemple de maison intelligente simplifiée.

L'objectif de ce TP est la prise en main de la carte NeuroBoard S3, notamment pour lire et écrire sur ses entrées/sorties numériques (GPIO).

I. Partie théorique et pré-expérimentale

A. Mise en route

Pour commencer, nous allons faire clignoter une LED avec la carte ECE NeuroBoard S3 et le Code C++ dans Arduino IDE ou VSCode PlatformIO 1.1 :

```
////////////////////////////////
// INCLUSION DES BIBLIOTHEQUES //
// Sous VSCode PlatformIO //
////////////////////////////////
//#include<NeuroBoardS3.h> //

//////////////////
// DECLARATIONS //
//////////////////
const int Led = 3;

////////////////////
// FONCTION SETUP //
////////////////////
void setup() {
  pinMode(Led, OUTPUT); // La broche 3 est une sortie
}

////////////////////
// FONCTION LOOP //
////////////////////
void loop() {
  digitalWrite(Led, HIGH); // Met la broche au niveau haut (3,3 V) : on allume la LED
  delay(500); // Pause de 500 ms
  digitalWrite(Led, LOW); // Met la broche au niveau bas (0 V) : on éteint la LED
  delay(500); // Pause de 500 ms
}
```

Code 1.1 – Code Arduino IDE ou VSCode Platformio pour faire clignoter une LED

Un code Arduino IDE ou VSCode PlatformIO est typiquement **écrit en langage C** et comprend quatre sections :

- l'**inclusion des bibliothèques** : permet d'importer les fichiers d'en-tête (obligatoire sous PlatformIO, géré automatiquement par Arduino IDE) ;
- les **déclarations** : assignation d'un nom de variable aux broches et déclaration des variables globales ;
- la **fonction SETUP** : exécutée qu'une seule fois, au démarrage, permet de définir si une pin est une entrée ou une sortie ;
- la **fonction LOOP** : la boucle infinie du programme où les instructions se répètent à la suite des autres à l'infini.

Prenez le temps de bien identifier les trois parties du code. C'est important. On commence par les déclarations, les paramétrages de la carte puis on entre dans la boucle infinie `loop()`.

T1. Dessiner le logigramme du code présenté-ci dessus.

On rappelle qu'un tel schéma se lit de haut en bas, que l'**étape initiale** et l'éventuelle **étape finale** sont représentées dans des rectangles aux bords arrondis, les **étapes de test** sont représentées par des losanges, les delays sont représentés par des rectangles avec un seul bord arrondi et que les **étapes de process** sont des rectangles. Un exemple est proposé à la page suivante.

On se propose d'utiliser la nomenclature flowchart de Visual Paradigm ou de draw.io, qui sont identiques. Pour la forme utilisée pour le `delay`, s'il existe plusieurs temporisations, il faudra agrandir ou réduire la taille de la forme en fonction du temps (un `delay(100)` sera plus petit qu'un `delay(1000)`).

Ci-dessous, un tableau qui regroupe quelques symboles utiles.

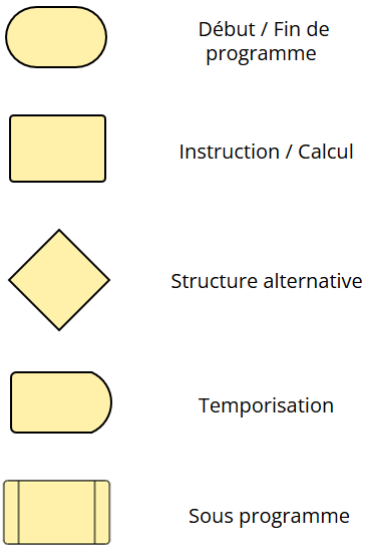


Figure 1.2 – Exemples de symboles

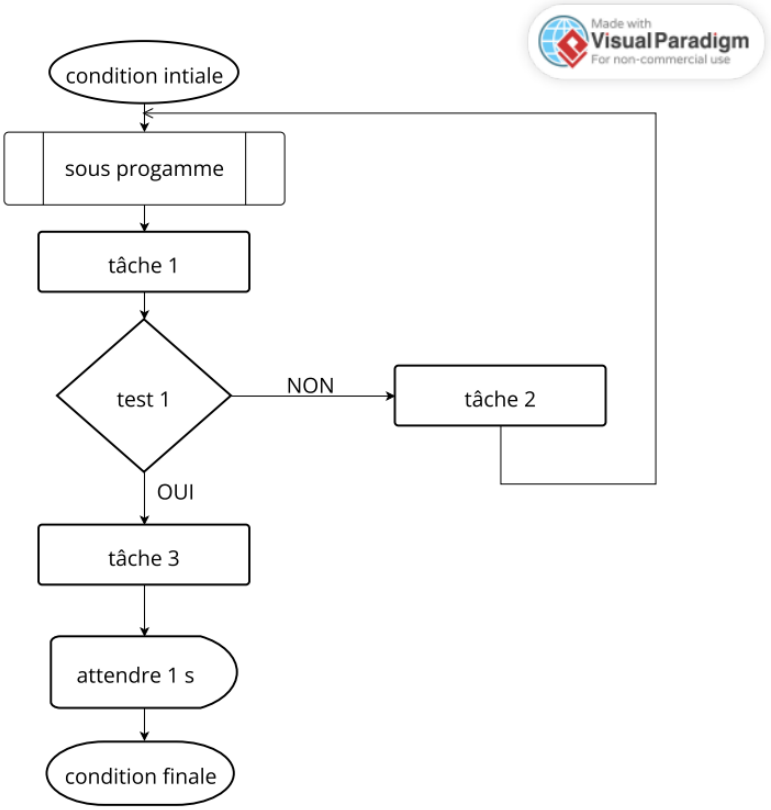


Figure 1.3 – Exemple de logigramme

On souhaite faire en sorte que ce code permette de faire clignoter une LED.

Remarque : Sur la carte NeuroBoard S3, les numéros spécifiés dans le code correspondent directement aux numéros de GPIO (par exemple, `1` pour le GPIO1). Il n'est plus nécessaire d'ajouter un préfixe « A » pour les broches analogiques. La masse se branche sur la broche marquée GND de la carte.

B. Entrées/sorties numériques

Toutes les entrées/sorties de la carte sont numériques car par essence, le microcontrôleur travaille avec des données numériques (des 0 et des 1). Il convient de distinguer deux cas :

- utilisation en **entrée** par rapport à la carte :
  - déclarer le numéro de pin en entrée :

    `pinMode(numeroDePin, INPUT);` dans la procédure `setup()` ;

  - lire la valeur numérique (`LOW` ou `HIGH`) :

    `digitalRead(numeroDePin);` dans la procédure `loop()`.

- utilisation en **sortie** par rapport à la carte :
  - déclarer le numéro de pin en sortie :

    `pinMode(numeroDePin, OUTPUT);` dans la procédure `setup()` ;

– écrire la valeur numérique :

`digitalWrite(numeroDePin, LOW);` ou `digitalWrite(numeroDePin, HIGH);` dans la procédure `loop()`.

Remarque : Pour des raisons pratiques, le numéro de pin peut être mis dans une variable, tel est le cas de Led dans l'exemple donné plus haut où Led correspond à la pin numérique 3.

C. Entrées analogiques

Certaines entrées/sorties (GPIO1 à GPIO20, associées aux convertisseurs internes ADC1 et ADC2) permettent de lire mais pas de générer des signaux analogiques. Elles sont dotées de ce que l'on appelle un **convertisseur analogique-numérique (CAN/ADC)** qui traduit le signal analogique en entrée (n'importe quelle valeur entre 0 et 3,3 V par défaut) en un mot binaire interprétable comme un nombre dans le code.

Pour ce faire :

- déclarer le numéro de pin en entrée : `pinMode(numeroDePin, INPUT);` en choisissant une pin parmi celles disposant d'un ADC (GPIO1 à GPIO20).
- lire la valeur analogique : `analogRead(numeroDePin);` dans la fonction `loop()`.

Le signal analogique (entre 0 et 3,3 V par défaut) est traduit en un mot binaire compris entre 0 et $2^{12} - 1 = 4095$. Cette conversion prend environ 10 µs à se faire. Ainsi, la plage de valeur [0 ; 3,3 V] est sous-divisée en tranches égales de $\frac{3{,}3\,\text{V}}{4096\text{ combinaisons}} \approx 0{,}805\,\text{mV}$.

Exemples :

- 3,3 V → 4095 (vérification : $4095 \times 0{,}805\,8\,\text{mV} \approx 3{,}3\,\text{V}$)
- 1,65 V → 2047 (vérification : $2047 \times 0{,}805\,8\,\text{mV} \approx 1{,}65\,\text{V}$)
- 0 V → 0 (vérification : $0 \times 0{,}805\,8\,\text{mV} = 0\,\text{V}$)

Travailler avec des nombres entre 0 et 4095 n'est pas ce qu'il y a de plus pratique...
Dans la pratique, on a recours à la fonction `map(value, fromLow, fromHigh, toLow, toHigh)` qui permet de transcrire l'intervalle [0 ; 4095] en un intervalle choisi.

Exemples : `valeurConvertie = map(valeurInitiale,0,4095,0,100);` → un nombre entre 0 et 4095 sera traduit entre 0 et 100.

- 3,3 V → 100 (vérification : $\frac{4095}{4095} \times 100 = 100$)
- 1,65 V → 50 (vérification : $\frac{2047}{4095} \times 100 \approx 50$)
- 0 V → 0 (vérification : $\frac{0}{4095} \times 100 = 0$)

 Remarque : la fonction map renvoie un entier.

D. Système d'éclairage automatique

La photorésistance (ou LDR pour Light Dependent Resistor) est un composant dont la résistance varie en fonction de la luminosité : dans la pénombre, la LDR présente une résistance de l'ordre de centaines de kΩ et dans un environnement très lumineux, quelques dizaines d'Ω. Afin de l'utiliser comme capteur de luminosité, on la branche dans un montage appelé pont diviseur de tension, de sorte à ce que la variation de la luminosité fasse varier une tension, qui sera ensuite interprétée par la NeuroBoard S3. La tension analogique image de la luminosité est lue sur la pin GPIO1, et si la valeur est inférieure à une certaine valeur, (« XX »), alors on allume la LED intégrée (GPIO48).

Le circuit étudié est décrit sur la Figure 1.4.

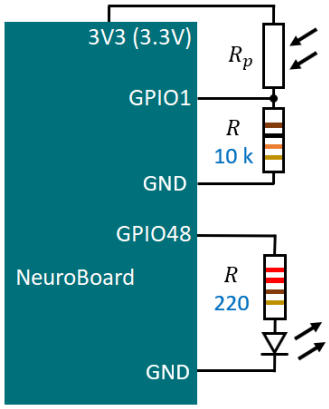


Figure 1.4 – Circuit du système d'éclairage automatique

Le potentiel électrique (signal analogique) sur la pin GPIO1 vaut :

$$V = \frac{R}{R + R_p} \cdot 3,3 \tag{1.1}$$

```
//#include<NeuroBoardS3.h> //  Sous VSCode PlatformIO

int sensorPin = _________;
int ledPin = _________;
int sensorValue = 0;

void setup() {
  pinMode(ledPin, OUTPUT);
  ________________________;
}

void loop() {
  // Lire la valeur du capteur
  sensorValue = ____________________;
  // Si la luminosité est inférieure à XX, allumer la LED pendant 5 secondes
  if(_________________________) {
    digitalWrite(_____, _____);
    _________________________;
    digitalWrite(_____, _____); // Éteindre la LED
  }
}
```

Code 1.2 – Code à compléter pour le système d'éclairage automatique

T2. Compléter et commenter le Code 1.2.

T3. Proposer un logigramme.

T4. Proposer un code qui lit un signal analogique sur la pin GPIO1 et qui traduit le nombre obtenu en une tension entre 0 V et 3,3 V (On suppose ici que la tension maximale est de 3,3 V).

E. Signal rectangulaire PWM

Contrairement à ce que l'on pourrait penser, la fonction `analogWrite` ne permet pas de générer un signal analogique. Elle permet de générer des signaux rectangulaires, encore appelés PWM pour *Pulse-Width Modulation*, sur l'ensemble des broches GPIO de la NeuroBoard S3 (grâce à son périphérique matériel LEDC). La Figure 1.5 présente l'allure d'un signal rectangulaire.

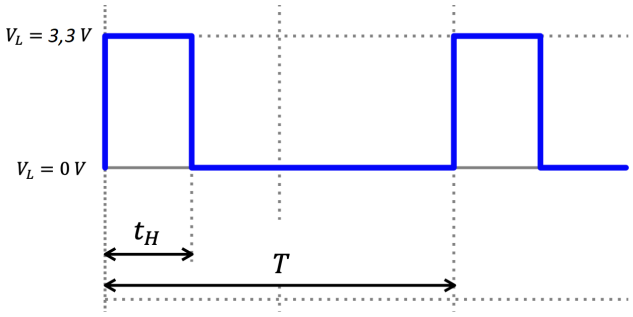


Figure 1.5 – Signal rectangulaire.

Ce dernier a les paramètres suivants :

- les **tensions à l'état bas** V_L **et à l'état haut** V_H qui valent respectivement 0 et 3,3 V ;
- la **période** : tous les combien de temps on retrouve un état haut suivi d'un état bas ;
- le **temps à l'état haut** : durée pendant laquelle le signal est à l'état haut pendant une période ;
- le **rapport cyclique** : rapport du temps à l'état haut sur une période.

$$\alpha = \frac{t_H}{T} \tag{1.2}$$

Sur la carte NeuroBoard S3, $V_L = 0\,\text{V}$ et $V_H = 3{,}3\,\text{V}$. Contrairement à l'Arduino Nano, la NeuroBoard S3 n'a pas de fréquences différentes selon les broches : le signal PWM est généré à une fréquence uniforme de 1 000 Hz (1 kHz) sur l'ensemble des GPIOs.

La commande C++ pour générer un signal PWM est la suivante :

```
analogWrite(numeroDeGPIO_PWM,  α × 255);
```

Sur la NeuroBoard S3, que ce soit sous Arduino IDE ou PlatformIO (avec le framework Arduino), la commande `analogWrite()` s'appuie sur le périphérique matériel LEDC. Par défaut, la résolution est configurée sur 8 bits (2^8), soit des valeurs allant de 0 à $2^8 - 1 = 255$. Le deuxième argument correspond donc au produit : $\alpha \times 255$.

Par exemple, pour faire varier la luminosité d'une LED connectée sur le GPIO 1 :

```
void setup() {
  // Aucune configuration préalable n'est requise pour analogWrite()
}

void loop() {
  // Génère un signal PWM avec un rapport cyclique d'environ 42% (107/255)
  analogWrite(1, 107);
}
```

Code 1.3 – Exemple d'utilisation de la PWM sur la NeuroBoard S3

T5. Quelle est la période et le temps à l'état haut d'un signal PWM émanant d'un GPIO (par exemple `GPIO1`) en utilisant la ligne de code suivante : `analogWrite(1, 107)` ? Représenter ce signal sur deux périodes.

II. Partie pratique

A. Prise en main

On souhaite tester le code présenté plus haut dans VS Code PlatformIO :

1. **Name** : donner un nom au projet ;
2. **Board** : sélectionner NeuroBoard S3 (ECE).
3. **Framework** : sélectionner Arduino. Tester puis conserver la configuration qui vous donne un téléversement réussi.

E1. Tester le code avec le matériel du kit afin de faire clignoter la LED. Vous pouvez vous inspirer de la Figure 1.6 **(a)**. La patte la plus longue de la diode est la patte (+) et doit donc être reliée à la pin GPIO 3 de la NeuroBoard S3.

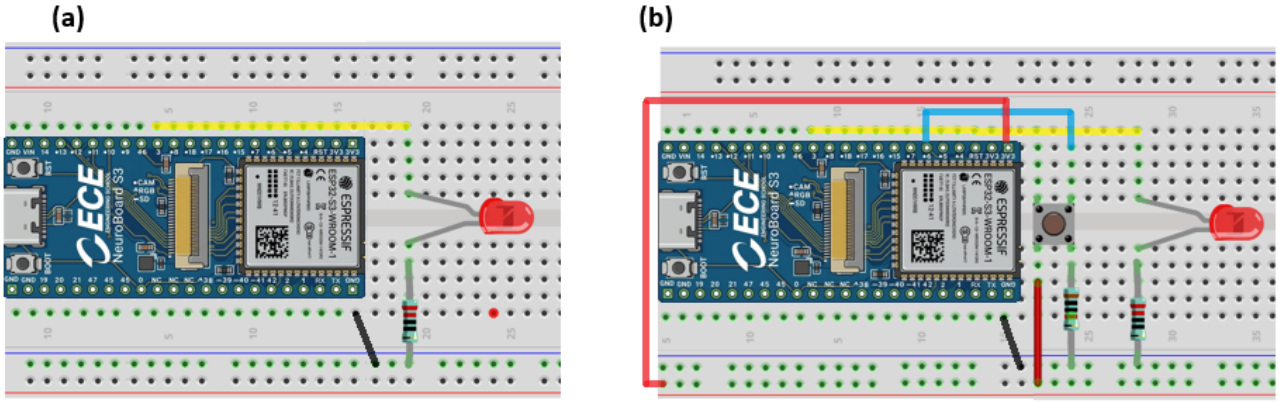


Figure 1.6 – a) Diode branchée sur la GPIO 3 de la NeuroBoard S3
b) Bouton en pull-down sur la GPIO 6 de la NeuroBoard S3.

E2. Modifier le circuit et coder de sorte à ce que lorsque l'on appuie sur un bouton, la LED s'allume pendant 2 secondes. Vous pouvez vous inspirer de la Figure 1.6 **(b)** en prenant une résistance de 10 kΩ. (Penser à utiliser la commande : `digitalRead()`). Proposer un logigramme.

B. Signal PWM

E3. Générer un signal PWM de rapport cyclique $\alpha = 0,7$ et l'observer à l'oscilloscope.

C. Capteur de température

Vous disposez d'un capteur de température dans votre kit : le LM35. Ce dernier génère une tension proportionnelle à la température, à raison de 10 mV par °C.

Attention ! Afin d'éviter tout risque de détérioration de la carte NeuroBoard S3, il faut brancher une résistance de 10 kΩ en série à la sortie du capteur. Nous verrons plus tard que c'est pour limiter le courant qui pourrait transiter par la pin de la NeuroBoard S3. En effet, plus la température est élevée, et plus la tension va augmenter, ce qui augmente le courant ($U = RI$), et cela prévient également un cas de détérioration du capteur : si ce dernier génère une trop grande tension alors qu'il ne devrait pas.

Câbler le circuit correspondant à celui de la Figure 1.7 **(b)**. Le code couleur de la résistance de 10 kΩ est mentionné sur l'étiquette de votre kit (Marron - Noir - Noir - Rouge - Or).

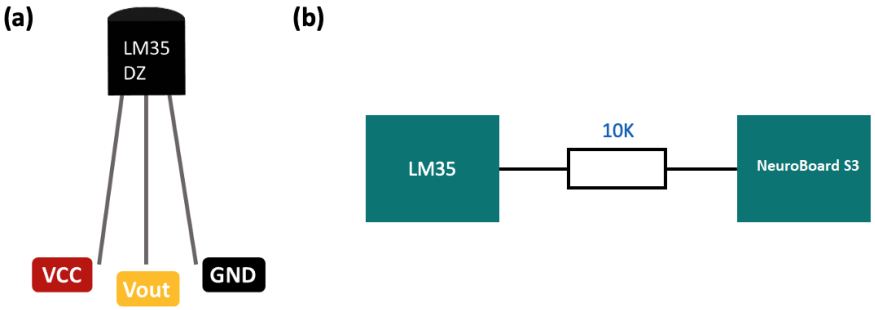


Figure 1.7 – **(a)** Schéma de branchement du LM35
(b) Vue synoptique du circuit à réaliser

E4. Proposer et tester un code permettant d'allumer des LEDs par paliers de 5 degrés comme suit :

- 1 LED allumée entre 10 et 15 °C ;
- 2 LEDs allumées entre 15 et 20 °C ;
- 3 LEDs allumées entre 20 et 25 °C ;
- 4 LEDs allumées entre 25 et 30 °C.

L'essentiel

- Carte électronique comprenant un microcontrôleur ESP32-S3 pouvant lire des entrées et écrire (actionner) des sorties
- Signaux analogiques et numériques
- Déclaration, procédure `setup()` et procédure infinie `loop()`
- Signal PWM

TP n° 2

L'écran OLED

Lors du TP « Initiation à NeuroBoard S3 », nous avons vu comment utiliser les entrées/sorties numériques, les entrées analogiques (capteur de lumière ou de température) et comment générer des signaux rectangulaires (PWM).

Au cours de ce TP, nous allons lire un signal analogique issu d'un potentiomètre et afficher la valeur en tension sur un écran OLED, comme illustré sur la Figure 2.1.

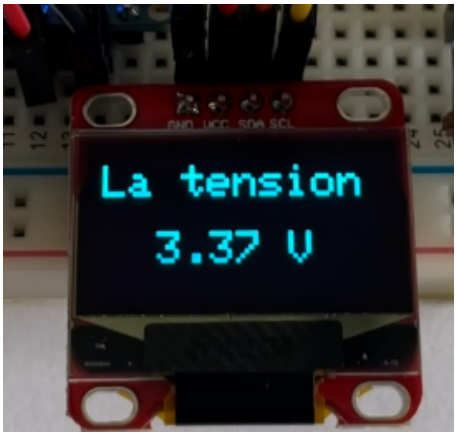


Figure 2.1 – Écran OLED affichant une tension.

L'objectif de ce TP est de revoir la lecture d'un signal analogique et d'apprendre à utiliser l'écran OLED.

I. Lecture d'une tension analogique

On vous propose de réfléchir à un moyen de lire la tension analogique retournée par un potentiomètre grâce à l'utilisation de la plateforme NeuroBoard S3.

Voici un petit rappel sur le fonctionnement de la conversion analogique-numérique. Lorsque la NeuroBoard S3 reçoit une tension analogique, cette dernière passe par un **convertisseur analogique-numérique** (ADC) afin de la transcrire en un nombre binaire (valeur numérique) sur 12 bits. Comme sur 12 bits, il y a $2^{12} = 4096$ combinaisons, la tension analogique lue entre 0 et 3,3 V est comprise entre les combinaisons `0000 0000 0000` (0 en décimal) et `1111 1111 1111` (4095 en décimal). Afin de convertir ce nombre en une tension en mV, on multiplie ce nombre lu par le max : 3 300 mV divisé par le nombre de valeurs possibles :

$$U_V = U_{read} \cdot \frac{3\,300\,\text{mV}}{4\,095\ \text{états}} \tag{2.1}$$

On vous propose ainsi le code suivant :

```
int sensorPin = 1; // Utilisation du GPIO 1 (ADC)
int reading = 0;
float voltage = 0;

void setup() {
  pinMode(sensorPin, INPUT); // Facultatif

}

void loop() {
  // Lecture de la tension analogique
  reading = analogRead(sensorPin);

  // Calcul de la tension (3.3V max et 4095 états)
  voltage = reading * 3.3 / 4095.0;

  delay(1000); // 1000 ms = 1 s
}
```

Code 2.1 – Code pour lire la tension d'un potentiomètre sur NeuroBoard S3

On vous propose ici de lire la tension analogique retournée par un potentiomètre afin *in fine* d'afficher sa valeur dans le moniteur série de l'ordinateur (le code que vous écrivez est interprété par le microcontrôleur !).

Le moniteur série permet la communication entre le microcontrôleur et l'ordinateur. On initie la communication à l'aide de la ligne de code suivante, dans la procédure `setup()` :

```
Serial.begin(115200);
```

Le moniteur série est accessible en cliquant sur l'icône en forme de *prise* : Serial Monitor, en bas à gauche de VS Code PlatformIO (à condition que la NeuroBoard S3 soit branchée).

L'instruction suivante permet alors d'afficher la valeur de la variable « maVariable » dans ce dit moniteur série :

```
Serial.print(maVariable);
```

Comme en C++, la variante `Serial.println` permet d'afficher la valeur d'une variable et d'aller à la ligne suivante.

E1. Proposer un code permettant de lire une tension analogique retournée par le potentiomètre et d'en afficher la valeur dans le moniteur série toutes les 0,8 s.

La relation entre l'angle du potentiomètre θ et la tension lue V s'écrit :

$$V = 3,3 \times \left(1 - \frac{\theta}{360}\right) \tag{2.2}$$

E2. Pour chaque mesure de tension faite, afficher également dans le moniteur série l'angle correspondant, et la combinaison binaire sur 12 bits. En particulier, donner les positions minimale, milieu et maximale d'angle.

E3. En supposant que le potentiomètre utilise une plage d'angles de 0 à 360°, et que le convertisseur analogique - numérique traduit la tension retournée par le potentiomètre sur 12 bits (donc $2^{12} - 1 = 4095$ valeurs possibles), quel est l'angle permettant d'incrémenter (ajouter 1) à la valeur lue par le convertisseur analogique - numérique ?

II. L'écran OLED

La technologie **OLED** (*Organic Led Emitting Diode*) est utilisée dans les applications d'affichage (écrans, TV, afficheurs en tout genres). Elle a été mise au point (brevet) par la société Kodak en 1987 et est apparue sur le marché en 1997. Par rapport à la technologie LCD (afficheurs à cristaux liquides), l'OLED a pour avantage de ne pas avoir besoin de rétroéclairage, lui permettant ainsi de proposer des afficheurs plus fins et présentant un meilleur contraste.

L'écran OLED proposé dans votre kit est un écran OLED 128 x 64. Cela veut dire qu'il présente 64 lignes de 128 pixels.

A. Hardware

L'écran OLED présente 4 broches pour permettre la gestion de l'affichage. Il est alimenté par le microcontrôleur et se connecte au bus I2C.

- **GND** : la masse, à relier à la masse de la NeuroBoard S3 ;
- **VCC** : l'alimentation 3,3 V, à relier à la pin 3V3 de la NeuroBoard S3 ;
- **SCL** : « Serial clock », à relier à la pin GPIO 9 (SCL) ;
- **SDA** : « Serial data », à relier à la pin GPIO 8 (SDA).

E4. En vous référant aux documentations officielles de la NeuroBoard S3 (« ESP32-S3 pinout »), et en précisant bien les noms des entrées/sorties, proposer deux schémas :

- la NeuroBoard S3, le potentiomètre et l'écran OLED ;
- le System on a Chip (SoC) ESP32-S3, le potentiomètre et l'écran OLED.

B. Software

(a) Installation des librairies

Afin d'utiliser l'écran OLED avec la NeuroBoard S3, nous aurons besoin de deux librairies : des fichiers dans lesquels se trouvent des fonctions permettant d'utiliser l'écran. Il s'agit des librairies SSD1306 et GFX d'Adafruit.

Pour ce faire, aller dans l'icône *Libraries* de PlatformIO. Dans la barre de recherche, entrer « `Adafruit_SSD1306` », cliquer dessus et cliquer sur « Add to Project ». Faire de même avec la seconde librairie en entrant « `Adafruit_GFX` » si celle-ci n'a pas été ajoutée automatiquement comme dépendance.

Pour vérifier que ces installations soient bien prises en compte, vous pouvez consulter la section `lib_deps` du fichier `platformio.ini`.

Voici un exemple de configuration dans le fichier `platformio.ini` illustrant l'intégration de ces bibliothèques :

```
[env:NeuroBoardS3]
platform = espressif32
board = NeuroBoardS3
framework = arduino
monitor_speed = 115200
lib_deps =
adafruit/Adafruit SSD1306 @ ^2.5.7
adafruit/Adafruit GFX Library @ ^1.11.5
```

Code 2.2 – Exemple de configuration platformio.ini pour ESP32-S3

(b) Affichage de texte

Le tableau suivant présente les principales fonctions permettant d'afficher du texte sur l'écran :

fonction	description
display.**clearDisplay()**	Éteindre tous les pixels
display.**drawPixel(x,y,color)**	Allumer un pixel aux coordonnées (x,y)
display.**setTextSize(n)**	Sélectionner la taille de police (entre 1 et 8)
display.**setCursor(x,y)**	Déplacer le curseur en coordonnées (x,y)
display.**Print("J'aime l'électronique")**	Afficher du texte aux coordonnées (x,y)
display.**display()**	Appeler cette fonction pour appliquer les changements / fonctions d'affichage

Dans un premier temps, nous allons afficher « Hello, world ! » sur l'écran à l'aide du code suivant :

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128 // Largeur de l'écran OLED, en pixels
#define SCREEN_HEIGHT 64 // Hauteur de l'écran OLED, en pixels

// Déclaration de l'écran connecté en I2C (pins SDA et SCL)
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

void setup() {
  Serial.begin(115200);
```

```
  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println(F("SSD1306 allocation failed"));
    for(;;);
  }
  delay(2000);
  display.clearDisplay();

  display.setTextSize(1);
  display.setTextColor(WHITE);
  display.setCursor(0, 10);
  display.println("Hello, world!");
  display.display();
}

void loop() {

}
```

Code 2.3 – Code NeuroBoard S3 pour afficher un texte sur l'écran OLED

Voyons comment ce code fonctionne.

On commence par inclure les librairies et à définir en variables globales les dimensions de l'écran :

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128 // Largeur de l'écran OLED, en pixels
#define SCREEN_HEIGHT 64 // Hauteur de l'écran OLED, en pixels
```

Les variables `SCREEN_WIDTH` et `SCREEN_HEIGHT` seront utilisables partout dans le code et permettront de rendre le code modulaire : en changeant d'écran, il suffit de changer les dimensions de ce dernier.

S'en suit l'initialisation de la communication I^2C entre la NeuroBoard S3 et l'écran OLED par le biais des broches GPIO dédiées (SDA = GPIO 8, SCL = GPIO 9). On parle ici de communication Maître - Esclave puisque c'est le microcontrôleur ESP32-S3 qui pilote l'écran qui répond à l'adresse `0x3C` en lui envoyant des ordres synchronisés sur l'horloge. Un message d'erreur est prévu de s'afficher dans le moniteur série (configuré à 115 200 bauds) si la communication ne s'initialise pas correctement.

```
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

void setup() {
  Serial.begin(115200);

  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println(F("SSD1306 allocation failed"));
    for(;;);
  }
```

Par la suite, on attend 2 000 ms (donc 2 secondes) afin de laisser le temps à l'écran de s'initialiser, et on efface l'écran à l'aide de la procédure `clearDisplay()` pour être certain de commencer avec un écran vide.

```
delay(2000);
display.clearDisplay();

display.setTextSize(1);
display.setTextColor(WHITE);
display.setCursor(0, 10);
display.println("Hello, world!");
display.display();
```

On choisit la taille de police en entrant en paramètre dans la procédure `setTextSize()` un nombre entre 1 et 8, et la couleur du texte à l'aide de `setTextColor()`. On place ensuite le curseur en coordonnées `(0,10)` et l'on affiche du texte à l'aide des procédures `println()` et `display()`.

E5. Modifier la taille de police afin d'afficher la tension en grande taille sur l'écran.

(c) Tracé de formes géométriques

Le tableau suivant présente les principales fonctions permettant d'afficher des formes géométriques à l'écran :

fonction	description
display.**drawPixel(64, 32, WHITE)**	Afficher un pixel aux coordonnées (64,32)
display.**drawLine(0, 0, 127, 20, WHITE)**	Dessiner une ligne du point (0,0) au point (127,20)
display.**drawRect(10, 10, 50, 30, WHITE)**	Dessiner un rectangle de points diagonalement opposées (10,10) et (50,30)
display.**drawRoundRect(10, 10, 20, 50, 2, WHITE)**	Dessiner un rectangle aux bords arrondis de points diagonalement opposées (10,10) et (50,30)
display.**fillRoundRect(10, 10, 20, 50, 2, WHITE)**	Dessiner un rectangle plein
display.**drawCircle(64, 32 ,10, WHITE)**	Dessiner un cercle de centre (64,32) et de diamètre 10
display.**fillCircle(64, 32, 10, WHITE)**	Dessiner un cercle plein
display.**drawTriangle(10, 10, 55, 20, 5, 40, WHITE)**	Dessiner un triangle de coins (10,10), (55,20) et (5,40)
display.**fillTriangle(10, 10, 55, 20, 5, 40, WHITE)**	Dessiner un triangle plein
display.**invertDisplay(true)**	Inverser les couleurs de l'écran

E6. Dessiner une jauge à l'écran proportionnelle à la tension entre 0 et 3,3 V, et afficher également la combinaison binaire sur 12 bits obtenue en sortie du convertisseur analogique - numérique.
Le rendu final devrait ressembler à ce qui est présenté sur la Figure 2.2.

Vous présenterez un algorigramme commenté, votre code sera commenté et expliqué (Comment obtient-on la valeur de la tension et de la combinaison binaire à afficher à l'écran ?), et vous présenterez des photos de votre montage que vous commenterez afin de démontrer que votre circuit fonctionne correctement.

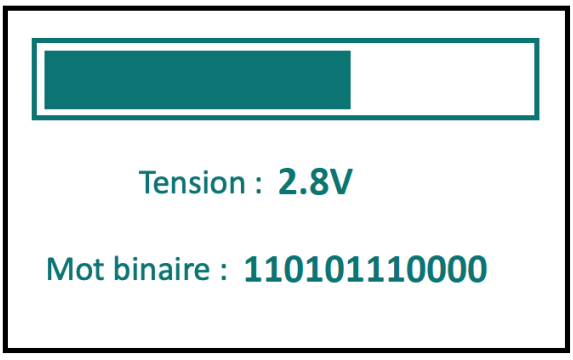


Figure 2.2 – Jauge de tension



L'essentiel

- Conversion analogique – numérique à l'aide de la fonction `analogRead()` qui fait appel à un convertisseur analogique-numérique interne au microcontrôleur. La fonction `analogRead()` retourne une valeur comprise entre 0 et 4095 proportionnelle au pourcentage de la tension de référence (3,3 V).
- Utilisation de l'écran OLED (protocole de communication I²C)

TP n° 3

La résistance

Motivation

Dans le premier TP, nous avons passé en revue les différents types de générateur de tension ($[U] = \text{V}$), permettant ainsi de mettre en mouvement des charges, ce qui se traduit par un flux d'électrons : un courant électrique ($[I] = \text{A}$). Une tension est par définition une différence de potentiel électrique, induite par une différence entre deux niveaux de charges électriques dans un circuit :

$$U_{AB} = \Delta V = V_A - V_B \tag{3.1}$$

Maintenant que nous savons générer une tension, voyons comment adapter la tension et le courant pour utiliser les composants électriques de manière adéquate en commençant par les résistances.

La **Loi d'Ohm** est une loi physique datant de 1827 qui permet de relier l'intensité du courant traversant un dipôle électrique à la tension entre ses bornes. Le lien entre les deux est sa résistance, c'est à dire son aptitude à s'opposer au passage du courant avec dissipation de chaleur. Nous allons voir qu'en branchant en série ou en parallèle des résistances, il est possible d'abaisser respectivement un courant ou une tension.

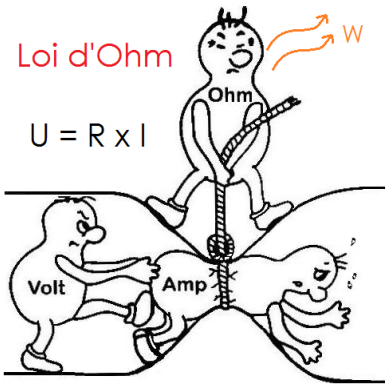


Figure 3.1 – Illustration de la loi d'Ohm par un dessin.
La résistance s'oppose au passage du courant en dissipant de l'énergie thermique (effet Joule)

L'objectif de ce TP est de mieux comprendre la notion de résistance, tant d'un point de vue physique que pratique, ainsi que deux de ses applications : abaisser un courant (résistance en série) et abaisser une tension (pont diviseur de tension).

I. Partie théorique

A. La résistance – notion physique

Un peu de physique... À une température donnée, la résistance d'un matériau filiforme homogène se calcule comme suit :

$$R = \rho \cdot \frac{\ell}{S} \tag{3.2}$$

où :

- R : la résistance en Ω ($[\mathrm{R}] = \mathrm{m}^2 \cdot \mathrm{kg} \cdot \mathrm{s}^{-3} \cdot \mathrm{A}^{-1}$) ;
- ρ : la résistivité en $\Omega \cdot \mathrm{m}$
  (Quelle est la capacité d'une barre d'1 m de matériau de section d'1 m^2 à s'opposer à la circulation du courant ?) ;
- ℓ : la longueur en m ;
- S : la section en m^2.

Remarque : d'un certain point de vue, la résistance ne dépend que de deux facteurs :

- les propriétés du matériau utilisé – la structure des atomes le constituant laisse plus ou moins bien passer le courant (ρ) ;
- sa géométrie ($\frac{\ell}{S}$).

À titre informatif, voici un tableau présentant la résistivité de certains matériaux conducteurs :

Matériau	**Résistivité ρ [$\Omega \cdot \mathrm{m}$]**
cuivre	17×10^{-9}
fer	100×10^{-9}
étain	120×10^{-9}
carbone	35×10^{-6}

T1. Établir l'équation reliant la longueur du fil à sa résistance pour un diamètre de fil d. On rappelle que l'aire d'un disque vaut πr^2.

T2. Combien doit mesurer un fil de carbone de 100 µm de diamètre pour être équivalent à une résistance de 100 Ω ?



On remarquera que n'importe quelle section de circuit ou que n'importe quel câble augmente la résistance totale d'un circuit, ce qui conduit à plus de pertes par effet Joule. Durant les TPs, nous négligerons les résistances des fils mais il est bon de garder ces pertes thermiques à l'esprit, notamment lorsque l'on travaille avec des courants ou des tensions plus élevées : plus un matériau est étroit et plus il chauffe.

B. La résistance – composant électrique

On distingue différents types de résistances. Les principaux sont les suivants :

- **les résistances de puissance** : elles permettent de produire de la chaleur ;
- **les résistances fixes** : elles permettent notamment de moduler le courant dans un circuit électrique ;
- **les résistances variables** : elles ont le même rôle que les résistances fixes mais ont un valeur qui peut être modifiée au cours du temps ;
- **certains capteurs** (de température, photorésistance, d'alcool, de force, etc.) qui dans la majorité des cas renvoient un courant qui dépend de la valeur de la grandeur physique mesurée. Ce courant est ensuite généralement converti en une tension.



(a) résistance fixe **(b)** résistance variable **(c)** potentiomètre **(d)** photorésistance

Figure 3.2 – Symboles des résistances fixe et variable, du potentiomètre et de la photorésistance

Nous n'étudierons dans ce TP que les résistances fixes.

Le code couleur d'une résistance permet de déterminer sa valeur à partir des couleurs de ses anneaux (4, 5 ou 6). Pour lire les couleurs dans le bon sens, il convient de placer l'anneau argenté ou doré à droite. Ensuite, la lecture se fait comme suit :

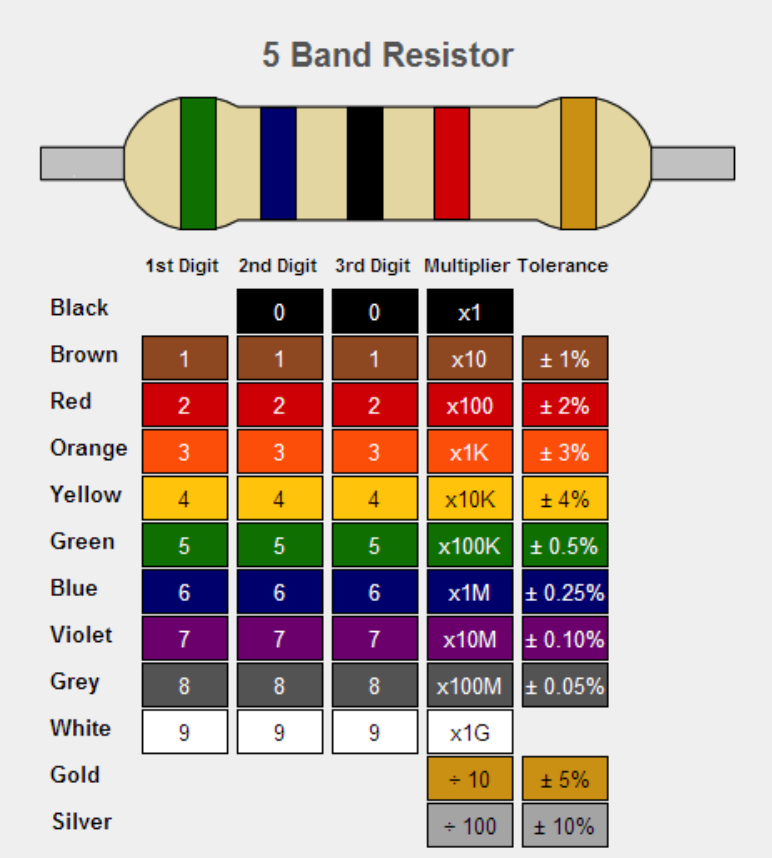


Figure 3.3 – Code couleur des résistances traversantes

Lorsqu'il y a plus de quatre anneaux (tronc vert ou bleu), les anneaux supplémentaires correspondent à des chiffres significatifs supplémentaires. Ce code couleur n'est pas à apprendre par cœur.

La **norme BS-1852** impose la notation des résistances en substituant Ω par « R », kΩ par « K » et MΩ par « M ». Ainsi en commandant des résistances sur internet, il conviendra par exemple de chercher « 2K2 » pour une résistance de 2,2 kΩ.

T3. Quelles sont les couleurs des quatre premières bagues des résistances $R_1 = 10\,\Omega$, $R_2 = 45\,\Omega$, $R_3 = 100\,\Omega$ et $R_4 = 1\,\text{k}\Omega$?

Une résistance se choisit selon principalement trois critères :

- sa **valeur** en Ω. Typiquement toutes les valeurs n'existent pas donc on cherchera la valeur normalisée la plus proche ;
- sa **précision** ou tolérance, qui traduit un pourcentage d'erreur sur sa valeur ;
- sa **puissance maximale** : selon la taille de la résistance, elle est capable de supporter une certaine puissance maximale.

T4. Identifier les valeurs de toutes les résistances du kit.

T5. Sachant que les résistance de votre kit sont toutes des résistances pouvant supporter un 0,5 W, calculer le courant maximal que chacunes d'entre elles peuvent supporter. On rappelle que la puissance se calcule comme suit :

$$P = U \cdot I \tag{3.3}$$

C. Association de résistances

En série, les valeurs de résistances se somment :

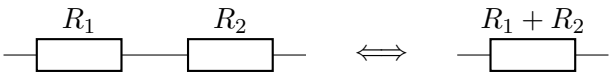


Figure 3.4 – Association de résistances en série

En parallèle (appelé aussi dérivation), on somme les inverses :

$$\frac{1}{R_{eq}} = \frac{1}{R_1} + \frac{1}{R_2} \iff R_{eq} = \frac{R_1 R_2}{R_1 + R_2} \tag{3.4}$$

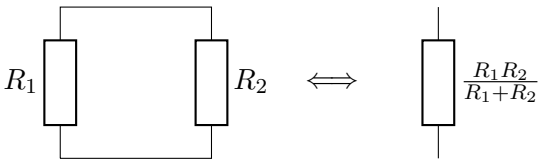


Figure 3.5 – Association de résistances en parallèle

D. Pont diviseur de tension

On considère le montage suivant :

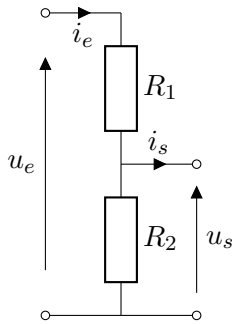


Figure 3.6 – Pont diviseur de tension

Si le courant en sortie est négligeable ($i_s \ll i_e$), alors on peut écrire par la loi d'Ohm :

- $u_e = (R_1 + R_2)i_e$;
- $u_s = R_2 i_e$.

Par substitution de i_e, on obtient la formule du **pont diviseur de tension** :

$$u_s = \frac{R_2}{R_1 + R_2} u_e \tag{3.5}$$

T6. Si u_e vaut 5 V et R_1 vaut 100 Ω, quelle valeur de résistance pour R_2 faut-il choisir afin d'avoir une tension en sortie de 4 V ? 2,5 V ? 1 V ?

Remarquez qu'un pont diviseur de tension permet de **diminuer** ($\frac{R_2}{R_1+R_2} < 1$) une tension par dissipation thermique en utilisant deux résistances. Il ne permet pas de l'augmenter. Remarquez aussi qu'une augmentation de la résistance induit une perte de courant (loi d'Ohm).

E. Principe de superposition

Le **principe de superposition** découle directement de la linéarité des équations de Kirchhoff : un dipôle constitué de dipôles linéaires est un dipôle linéaire.
Il stipule que l'intensité du courant dans une branche d'un réseau comprenant plusieurs générateurs est la somme des intensités, que ferait passer, dans cette branche, chaque générateur considéré isolément comme actif, les autres générateurs du réseau étant alors passifs.

NB : Pour rendre passif un générateur, il faut le remplacer par sa résistance interne. Dans ce TP, nous étudierons le cas idéal en remplaçant les générateurs par des fils.

Soit le circuit suivant :

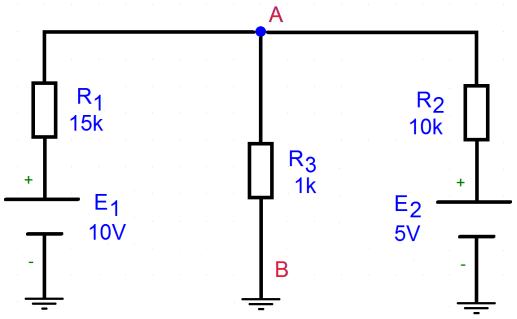


FIGURE 3.7 – Exemple de circuit à deux générateurs de tension

Le **théorème de superposition** permet de sommer les contributions de différentes sources de tensions en les considérant de manière distincte et isolément active et en court-circuitant toutes les autres sources au moment où on les prend en compte.

T7. Exprimer la tension U_{AB} puis faire l'application numérique pour $E_1 = 10\,\text{V}$, $E_2 = 5\,\text{V}$, $R_1 = 15\,\text{k}\Omega$, $R_2 = 10\,\text{k}\Omega$ et $R_3 = 1\,\text{k}\Omega$.

Commencez par exprimer U_{AB} en faisant la somme de U'_{AB} et de U''_{AB} comme suit :

$$U_{\text{AB}} = \underbrace{U'_{\text{AB}}}_{\text{E1 remplacé par un fil}} + \underbrace{U''_{\text{AB}}}_{\text{E2 remplacé par un fil}} \tag{3.6}$$

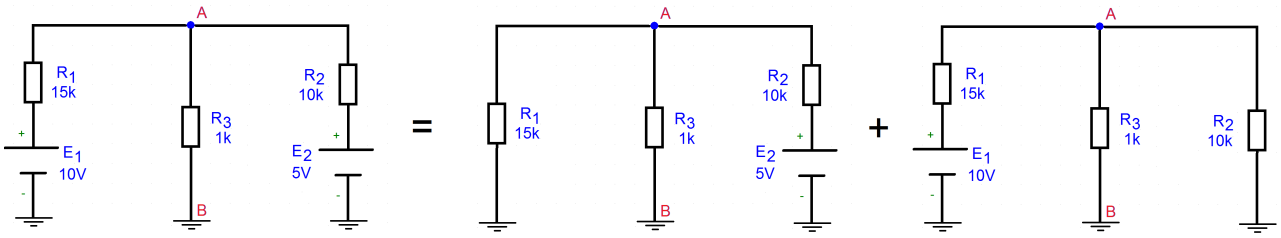


FIGURE 3.8 – Application du théorème de superposition au circuit

T8. En déduire l'intensité I traversant la résistance R_3.

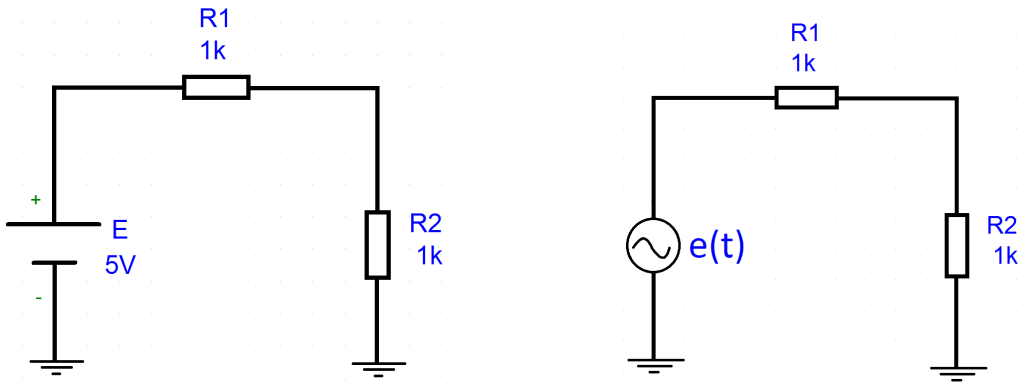
II. Simulations

Nous allons à nouveau travailler avec notre cher logiciel de simulations Proteus.

A. Pont diviseur de tension en régime alternatif

On considère les résistances comme étant idéales (loi d'Ohm vérifiée), nous permettant ainsi d'appliquer le principe de superposition.

E1. Sous Proteus, mesurer la tension en sortie d'un pont diviseur de tension avec $U_e = 5\,\text{V}$, $R_1 = 1\,\text{k}\Omega$ et $R_2 = 1\,\text{k}\Omega$.
Attention, on vous rappelle qu'une tension est une différence de potentiel. Ainsi, si vous branchez vos sondes à la masse, votre tension sera nulle. Identifiez bien les différents potentiels et différentes tensions sur votre circuit. Si vous avez un doute, demandez à votre chargé de TP.



(a) tension continue **(b)** tension sinusoïdale

Figure 3.9 – Schéma du montage « Pont diviseur de tension »

E2. Remplacer le générateur de tension continue par un générateur de fonction sinusoïdale (SINE) d'amplitude 5 V de fréquence $f = 1\,\text{Hz}$. Observer le signal sur deux périodes à l'aide d'une simulation Analogue Analysis (Graphs > Analogue).

E3. Remplacer R_1 par 5 valeurs d'ordre de grandeur (puissance de 10) différents de façon à avoir 5 courbes distinctes (copiez-collez le circuit en utilisant« **Copy block** » et reliez les 5 sondes de tension à votre simulation Analogue Analysis afin d'afficher simultanémet les 5 courbes) Conclure.

En régime sinusoïdal, le pont diviseur de tension modifie l'amplitude du signal : $u_s = \frac{R_2}{R_1+R_2} \cdot u_e = \frac{R_2}{R_1+R_2} \cdot A sin(\omega t + \varphi)$ mais ne modifie pas ses caractéristiques temporelles.

B. Théorème de superposition

On cherche à présent à simuler sur Proteus le montage de la partie « Théorème de superposition » avec $E_1 = 10\,\text{V}$, $E_2 = 5\,\text{V}$, $R_1 = 15\,\text{k}\Omega$, $R_2 = 10\,\text{k}\Omega$ et $R_3 = 1\,\text{k}\Omega$.

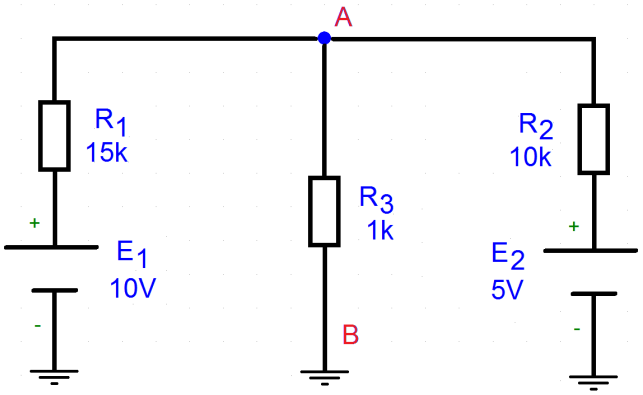


Figure 3.10 – Schéma du montage « théorème de superposition »

E4. Mesurer U_{AB} et I_{R3}.

E5. Mesurer les tensions et les courants aux différents endroits du circuit. Vérifier la loi des nœuds et trouver trois manières de vérifier la loi des mailles.

III. Partie pratique

A. Prise en main du GBF et de l'oscilloscope

E6. Générer un signal sinusoïdal d'amplitude 5 V, de fréquence 10 kHz, observer le signal sur deux périodes à l'aide d'un oscilloscope.

Pour cela, suivre les étapes suivantes :

1. **branchement du circuit** ;
2. **branchement du signal d'entrée** en se branchant sur la sortie « OUTPUT 50 Ω ».
   Afin de générer un signal sinusoïdal, vous utiliserez un GBF et vous choisirez le signal sinusoïdal en appuyant une ou plusieurs fois sur le bouton « WAVE ». Le nombre que vous entrerez puis validerez à l'aide du bouton « Hz », « kHz » ou « MHz » correspondra à la fréquence de votre signal. Enfin, la tension à l'état haut se règle à l'aide du bouton « AMPLITUDE » ;
3. **branchement de l'appareil de mesure (oscilloscope)** : se brancher sur « CHANNEL 1 » pour mesurer la tension délivrée par le GBF ;
4. **calibration de l'oscilloscope et observation des signaux** : appuyer dans un premier temps sur « AUTO SET » et changer les calibres horizontal (base de temps : bouton rotatif sur la droite) et vertical (Tension : bouton rotatif associé à « CHANNEL 1 »).

Prenez une capture d'écran et exportez la à l'aide d'une clé USB de faible mémoire (sinon elle sera considérée comme un disque dure et non reconnue).

Ajoutez un OFFSET de 2 V. Pour ce faire, tirer le bouton « OFFSET » et tournez le afin de donner une valeur de composante continue au signal.

E7. Quel est l'effet du bouton de choix de couplage AC / DC / GND ? Prendre des captures d'écran pour chacun de ces couplages et commenter les courbes.

E8. Quelle est l'équation de ce signal ?

E9. À partir de la capture d'écran, déterminer la fréquence et l'amplitude du signal. Justifier à l'aide de calculs simples.

B. Pont diviseur de tension

E10. Trouver un moyen de diviser la tension par un facteur quatre en utilisant un pont diviseur de tension. (Vous pouvez faire des tests au préalable sur Proteus en tenant compte des valeurs des résistances présentes dans votre kit.)

Vérifier que cela fonctionne en testant le montage avec l'oscilloscope. La tension délivrée par le générateur est affichée sur CHANNEL 1 et la sortie du pont diviseur de tension sur CHANNEL 2.

BONUS. Se documenter sur le potentiomètre et le brancher de telle sorte à ce que lorsque l'on tourne le bouton, on module l'amplitude du signal. Vérifier cela à l'oscilloscope.



L'essentiel

- La résistance s'oppose au passage des électrons, donc au courant, et émet de la chaleur par effet Joule
- Elle dépend du matériau utilisé et de sa géométrie : $R = \rho \frac{l}{s}$
- **Pont diviseur de tension : $u = \frac{R_2}{R_1 + R_2} u_e$**
- Principe de superposition

TP n° 4

Le condensateur

Motivation

Nous avons étudié au cours des TPs précédents les générateurs de tension et la résistance qui, avec le condensateur et la bobine sont trois **composants passifs**, c'est à dire qui ne permettent pas d'augmenter la puissance d'un signal contrairement aux **composants actifs** que sont les semi-conducteurs (diode, transistor, etc.) et qui seront étudiés en ING2.

À l'aide de résistances, on peut moduler la tension (pont diviseur de tension) et le courant (loi d'Ohm), ce qui permet d'alimenter correctement des composants plus complexes et/ou d'adapter un signal.

Nous allons dans ce TP étudier la charge et la décharge du circuit RC et continuer à nous familiariser avec le matériel du laboratoire d'électronique. Des courbes de charge et de décharge typiques de condensateurs (voir **circuit RC**) sont représentées ci-après :

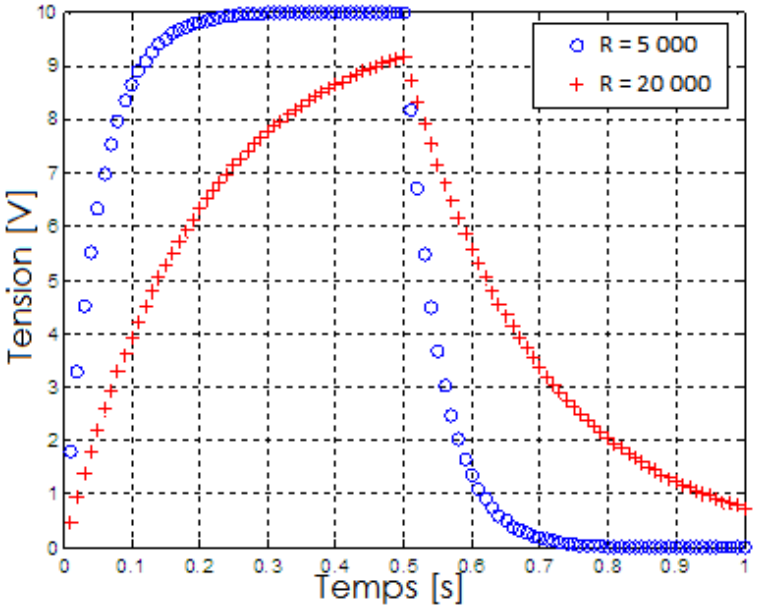


Figure 4.1 – Charge et décharge d'un circuit RC alimenté par un signal rectangulaire identique mais pour deux valeurs de résistance différentes

Au cours de ce semestre, nous ne ferons que l'analyse temporelle ($u = f(t)$). Le filtrage analogique (analyse fréquentielle) sera traité au second semestre.

I. Partie théorique

A. Le condensateur

(a) Origine physique

Le condensateur est un composant électronique élémentaire qui est constitué de **deux armatures conductrices** séparées par un isolant polarisable : **le diélectrique**. Il permet de stocker de l'énergie sous la forme d'un champ électrostatique. Le condensateur est caractérisé par sa **capacité en Farad F** : la quantité de charges électriques qu'il peut porter pour un potentiel électrique donné ($C = \frac{q}{U}$). Cette capacité dépend de la géométrie des armatures, de l'espace entre ces dernières et de la nature de l'isolant :

$$C = \varepsilon_0 \varepsilon_r \cdot \frac{S}{d} \tag{4.1}$$

avec :

- ε_r la permittivité relative du milieu (1 pour l'air et le vide) ;
- ε_0 la permittivité du vide ($8{,}84 \times 10^{-12}\ \mathrm{A^2 \cdot s^4 \cdot kg^{-1} \cdot m^{-3}}$)
- S la surface des armatures ;
- d la distance séparant les deux armatures.

L'énergie stockée par un condensateur s'écrit :

$$E = \frac{1}{2} C U^2 = \frac{q^2}{2C} \tag{4.2}$$

(b) Composant électrique

Il existe plusieurs types de condensateurs qui emploient différentes technologies de fabrications et matériaux, ce qui influe principalement sur le coût, la taille et la précision du composant. On retiendra que les condensateurs au tantale sont plus onéreux mais prennent moins de place que les condensateur à l'aluminium. Pour des circuits miniatures, on peut également choisir des **composants montés en surface (CMS)**.

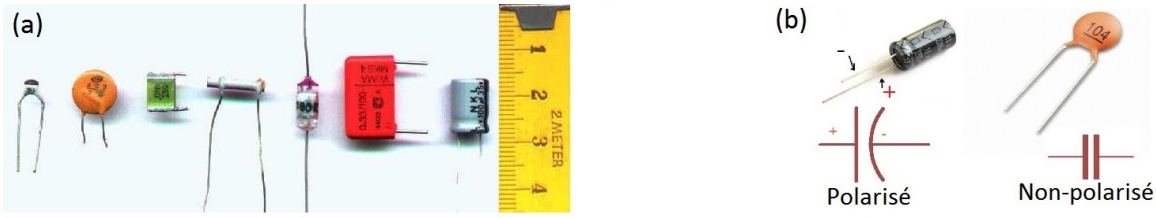


Figure 4.2 – **(a)** Plusieurs types de condensateurs. De gauche à droite : céramique multicouche, céramique disque, film polyester multicouche, céramique tubulaire, polystyrène, film polyester métallisé, électrolytique aluminium. **(b)** Schéma électrique et composant électronique pour des condensateurs polarisés et non-polarisés

Un condensateur se choisit en fonction de la capacité qu'il doit avoir pour répondre à une fonction dans un circuit mais aussi en fonction de la **tension de service**, c'est à dire la tension maximale/ **tension de claquage** pour ne pas que le diéléctrique devienne conducteur, ce qui endommagerait le composant. On veillera donc à choisir un composant à tension de service supérieure à celle du circuit tout en gardant à l'esprit qu'en règle générale, plus la tension de service est élevée, plus le composant est onéreux et plus il vieillit vite.

Certains condensateurs sont polarisés (capacité supérieure à 1 µF en général) alors que d'autres ne le sont pas. Il faut donc veiller à les brancher dans le bon sens, sens qui est soit mentionné par un signe « + » ou une colerette pour désigner le côté du composant qui recevra le potentiel électrique le plus élevé ou soit par une ligne de flèches marquées d'un signe « - » qui désigne le côté à potentiel le moins élevé (souvent la masse). Une autre règle plus générale : la patte la plus longue d'un composant désigne très souvent la patte « + ». Deux condensateurs polarisés en série dont les côtés « - » se font face sont équivalent à un condensateur non-polarisé de capacité $\frac{C_1+C_2}{4}$.

(c) Condensateur de découplage

La relation caractéristique d'un condensateur idéal est :

$$i = C \cdot \frac{du}{dt} \tag{4.3}$$

On remarque ainsi qu'un condensateur alimenté en courant continu n'est autre qu'un interrupteur ouvert car le courant le traversant est nul : $I = C \cdot \frac{dU}{dt} = 0$

On remarque également qu'aux hautes fréquences, le condensateur se comporte comme un fil : $i = C \cdot \frac{du}{dt} = C \cdot \frac{d}{dt}\left(Asin(\omega t) + V_{\text{OFF}}\right) = CA\omega cos(\omega t) = CA \cdot 2\pi \cdot f \cdot cos(\omega t)$

En utilisant cette propriété, on utilise des **condensateurs de découplage** pour évacuer le bruit à la masse : des condensateurs reliés à la masse permettent d'évacuer les composantes de haute fréquence (souvent du bruit) d'un signal. En théorie, il faudrait en mettre partout dans les circuit électriques mais il faut également réfléchir à l'encombrement et au coût des composants en réalisant un circuit.

En fonction de la fréquence de travail du circuit, on choisit différents matériaux et des valeurs typiques pour les condensateurs de découplage. Cela sera expliqué plus en détail lors du second semestre lors du cours sur le filtrage analogique.

B. Le circuit RC

Le circuit RC classique est un circuit électrique composé d'une résistance et d'un condensateur montés en série.

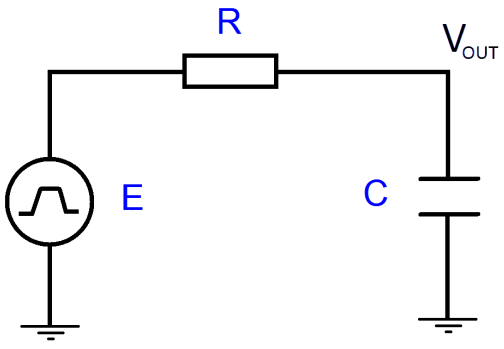


Figure 4.3 – Cchéma du montage « dipôle RC »

Dans cette partie, nous allons étudier la charge et la décharge d'un condensateur, ce qui revient à prédire comment se comporte la tension aux bornes du condensateur u_c en fonction du temps.

Étudier la charge d'un condensateur revient donc à trouver la fonction $u_c(t)$ décrivant le remplissage du condensateur par des électrons. La tension $u_c(t)$ vaut donc une certaine tension au début de la charge (0 si le condensateur est déchargé) et vaut la tension d'entrée U_0 une fois le condensateur chargé.

T1. Reproduire le schéma électrique du circuit RC en série et flécher les tensions et le courant en convention récepteur.

NB : En convention récepteur, contrairement au cas de la convention générateur, on se place du point de vue des composants et le courant traverse les composants dans le sens des potentiels décroissants : U et I sont de sens opposés.

(a) Charge du condensateur

T2. En appliquant la **loi des mailles**, montrer que l'équation différentielle qui régit l'évolution de la tension u_c dan le cas d'une alimentation constante s'écrit :

$$E = u_C + RC \cdot \frac{du_c}{dt} \tag{4.4}$$

T3. Vérifier que la solution de l'équation différentielle est de la forme $u_c = U_0 \cdot (1 - e^{-t/\tau})$. Donner les expressions de U_0 et de τ. Pour cela, partez de l'expression de u_c et dérivez là afin de remplacer u_c et $\frac{du_c}{dt}$ dans l'équation 4.4.

Plus la constante de temps τ =RC est grande, plus le condensateur met du temps à se charger. À résistance constante, cela est intuitif car cela veut dire que plus C est grand, plus il met du temps à se charger. Or, par définition, C=$\frac{Q}{\Delta V}$: il s'agit de la charge qu'il peut contenir à une tension donnée, à la manière d'un seau d'eau que l'on remplirait avec des électrons.

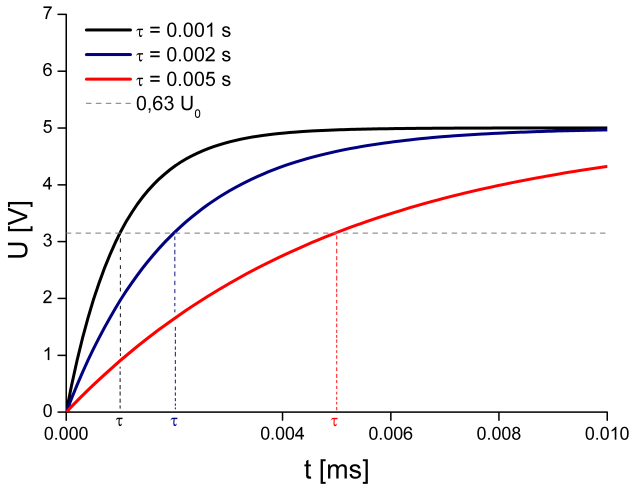


Figure 4.4 – Courbe de charge u_c de condensateurs pour trois constantes de temps différentes.

T4. Comment s'appelle la grandeur τ ? Par analyse dimensionnelle, déterminer son unité.

T5. Quelle est la valeur de u_c à la date $t = \tau$?

(b) Décharge du condensateur

Par un raisonnement analogue, on peut montrer que la décharge du condensateur est régie par l'équation différentielle :

$$\frac{du_c}{dt} + \frac{1}{RC} \cdot U_c = 0 \tag{4.5}$$

dont la solution est :

$$u_c = E \cdot e^{\frac{-t}{RC}} \tag{4.6}$$

T6. À l'aide d'un tableur, représenter la courbe de décharge du condensateur alimenté en $E = 5\,\text{V}$ sur une période de 7τ (On considère que l'on atteint le régime permanent au bout de 5τ) dans un circuit RC où $R = 10\,\Omega$ et $C = 0{,}1\,\mu\text{F}$ en calculant 10 valeurs avec le tableur.

II. Simulations

E1. Simuler le montage suivant ($R = 1\,\text{k}\Omega$ et $C = 47\,\text{nF}$) en alimentant votre circuit à l'aide d'un signal continu d'amplitude 5 volt. Qu'observe t-on ? (deux conclusions : une sur le courant, une sur la tension aux bornes de C).

E2. Simuler le même montage en alimentant cette fois-ci votre circuit à l'aide d'un signal rectangulaire d'amplitude 5 V crête à crête, oscillant entre 0 et 5 V, de rapport cyclique 0,5 et de fréquence 1 kHz. Faire une copie d'écran et la commenter.

E3. Comparer les valeurs théoriques (expression théorique de u_c donnée à la question **T3.**) avec vos données pratiques pour $t_1 = 15\,\mu\text{s}$ et $t_2 = 150\,\mu\text{s}$. D'où peut provenir l'erreur ?

E4. Remplacer le condensateur de 47 nF par un condensateur de 1 µF et visualiser les signaux d'entrée et de sortie sur deux périodes. Commenter les résultats.

E5. Que se passe t-il lorsque l'on augmente la fréquence à 10 kHz ? Lorsque l'on abaisse le rapport cyclique ? Expliquer clairement l'allure des courbes.

III. Le capacimètre

On souhaite concevoir un capacimètre basé sur une carte Arduino Nano.

T7. Pour le circuit ci-dessous, proposer un algorigramme permettant de mesurer la capacité d'un condensateur inconnu.

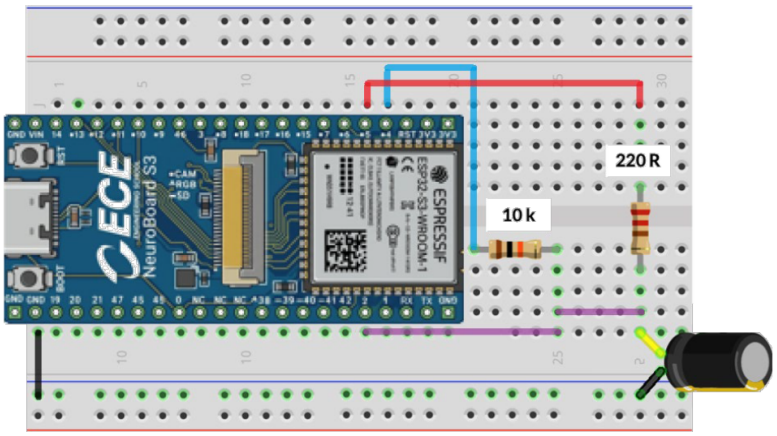


Figure 4.5 – Circuit permettant de mesurer une capacité à l'aide d'une NeuroBoard S3

E6. Tester le Code 4.1 (également disponible sur la page Boostcamp) en mesurant les capacités des condensateurs du kit. Estimer et discuter l'erreur (mesure et fabrication).

```
#include <NeuroBoardS3.h>
const int analogPin = 2;       // GPIO2 : Entrée analogique pour mesurer la tension du
condensateur
const int chargePin = 4;       // GPIO4 : Broche numérique pour charger le condensateur
const int dischargePin = 5;    // GPIO5 : Broche numérique pour décharger le condensateur
const float resistorValue = 9912.0; // Valeur réelle de la résistance mesurée au multimètre
(en Ohms)
const float adcCalibrationFactor = 4.07; // Facteur de compensation d'impédance du CAN de l'
ESP32-S3
unsigned long startTime;
unsigned long elapsedTime;
float microFarads;

void setup() {
  Serial.begin(115200);

  // Configuration spécifique de l'ADC pour l'ESP32-S3
  analogReadResolution(12);        // Résolution 12 bits (0 - 4095)
  analogSetAttenuation(ADC_11db);  // Atténuation 11 dB pour couvrir la plage de 0V à 3.3V

  pinMode(chargePin, OUTPUT);
  digitalWrite(chargePin, LOW);
  pinMode(dischargePin, INPUT);
}
void loop() {
  // 1. Décharge complète du condensateur pendant 500 ms
  pinMode(dischargePin, OUTPUT);
  digitalWrite(dischargePin, LOW);
  digitalWrite(chargePin, LOW);
  delay(500);
  pinMode(dischargePin, INPUT); // Libération de la broche de décharge

  // 2. Démarrage de la charge et démarrage du chronomètre
  digitalWrite(chargePin, HIGH);
  startTime = micros();

  // 3. Attente jusqu'à atteindre le seuil de 63.2% (2588)
  int adcVal = 0;
  do {
    adcVal = analogRead(analogPin);
  } while (adcVal < 2588);
    elapsedTime = micros() - startTime;
    // 4. Arrêt de la charge
  digitalWrite(chargePin, LOW);
  // 5. Calcul de la capacité : C = (tau / R) * Facteur de calibration
  microFarads = ((float)elapsedTime / resistorValue) * adcCalibrationFactor;
  // Affichage du temps réel en ms et de la valeur en uF
  Serial.print("Temps : ");
  Serial.print(elapsedTime / 1000.0, 2);
  Serial.print(" ms | Capacite : ");
  Serial.print(microFarads, 2);
  Serial.println(" microFarads");
  delay(1000);
}
```

Code 4.1 – Code Arduino de capacimètre

L'essentiel

- Un condensateur est un composant capable d'emmagasiner des charges électriques.
- Le matériau est la structure du condensateur influe sur ses caractéristiques électriques.
- Les caractéristiques principales d'un condensateur sont :
  - sa **capacité** en F ;
  - sa **tension de service** en V.
- Un condensateur agit comme :
  - un circuit ouvert en courant continu (fréquence nulle) ;
  - un fil en hautes fréquences.
- Il est souvent utilisé pour filtrer les hautes fréquences dans un circuit électronique (**condensateur de découplage**).
- La charge d'un condensateur à travers une résistance (circuit RC) suit une fonction exponentielle : $u_c = U_0 \cdot (1 - e^{-t/\tau})$

TP n° 5

L'amplificateur opérationnel

Motivation

L'**amplificateur opérationnel** (AOP) est un composant électronique qui permet d'amplifier une différence de potentiel, et d'appliquer des **opérations mathématiques de base** à des signaux analogiques (addition, soustraction, multiplication, inversion de signe, multiplication par une constante, intégration, dérivation, etc.) mais aussi de comparer deux signaux. Il est intrinsèquement constitué de transistors mais nous le considérons à un niveau d'abstraction supérieur c'est à dire comme un circuit intégré à minimum cinq pattes comprenant **2 broches d'alimentation** du composant nommées V_{DD} et V_{SS} (ou V_{CC} et V_{EE}), **1 entrée inverseuse** V^-, **1 entrée non-inverseuse** V^+ et **1 sortie** V_{OUT}. Pour ne pas surcharger les schémas, les broches d'alimentation V_{CC} et V_{EE} ne sont pas toujours représentées et en l'absence d'information à leur sujet, elles valent respectivement -15 et $+15$ V.

Comme pour tout circuit intégré, la sortie de l'AOP ne peut jamais être à un potentiel supérieur à son alimentation.

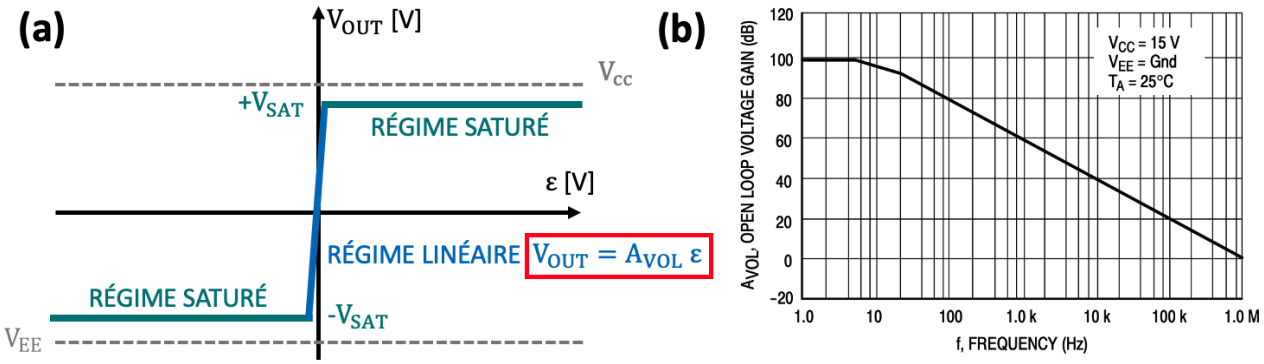


Figure 5.1 – **(a)** Caractéristique générale de l'AOP et **(b)** Facteur d'amplification A_V en fonction de la fréquence du signal d'entrée du LM358.

D'un point de vue pratique, on considère que l'AOP est en régime linéaire (point de fonctionnement dans la zone linéaire de sa caractéristique) s'il y a une **contre-réaction** sur l'entrée inverseuse.
Cela signifie que tout ou partie de la tension de sortie V_{OUT} est ramenée sur l'entrée V^-, via un fil ou des composants (résistances, condensateurs, ...), ce qui a pour effet de limiter le gain du montage.
On considérera alors que $V^+ = V^-$ et on pourra chercher une expression entre la sortie et l'entrée.

L'objectif de ce TP est de relever la caractéristique d'un AOP et de simuler le comportement de montages classiques à base d'AOPs en régime linéaire.

I. Partie théorique

Dans la majorité des cas, l'AOP est présenté en boitier à 8 pattes DIL (Dual In Line) ou PDIP (Plastic Dual In Line Package). En fonction du composant, l'alimentation doit être nécessairement symétrique ou non et en fonction des schémas, elle peut ne pas être dessinée par souci d'ergonomie. Le composant est alors supposé comme étant alimenté par $+V_{CC} = +\ 15\,\text{V}$ et $-V_{CC} = -15\,\text{V}$.

Le brochage d'un composant se détermine en lisant sa datasheet. La patte numéro 1 se trouve en haut à gauche du composant lorsque l'on a le petit rond en haut à gauche. Puis, le numéro des pins suit l'ordre inverse des aiguilles d'une montre.

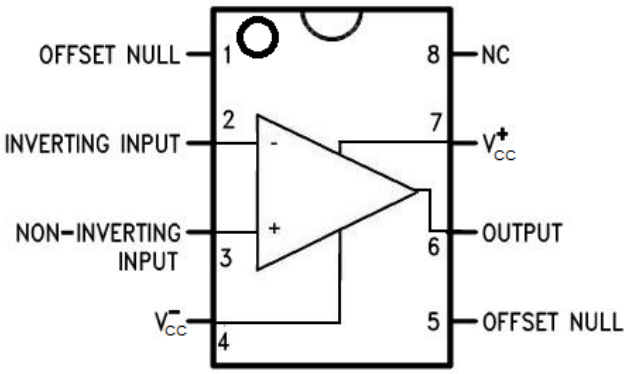


Figure 5.2 – Exemple de schéma de cablage d'un AOP

On parle de **montage inverseur** si l'entrée du montage est appliquée, directement ou non, sur l'entrée inverseuse de l'AOP, ce qui a pour effet d'**inverser le signe** de la tension d'entrée.
A contrario, on parle de **montage non inverseur** si l'entrée du montage est appliquée, directement ou non, sur l'entrée non inverseuse de l'AOP. Dans ce cas, la tension de sortie a **le même signe** que la tension d'entrée.

A. Lecture d'une datasheet

T1. Consulter la datasheet de l'OPA 454, reproduire le schéma de branchement de la section *Pin Configuration and Functions* et expliciter le rôle de chacune des pins.

T2. La plupart des AOP ont besoin d'être alimenté par une alimentation symétrique. Est-ce le cas de cet AOP ? Même question pour le LM358, justifier. Quel peut-être l'avantage d'avoir une alimentation asymétrique ?

B. Montages amplificateur

La Figure 5.3 présente les montages amplificateur inverseur et non-inverseur où, dans les deux cas, $u_s = G \cdot u_e$. Le montage inverseur permet entre autre d'amplifier ou d'atténuer un signal alors que l'amplificateur non-inverseur ne peut qu'amplifier un signal puisque $G > 1$ dans ce dernier cas.

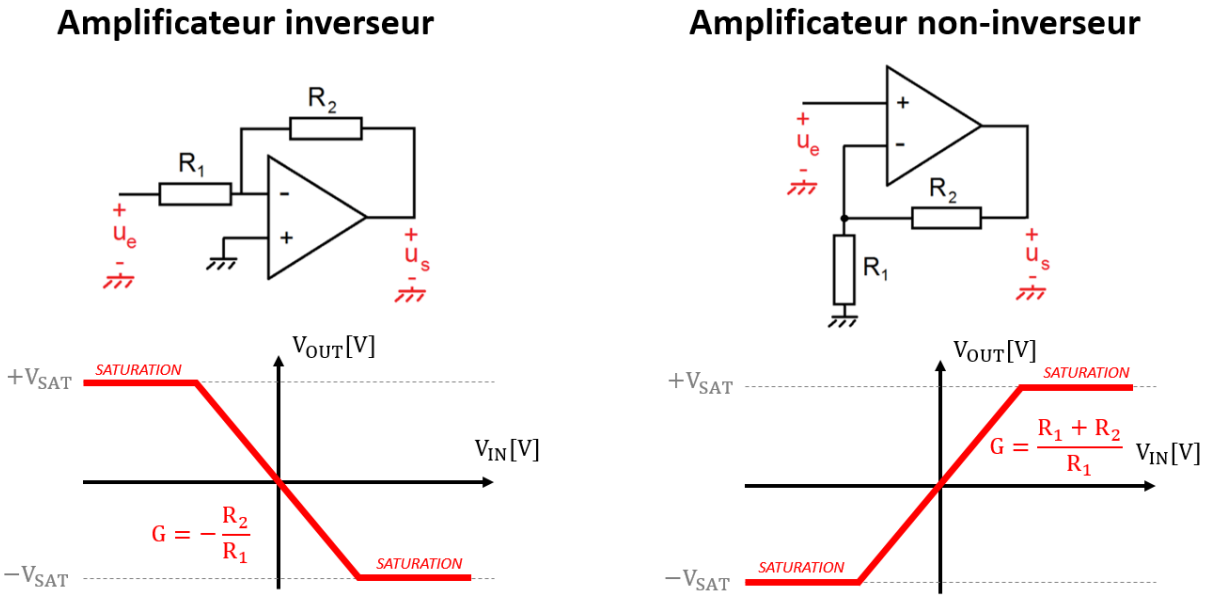


Figure 5.3 – Montages amplificateur inverseur et non-inverseur.

Soit le schéma suivant :

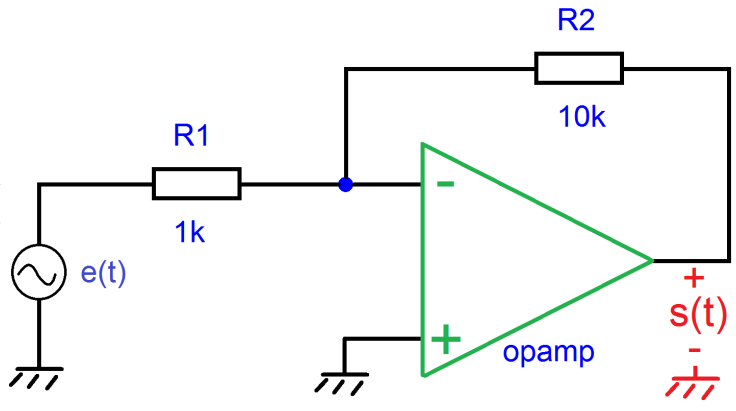


Figure 5.4 – Montage amplificateur inverseur

T3. Donner l'expression de la tension de sortie s(t) en fonction de e(t) et déterminer le gain.

T4. Quelle est la fonction réalisée par ce montage ?

T5. De quoi parle t-on lorsque l'on parle de fonctionnement linéaire et de fonctionnement en saturation ?

T6. Que vaut le gain de cet amplificateur si l'on remplace les résistances par des fils ?

C. Montage intégrateur

On considère le montage suivant qui est communément appelé « montage intégrateur ».

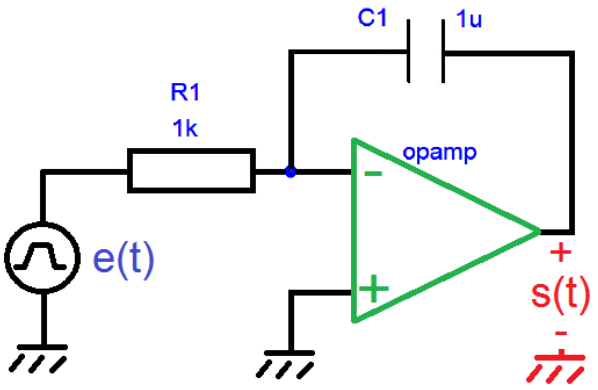


Figure 5.5 – Montage intégrateur

T7. Déterminer l'expression de la tension de sortie s(t) en fonction de la tension d'entrée e(t). Tracer sur un même graphique l'allure de e(t) et de s(t) sur deux périodes si e(t) est un signal carré variant entre $-5\,\mathrm{V}$ et $+5\,\mathrm{V}$ à $10\,\mathrm{kHz}$.

II. Simulations

A. Suiveur de tension

E1. Simuler un pont diviseur de tension sur Proteus à l'aide d'un DC VOLTMETER comme suit ($E = 5\,\mathrm{V}$, $R_1 = R_2 = 10\,\mathrm{k\Omega}$) comme sur la Figure 5.6 (a), mais sans la résistance R. Déterminer théoriquement et mesurer la tension en sortie du pont.

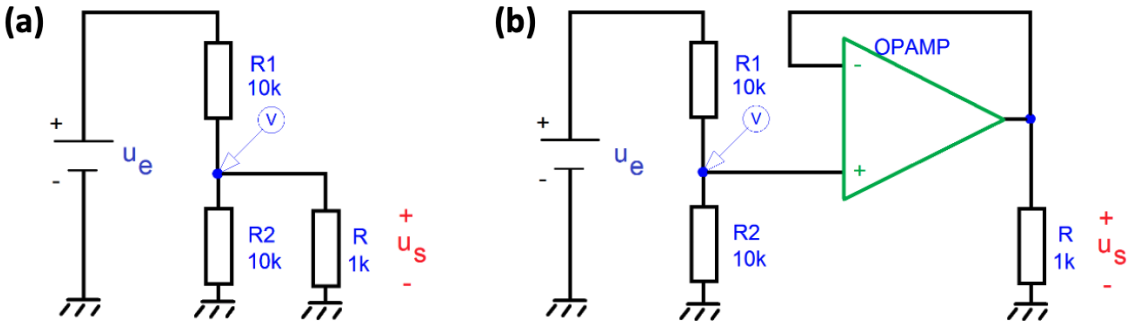


Figure 5.6 – Pont diviseur de tension **(a)** sans suiveur et **(b)** avec suiveur

E2. Ajouter la résistance R et mesurer la tension en sortie du pont. Que s'est-il passé ?

E3. Ajouter un AOP monté en suiveur non-inverseur comme sur la figure 5.6 (b) et mesurer la tension aux bornes de R. Conclure sur l'utilité du suiveur.

B. Montage amplificateur inverseur

On considère le montage suivant :

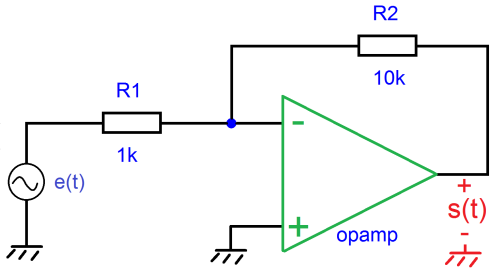


Figure 5.7 – Montage amplificateur inverseur

E4. Simuler le circuit de la figure 5.7 avec un DC VOLTMETER et comparer ces résultats avec votre partie théorique. Vous prendrez par exemple un générateur de tension continue d'amplitude 5 V.

On considère dorénavant le même montage mais avec $R_1 = 10\,\text{k}\Omega$ et $R_2 = 47\,\text{k}\Omega$, alimenté par une tension d'entrée constante E.

E5. À l'aide de la fonction DC SWEEP, tracer la caractéristique de l'AOP (la tension de sortie en fonction de la tension d'entrée) puis renseigner les domaines de fonctionnement linéaire et saturé dans votre rapport. Commenter la tension des deux plateaux et la pente de la caractéristique dans sa zone linéaire.

E6. Remplacer le générateur de tension continue par un générateur de basse fréquence (VSIN) d'amplitude 10 V et de fréquence 10 Hz. Observer le signal de sortie et commenter le résultat obtenu.

E7. Proposer un montage pour que le signal de sortie ne soit pas inversé en utilisant deux AOP.

C. Montage intégrateur inverseur

On considère le montage intégrateur inverseur étudié dans la partie théorique que revoici :

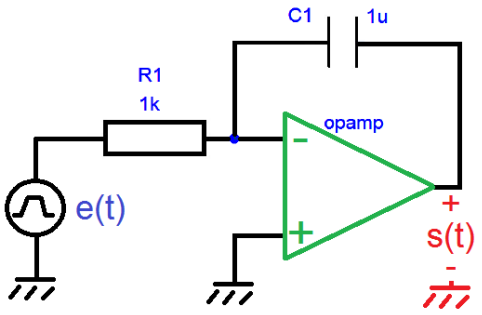


Figure 5.8 – montage intégrateur

E8. Appliquer à l'entrée un signal carré symétrique d'amplitude ±5 V et de fréquence 1 Hz. Visualiser le signal de sortie (ANALOGUE ANALYSIS) et commenter chaque portion de votre signal à l'aide de votre partie théorique.

NB : Pour obtenir de meilleurs résultats, on peut ajouter une résistance de 10 kΩ en parallèle du condensateur.

E9. Mettre en évidence le bon fonctionnement du circuit à l'oscilloscope. Commenter toutes les portions de la courbe.



L'essentiel

- Savoir relever la caractéristique d'un AOP et identifier les régimes linéaires et saturés
- Savoir pourquoi l'AOP sature et à quelles valeurs
- Comprendre l'intérêt du montage suiveur
- Savoir proposer un circuit permettant d'amplifier un signal (montage amplificateur inverseur) : montage et choix des valeurs des résistances ainsi que des tensions d'alimentation de l'AOP.
- Le montage « intégrateur » n'est pas à connaître. Il s'agissait d'un cas d'application concret d'opération mathématique en régime linéaire.

TP nº 6

Le filtrage analogique

Motivation

Les filtres sont des circuits électroniques à la base du traitement du signal. Leur facteur d'amplification appelé **gain** $G = 20 \log\left(\frac{V_s}{V_e}\right)$ ($G > 0$: amplificateur, $G < 0$: atténuateur) dépend de la fréquence, ce qui permet d'atténuer, d'amplifier ou de laisser passer certaines composantes spectrales d'un signal. En effet, un signal, quel qu'il soit (continu (fréquence nulle), carré, sinusoïdal, triangulaire, etc.) peut être écrit comme la somme de fonction sinusoïdales (théorème de Fourier). L'application de filtre(s) sur un signal permet donc de modifier ce dernier en modifiant certaines de ses composantes.

Les filtres permettent notamment d'éliminer ou d'affaiblir des fréquences parasites indésirables ou encore d'isoler dans un signal complexe la ou les bandes de fréquences utiles. En conséquence, les filtres servent donc principalement dans les systèmes de télécommunication (téléphone, télévision, radio), dans les alimentation électriques et dans les systèmes d'acquisition de signaux physiques (radars, appareils de mesure médicaux, etc.).

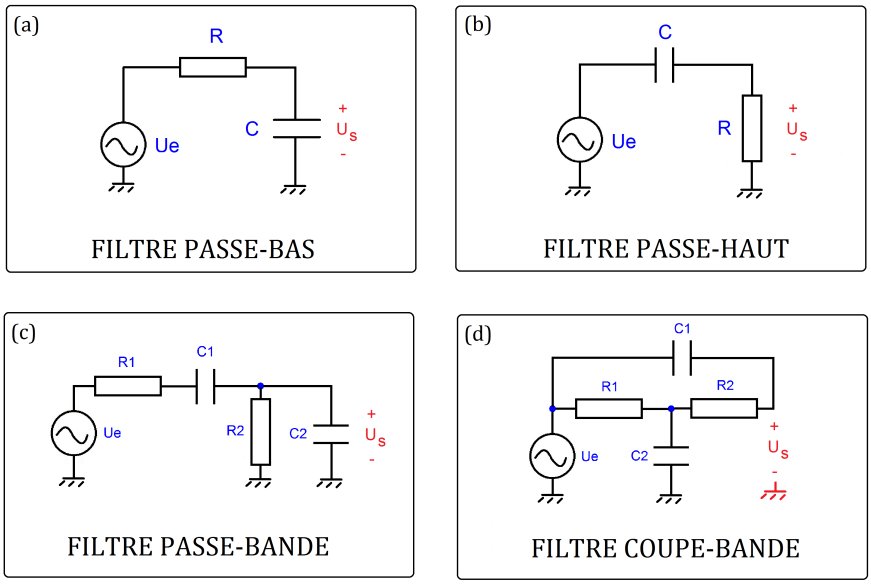


Figure 6.1 – Circuits simples de filtres passe-bas, passe-haut, passe-bande et coupe-bande analogiques

Le but de ce TP est de construire et tester des filtres analogiques simples du premier ordre.

I. Partie théorique

A. Représentation fréquentielle d'un signal

Jusqu'à présent, nous avons représenté les signaux électriques sous forme de courbes présentant l'intensité du signal en fonction du temps. Nous avons vu par exemple qu'une fonction sinusoïdale composite s'écrit comme suit :

$$u(t) = \overbrace{U_0 \cdot sin(\underbrace{2\pi f}_{\omega} t - \varphi)}^{\text{tension alternative}} + \overbrace{V_{OFF}}^{\text{tension continue}} \tag{6.1}$$

Une autre représentation d'un signal est la **représentation fréquentielle** encore appelée **spectre** ou **représentation harmonique** qui présente cette fois-ci la variation de l'amplitude d'un signal en fonction de la fréquence. Par exemple, une sinusoïde de fréquence 1 kHz aura pour spectre une unique **raie spectrale** en $f = 1\,\text{kHz}$. La **transformée de Fourier** permet d'établir l'équation du spectre d'un signal.

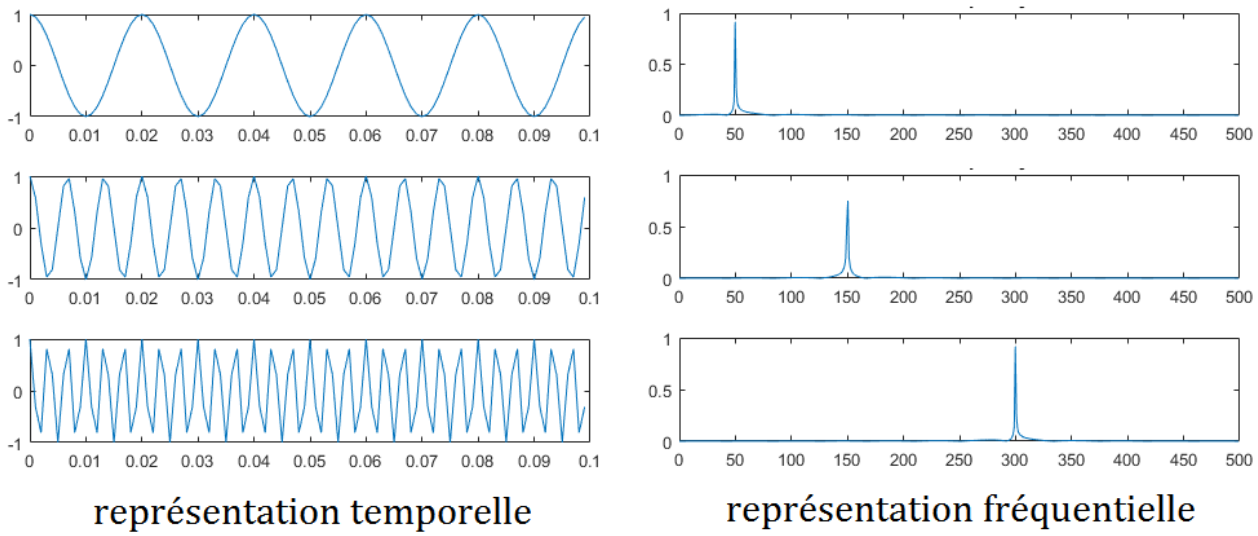


Figure 6.2 – Représentation temporelle de trois signaux sinusoïdaux et spectres associés

Nous venons de voir le cas simple et idéal d'une sinusoïde. En réalité, tout signal peut être exprimé comme la somme de fonctions trigonométriques (Théorème de Fourier). Ci-après est représenté un enregistrement sonore et le spectre associé après détermination de la transformée de Fourier associée. On retiendra que l'intervalle fréquentiel de la voix humaine est de l'ordre de [20 Hz ; 450 Hz].

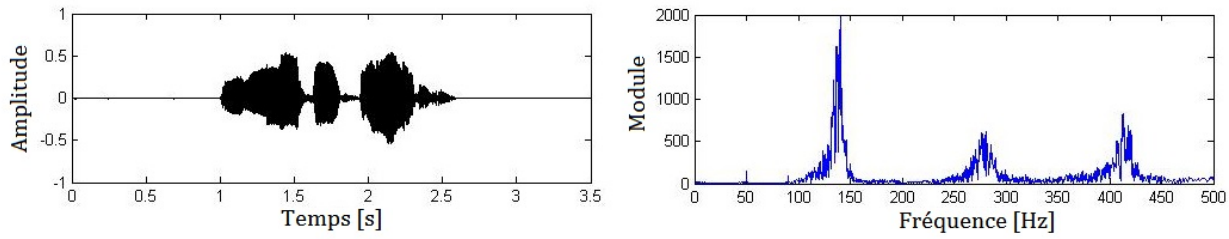


Figure 6.3 – Exemple d'enregistrement sonore et spectre associé

T1. Faites une recherche sur internet pour trouver le domaine de fréquences audibles par l'oreille humaine (...et retenez ces valeurs !)

T2. Soit un signal sinusoïdal d'amplitude 5 V et de période 10 ms. Quelle est l'équation de ce signal ? Tracer la représentation graphique de ce signal dans le domaine temporel sur deux périodes.

T3. Un second signal d'amplitude 2 V et de période 5 ms est ajouté au signal de la question T2. Sur le même graphique, tracer la résultante de la somme de ces deux signaux.

B. Filtrage analogique

Maintenant que nous avons vu comment tracer le spectre d'un signal, nous pouvons nous intéresser au vif du sujet : le **filtrage**. Lors du TP sur les condensateurs, nous avons déjà abordé cette notion sans vous la présenter de la sorte : les **condensateurs de découplage** permettent d'éliminer les composantes sinusoïdales (haute fréquence). Un filtre sert donc à laisser passer ou atténuer des composantes sinusoïdales.

On appelle **bande passante** la plage de valeurs fréquentielles pour lesquelles le gain est **3 dB en-dessous du gain maximal**. On a $G(f_c) = G_{max} - 3$. Cette valeur seuil de 3 dB découle en fait d'une autre définition : la valeur pour laquelle on perd la moitié de la puissance d'entrée, ou encore $\frac{1}{\sqrt{2}} = 70{,}7\,\%$ de l'amplitude du signal d'entrée :

$$20 \log_{10}\left(\frac{1}{\sqrt{2}}\right) \approx -3{,}01\,\text{dB} \tag{6.2}$$

Cette approximation n'est pas à retenir mais elle permet de vous montrer que la valeur seuil de 3 dB n'est pas choisie par hasard. En revanche, on retiendra que la fréquence correspondante à cette valeur seuil d'atténuation de 3 dB sera référée comme étant **la fréquence de coupure**. Il peut y avoir une ou deux fréquences de coupure en fonction de la nature du filtre. Les filtres passe-bas et passe-haut ont une unique fréquence de coupure f_c alors que les filtres passe-bande et coupe-bande en ont deux : $f_{c,1}$ et $f_{c,2}$. On parle de **bande coupée** en parlant de la ou des plages de fréquences où le signal est atténué de plus de 3 dB.

La figure suivante est un exemple de diagramme de Bode en gain pour un filtre passe-bas du premier ordre dont la fréquence de coupure est de $f_c = \frac{\omega_c}{2\pi} \approx 0{,}16\,\text{Hz}$.

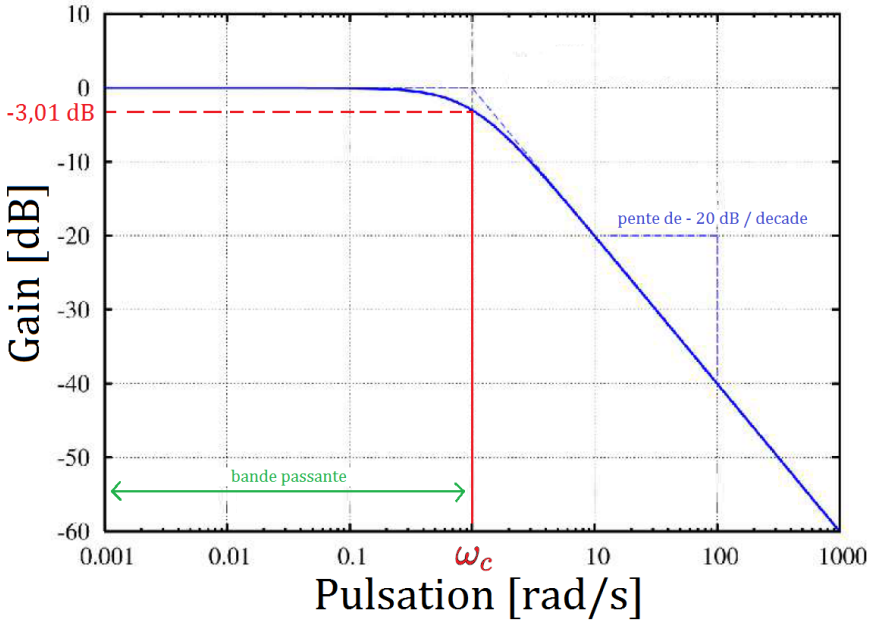


Figure 6.4 – Diagramme de Bode d'un filtre passe-bas (la bande passante est avant la fréquence de coupure f_c) et d'ordre 1 (la pente de la droite d'atténuation est de -20 dB par décade).

Ainsi, si l'on reprend la figure 6.3 et que l'on applique un filtre passe-bas dont la fréquence de coupure est de $f_c = 200\,\text{Hz}$ au signal d'entrée, on atténuerait toutes les raies fréquentielles correspondant aux fréquences supérieures à 200 Hz :



Figure 6.5 – Spectres des signaux d'entrée ($U_e(f)$) et de sortie ($U_s(f)$) aux bornes d'un filtre passe-bas

Une autre manière de présenter le filtre passe-bas serait de dire que si un signal d'entrée a une pulsation plus rapide que la pulsation de coupure du filtre, alors, le signal sera atténué (de manière plus ou moins importante en fonction de l'écart entre la pulsation du signal d'entrée et cette pulsation de coupure).

T4. On considère un signal sinusoïdal d'amplitude $A = 4\,\text{V}$ et de fréquence $f = 500\,\text{Hz}$. Quelle est l'équation de ce signal ? Dessiner l'allure du signal avant et après le filtre passe-bas dont le diagramme de Bode en gain est présenté à la Figure 6.4 ?

T5. Même question pour un signal de fréquence $f = 0{,}5\,\text{Hz}$.

Selon la même logique que les filtres passe-bas, nous avons :

- les **filtres passe-haut** : laissent passer les hautes fréquences (atténuent les basses fréquences $< f_c$) ;
- les **filtres passe-bande** : laissent passer qu'un intervalle de fréquences donné ;
- les **filtres coupe-bande** : atténuent une plage de fréquence donnée.

L'allure ou **gabarit** de ces filtres sont représentées au début de l'énoncé de ce TP, en figure 6.1.

Parmi les **filtres analogiques** (travaillant sur des signaux continus), on distingue les **filtres passifs** des **filtres actifs**. Les filtres actifs sont similaires aux filtres passifs, à la différence qu'ils emploient des AOP, ce qui permet non seulement d'atténuer des raies spectrales mais également d'amplifier le signal.

C. Prédéterminations pour la partie expérimentale

Il est temps d'appliquer le cours pour prédéterminer les résultats que nous allons obtenir expérimentalement. On considère le circuit suivant :

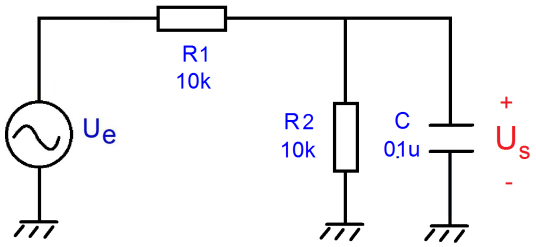


Figure 6.6 – Schéma électrique du filtre à étudier

T6. Quel montage fait à partir de deux résistances reconnaissez-vous ?

(a) Expression de la fonction de transfert complexe

T7. Déterminer l'expression de la fonction de transfert (**Transmittance**) $\underline{T}$ du filtre et la mettre sous la forme :

$$\underline{T} = \frac{K}{1 + j\frac{f}{f_c}} \tag{6.3}$$

T8. Déterminer les expressions de K et de f_c et faites les applications numériques en prenant soin de bien spécifier les unités.

(b) Expression du module et de l'argument de T

T9. Donner l'expression du module de $\underline{T}$ puis de l'argument.

T10. Tracer les courbes asymptotiques du gain et de la phase de ce filtre.

D. Filtres passe-bande et coupe-bande

La façon la plus simple d'obtenir un filtre passe-bande, c'est à dire ne laissant passer qu'un intervalle de fréquences donné est de mettre en série un filtre passe-haut avec un filtre passe-bas.

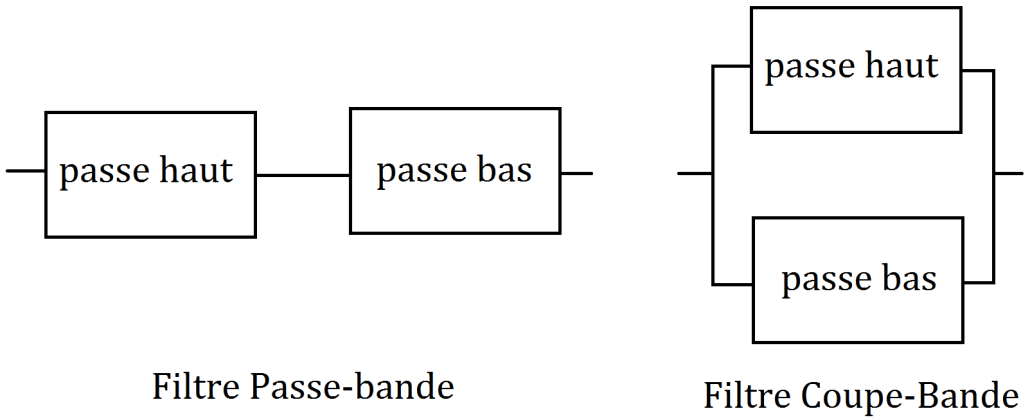


Figure 6.7 – Filtres passe-bande et coupe bande à partir de filtres passe-bas et passe-haut

T11. On désire concevoir un filtre passe-bande laissant passer les fréquences comprises entre 100 Hz et 1,5 kHz. En revanche, nous sommes limités en composants : nous n'avons que des résistances de 1 kΩ. Ainsi, sur le montage de la Figure 6.1 (c), que l'on appelle également **Pont de Wien**, on a $R_1 = R_2$. Quelles doivent être les valeurs de C_1 et de C_2 afin de réaliser ce filtre ?

La **fréquence centrale** est la fréquence pour laquelle le gain est à sa valeur maximale. On la calcule comme suit :

$$f_0 = \sqrt{f_{c,PB} \cdot f_{c,PH}} \tag{6.4}$$

où $f_{c,PB}$ et $f_{c,PH}$ sont respectivement les fréquences de coupures des filtres passe-bas et passe-haut. La pulsation centrale est la pulsation correspondante à la fréquence centrale.

T12. Quelle est la fréquence centrale du filtre étudié à la question T11 ? Calculer la pulsation centrale associée.

Le **facteur de qualité** Q traduit la sélectivité du filtre. Par définition,

$$Q = \frac{f_0}{\Delta f} \tag{6.5}$$

où f_0 est la fréquence centrale et Δf l'amplitude de la bande passante. La sélectivité du filtre est ainsi d'autant meilleure que le facteur de qualité est élevé (la bande passante Δf au dénominateur étant plus étroite).

À titre illustratif, ci-dessous est représenté l'influence du facteur Q. Plus Q est élevé, plus le filtre est sélectif.

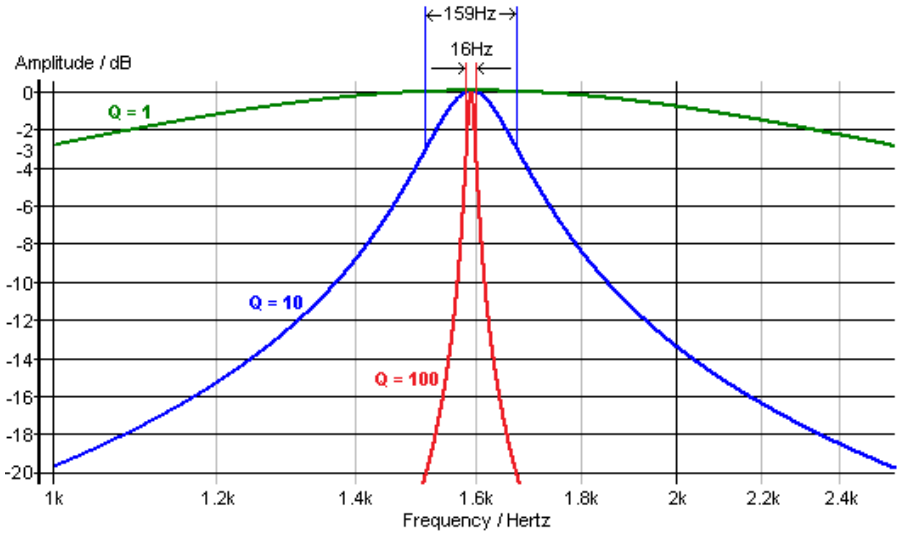


Figure 6.8 – Influence du facteur de qualité sur la sélectivité du filtrage d'un filtre passe-bande

T13. Calculer le facteur de qualité du filtre de la question T11.

II. Partie expérimentale

A. Première partie : filtres passe-bas et passe-haut

(a) Réponse en tension : $U_s(t)$

Pour avoir un aperçu de la réponse du filtre à un signal sinusoïdal, on simule classiquement le circuit avec l'outil ANALOGUE ANALYSIS.

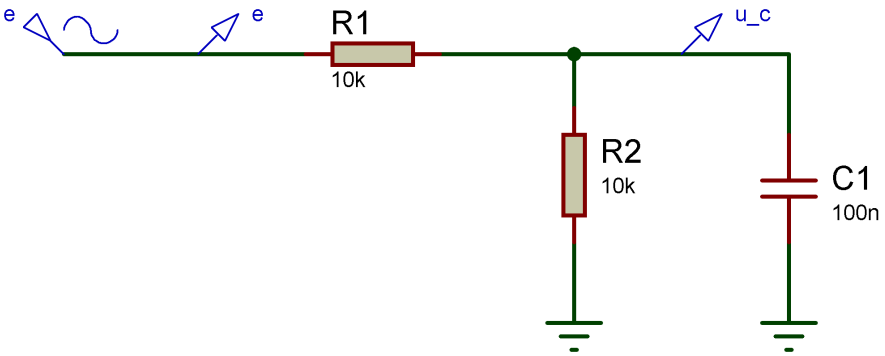
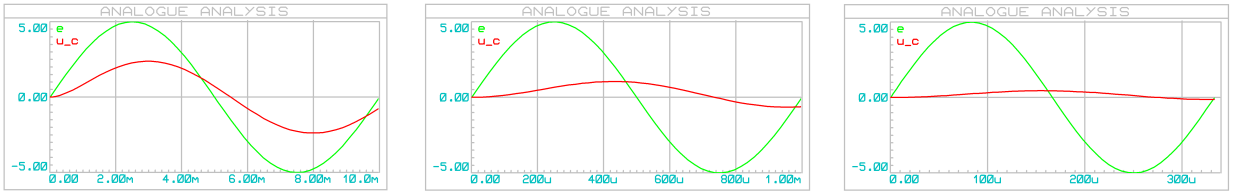


Figure 6.9 – Mesure de tensions sur Proteus



(a) 300 Hz **(b)** 1 kHz **(c)** 3 kHz

Figure 6.10 – Réponse des filtres pour des fréquences d'entrée de 300 Hz, 1 kHz et 3 kHz

On remarque que plus la fréquence est élevée et plus l'amplitude baisse. De même, un déphasage entre les signaux d'entrée et de sortie est de plus en plus prononcé.

E1. Faites de même pour un signal d'amplitude $A = 5\,\text{V}$ et de fréquence 100 Hz puis 3 kHz et enfin 200 kHz.

(b) Diagramme de Bode « a mano »

On rappelle que :

Grandeur	**Formule**
Tension efficace	$U_{eff} = \frac{U_{max}}{\sqrt{2}}$
Gain	$G(f) = 20 \cdot log(\frac{U_{s,eff}(f)}{U_{e,eff}(f)})$
Phase	$\varphi(f) = \frac{2\pi}{T}\Delta t(f)$

E2. Reproduire et compléter le tableau suivant :

f[Hz]	1	100	200	1 k	1,5 k	2 k	10 k	100 k
ω[rad · s^{-1}]								
$U_{s,eff}(f)$[V]								
$G(f)$[dB]								
$\varphi(f)$[rad]								

E3. À l'aide d'un tableur (Ex : Excel), tracer le diagramme de Bode en gain $G(f)$ et en phase $\varphi(f)$ en échelles linéaires.

E4. Modifier l'échelle des axes de pulsation afin qu'elle soient logarithmiques (clic droit sur l'axe, ...) puis conclure sur l'utilité de tracer un diagramme de Bode en échelle logarithmique.

E5. Que pouvez-vous dire de ce filtre ? Quelle est la fréquence de coupure ? (La fréquence de coupure est la fréquence pour laquelle le gain est de $G_{max} - 3\,\text{dB}$).

(c) Diagramme de Bode « automatique »

Sur Proteus, on peut directement obtenir le diagramme de Bode en utilisant l'outil graphique « FREQUENCY RESPONSE ». Le paramètre « reference » est le nom du label du signal d'entrée. C'est dans ce dernier que le simulateur va tester différentes fréquences. Il faut ainsi définir les labels IN et OUT et positionner une sonde de tension que l'on vient faire glisser sur le graphique en haut à gauche pour le gain et en bas à droite pour la phase. La simulation se lance dans le menu déroulant en effectuant un clic droit sur le graphique ou en cliquant sur la barre espace du clavier et non en bas à gauche de l'écran. Il faut parfois appuyer sur la touche espace pour relancer la simulation.

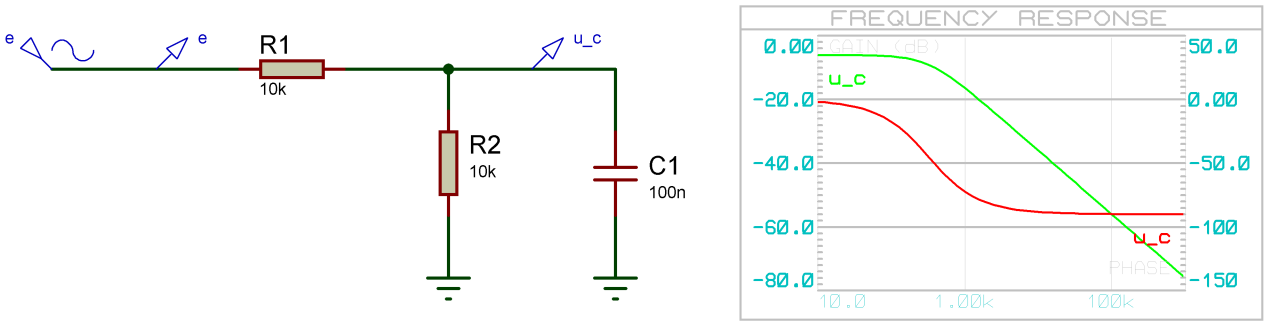


Figure 6.11 – Tracé d'un diagramme de Bode sur Proteus.

NB : le tracé est ici pour une plage de fréquence de 10 à 150 kHz en échelle semi-log. Il peut parfois être judicieux de modifier ce fenêtrage pour « mieux voir » le comportement du filtre.

On considère à présent le circuit suivant :

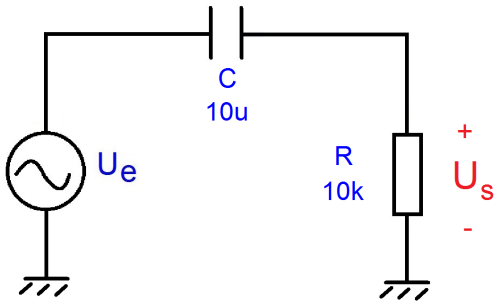


Figure 6.12 – Circuit dont on veut obtenir le diagramme de Bode en gain

E6. Afficher le diagramme de Bode en gain et en phase en utilisant l'outil FREQUENCY RESPONSE. Les diagrammes $G(f)$ et $\varphi(f)$ peuvent alors être traités séparément, exportés, etc. en double-cliquant dessus.

B. Deuxième partie : filtres passe-bande et coupe-bande

(a) Filtre passe-bande

E7. Simuler le montage de la Figure 6.1 (c) avec $R_1 = 1\,\text{k}\Omega$, $R_2 = 200\,\Omega$, $C_1 = 1{,}5\,\mu\text{F}$ et $C_2 = 2{,}5\,\mu\text{F}$. On alimentera le circuit avec un signal sinusoïdal d'amplitude $A = 5\,\text{V}$. Déterminer la bande passante de ce filtre.

E8. Que se passe t-il si l'on modifie R_2 pour qu'elle vale maintenant $2\,\Omega$?

E9. Déterminer le facteur de qualité Q.

E10. Modifier le circuit afin d'avoir un filtre passe-bande de bande passante 100 Hz à 2,5 kHz.

(b) Filtre coupe-bande

E11. En s'inspirant de la Figure 6.1 (d), modifier la configuration des composants afin d'obtenir un filtre coupe-bande. Simuler le nouveau circuit et commenter le diagramme de Bode.

(c) Influence de l'ordre d'un filtre

Nous allons tracer des diagrammes de Bode à fréquence de coupure constante mais à ordre de filtre croissant. Pour ce faire, nous allons simuler un simple filtre passe-bas sous Proteus et nous allons dupliquer le filtre autant de fois que l'on veut augmenter l'ordre du filtre.

E12. Lancez une simulation du circuit suivant en AC SWEEP de 10 à 100 kHz et relever la pente de la courbe de gain. Que remarque t-on ?

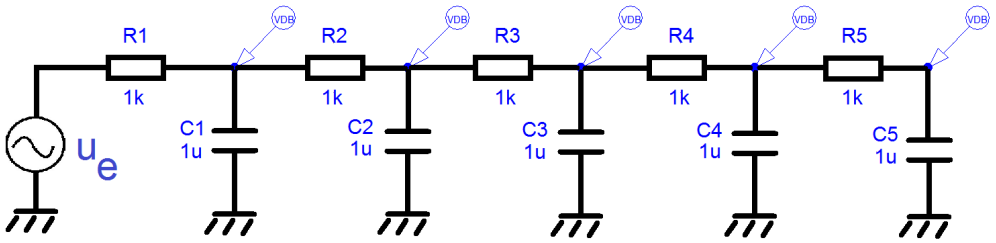


Figure 6.13 – Circuit électrique permettant de mettre en évidence l'impact de l'ordre d'un filtre passe-bas à fréquence de coupure constante. Les sondes relèvent le gain V_s/V_e pour respectivement des filtres passe-bas d'ordre 1, 2, 3, 4 et 5.

E13. Remplacez les sondes en gain par des sondes de phase. Quel est l'impact de l'ordre d'un filtre sur le déphasage ?

L'essentiel

- Conception d'un filtre passe-bas, passe-haut, passe-bande et coupe-bande (de fréquence de coupure $f_c = \frac{1}{2\pi RC}$).
- Tracé du diagramme de Bode sur Proteus.
- Détermination de la fréquence centrale, du facteur de qualité d'un filtre passe-bande.
- Détermination de la bande passante d'un filtre.
- Lien qualitatif entre l'ordre d'un filtre et la forme de ses diagrammes en gain et en phase.

TP n° 7

La diode à jonction PN

Motivation

La diode est un composant électronique **non-linéaire** (le courant ne varie pas linéairement avec la tension à ses bornes comme pour une résistance) et **polarisé** : son sens de branchement est crucial. Ce composant permet de laisser passer le courant que dans un sens et, peut-être sans le savoir, vous l'avez déjà utilisé maintes fois en électronique. La Figure 7.1 présente le symbole et la représentation schématique de la diode classique et de la diode électro-luminescente (DEL).

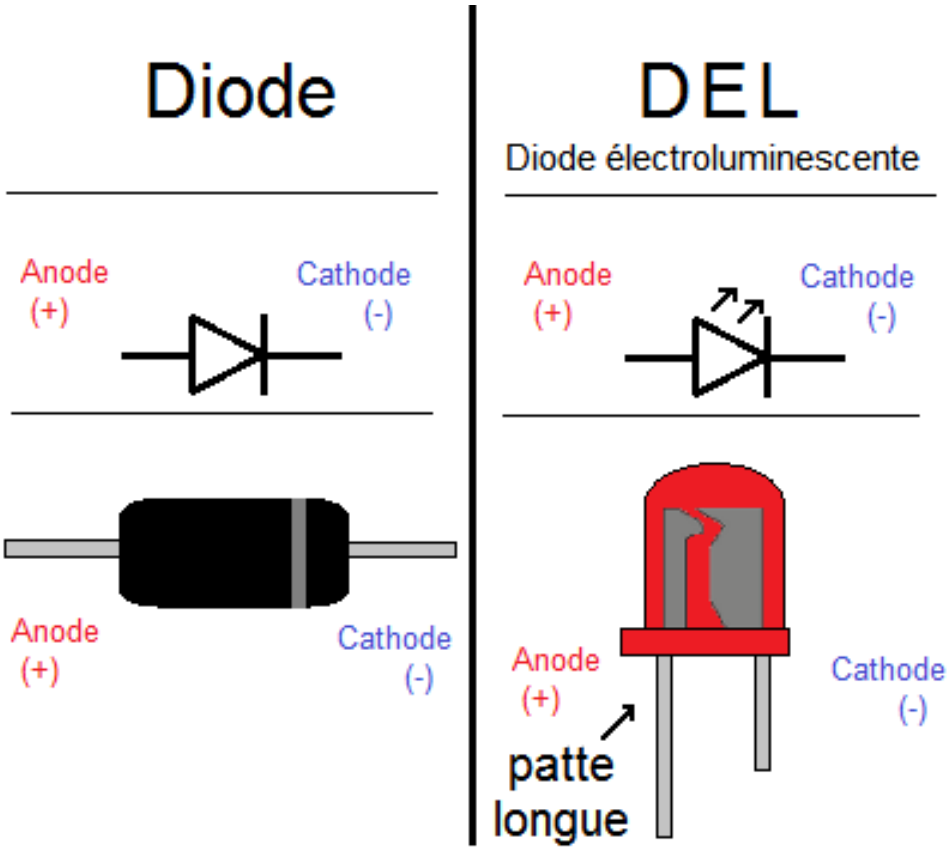


Figure 7.1 – Diode et DEL : schéma et composant électriques

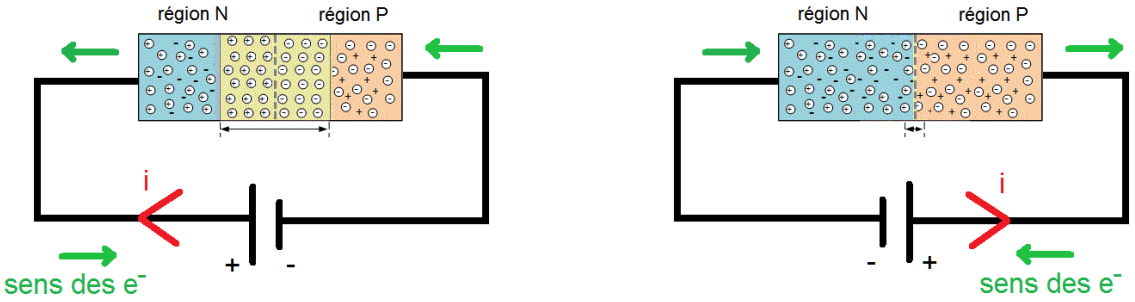
L'objectif de ce TP est de retrouver par simulation la caractéristique et le point de fonctionnement d'une diode.

I. Partie théorique

A. Fonctionnement d'une diode

(a) Polarisation de la diode

On parle de **polarisation indirecte** lorsque l'on branche la borne (+) d'un générateur à la jonction N et la borne (-) à la jonction P. Il en résulte un élargissement de la zone de déplétion, donc une augmentation de la barrière d'énergie pour passer de la Bande de Valence à la Bande de Conduction. On dit que la diode est **bloquée** (voir Figure 7.2 (a)). On notera que la barrière de potentiel en polarisation inverse augmente pour les trous et augmente aussi pour les électrons à cause du signe négatif de leur charge.



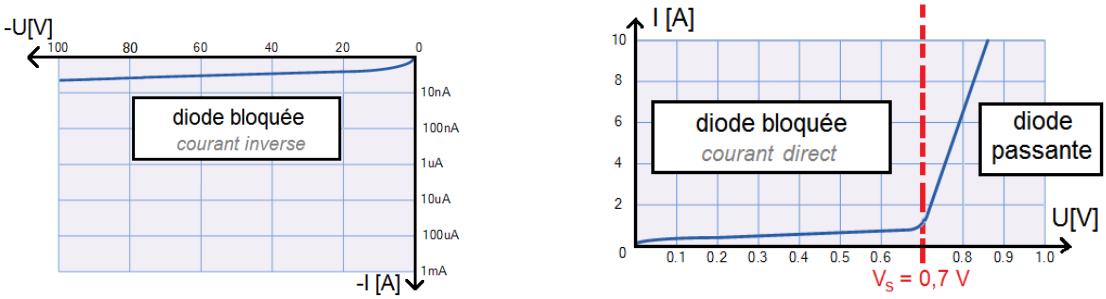
(a) polarisation inverse **(b)** polarisation directe

FIGURE 7.2 – polarisation d'une diode

En **polarisation directe**, la borne (+) est reliée à la jonction P (Positive) et la borne (-) à la jonction N (Négative). Le générateur joue un rôle de pompe à électron et le courant va du (+) vers le (-). Les électrons se déplaçant dans le sens opposé, la zone de déplétion est amincie et les électrons peuvent librement circuler à condition de franchir une faible barrière pour passer de la bande de valence à la bande de conduction : la tension de seuil V_s.

(b) Caractéristique de la diode

Une **diode** est un composant électronique obtenu en recouvrant une jonction PN par une couche isolante afin d'en faciliter l'utilisation.



(a) polarisation inverse **(b)** polarisation directe

FIGURE 7.3 – Caractéristique de la diode en polarisation inverse et en polarisation directe

En **polarisation directe**, le courant s'établit une fois la barrière de potentiel dépassée, ce qui se produit à la tension de seuil $V_s \sim 0,7V$. On a alors une chute de tension due à l'annulation de la zone de déplétion et donc une tension aux bornes du composant :

$$u_D = u_e - u_s \tag{7.1}$$

Le courant quant à lui suit la **loi de Shockley** :

$$i = I_0 \left(e^{\frac{uq}{k_B T}} - 1 \right) \tag{7.2}$$

En **polarisation inverse**, la diode est bloquée, tout comme en polarisation directe avec une tension inférieure à la tension de seuil.

(c) Les trois modèles de diode

Les trois modèles de diode sont les suivants :

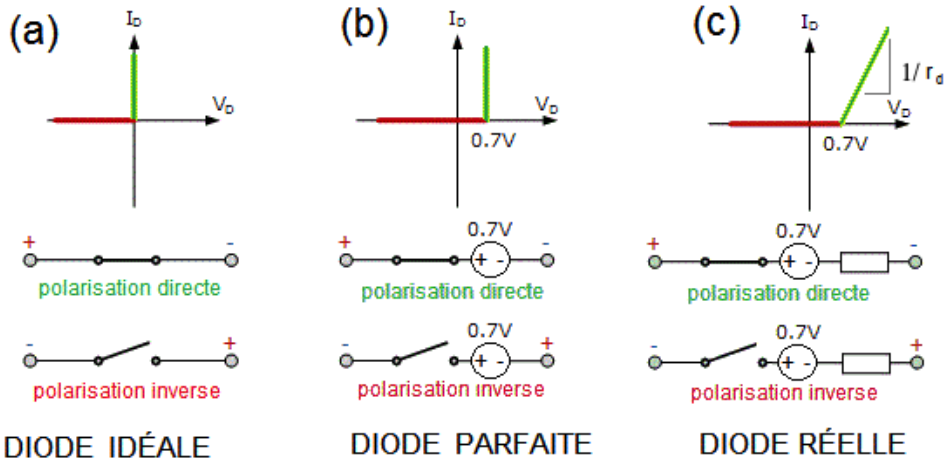


Figure 7.4 – Les trois différents modèles de diode : **(a)** la diode idéale, **(b)** la diode parfaite et **(c)** la diode réelle

Lors des TPs, nous ne considèrerons que le modèle de diode réelle puisqu'il est le plus proche de la réalité. La différence avec le modèle de la diode parfaite est qu'il faut prendre en compte la tension de seuil (diode idéale) mais également la résistance interne r_d de la diode. La caractéristique dans la zone passante est ainsi linéaire tout comme l'est la caractéristique d'une résistance idéale (régie par la loi d'Ohm : $I = \frac{1}{R} \cdot U$:

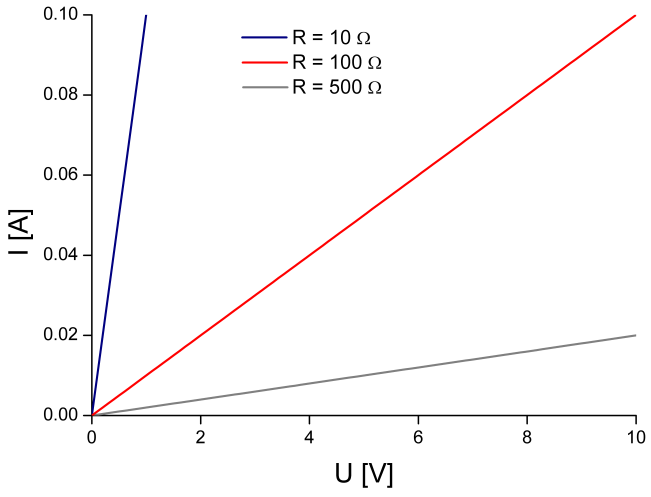


Figure 7.5 – Caractéristique de la résistance pour différentes valeurs de R

B. Point de fonctionnement

Nous venons de voir que la diode (tout comme les transistors) a une caractéristique courant-tension qui n'est pas linéaire comme l'est la résistance (loi d'Ohm). Ainsi, déterminer le courant la traversant et la tension à ses bornes est un petit peu plus complexe et cela revient à déterminer le **point de fonctionnement**.

Pour déterminer le point de fonctionnement, on cherche à égaliser l'équation de la caractéristique de la diode avec la **droite de charge** du circuit dans lequel elle est utilisée. Pour déterminer cette dernière, on utilise le théorème de Thévenin pour se ramener à une maille simple associant un résistance et une diode en série comme suit :

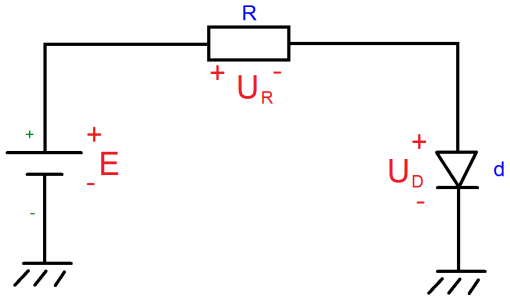


Figure 7.6 – Maille simple comprenant une diode et une résistance en série

La loi des mailles donne : $E = U_\mathrm{R} + U_\mathrm{D} \Leftrightarrow E = Ri + U_\mathrm{D}$
Puisque $i = i_\mathrm{R}$, on a :

$$I_\mathrm{d} = -\frac{1}{R}U_\mathrm{D} + \frac{E}{R} \tag{7.3}$$

L'équation de la droite de charge est alors écrite sous la forme d'une fonction affine (y=ax+b) d'ordonnée à l'origine $\frac{E}{R}$ et de coefficient directeur -$\frac{1}{R}$.

Le point de fonctionnement peut être déterminé en égalisant l'équation de la droite de charge 7.3 avec l'équation de la caractéristique de la diode (points expérimentaux ou équation de Shockley). La plupart du temps, résoudre l'équation analytiquement est laborieux. On résout ainsi le système graphiquement comme suit :

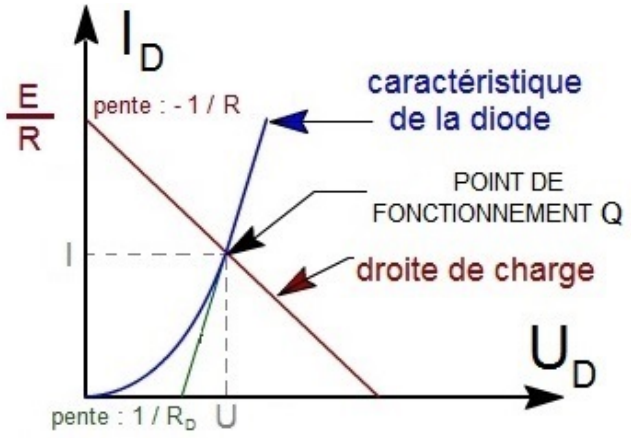


Figure 7.7 – Détermination graphique du point de fonctionnement

La caractéristique de la diode est intrinsèque au dipôle et donc indépendant du circuit. La droite de charge quant à elle ne dépend pas des caractéristique de la diode, elle ne reflète que les informations

du circuit dans lequel la diode est insérée.

C. Les principales diodes

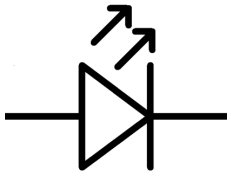
Il existe différents types de diodes qui ont chacune une utilité singulière. Cette section présente les diodes les plus communément utilisées afin que vous appreniez à les reconnaître tout en comprenant leurs principes de fonctionnement/ cas d'utilisation.

(a) la diode électroluminescente (DEL)

Pas besoin de la présenter, c'est certainement le premier composant que vous avez testé lors de votre initiation à Arduino.



(a) exemple de composant



(b) symbole

Mais avez-vous déjà pensé à alimenter correctement vos DELs ? En effet, en fonction de la couleur, les DEL ont des tensions de seuil différentes et supérieures à celle d'une diode classique.

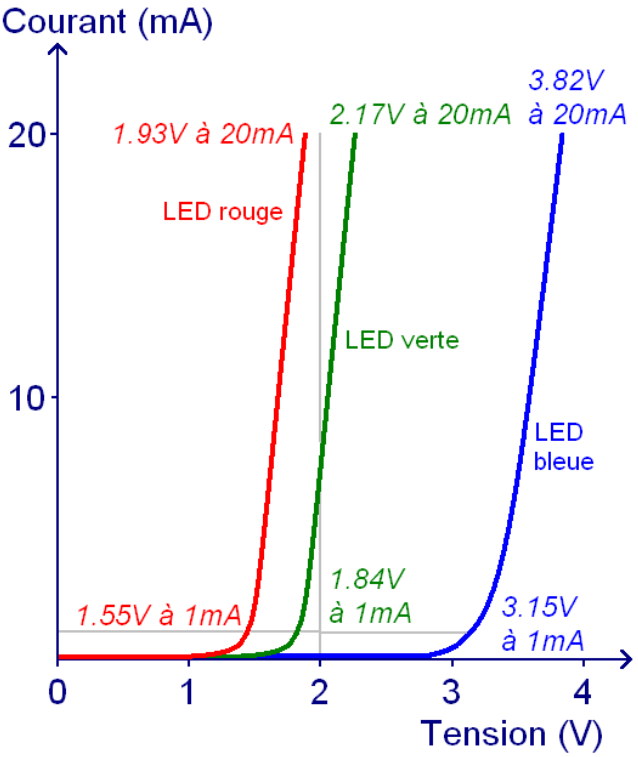


Figure 7.9 – Caractéristique de DEL de différentes couleurs

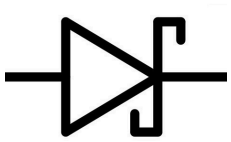
Ainsi, pour alimenter un DEL rouge à l'intensité lumineuse maximale que peut délivrer une pin d'Arduino (40 mA), et dont on approximera la tension de seuil à $V_{seuil} = 1{,}2$ V, la résistance à mettre en série est de $R = \frac{E-V_s}{0{,}04} = 95\ \Omega$, soit une résistance de 100 Ω, pile la valeur donnée dans le poly de TP ING1 du premier semestre !

(b) la diode Schottky

La diode Schottky est constituée d'une jonction métal/semi-conducteur ce qui lui procure une chute de tension directe réduite (0,3 V environ) et une dynamique nettement améliorée du fait de l'absence de porteurs minoritaires engagés dans le processus de conduction.



(a) exemple de composant



(b) symbole

Son désavantage est son coût et le fait qu'elle ne supporte pas les tensions supérieures à 50 V.

(c) la diode Zener

les **diodes Zener** sont des diodes spécialement conçues de façon à laisser passer le courant inverse, mais ceci uniquement si la tension à ses bornes est plus élevée que le seuil de l'effet d'avalanche. Ce seuil en tension inverse (tension Zener) est de valeur déterminée pouvant aller de 1,2 V à plusieurs centaines de volts. Certaines d'entre elles disposent d'une troisième patte permettant de régler la tension Zener.

Elles sont fréquemment employées pour réguler des tension. Cela sera notamment l'un des principaux objets d'étude du TP3.



(a) exemple de composant



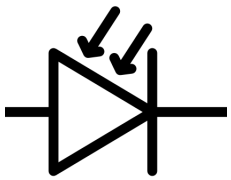
(b) symbole

(d) la photodiode

La photodiode génère un courant à partir des paires électrons-trous produites par l'incidence d'un photon suffisamment énergétique dans le cristal (source de lumière extérieure). L'amplification de ce courant permet de réaliser des commandes en fonction de l'intensité lumineuse perçue par la diode (interrupteur à seuil de luminosité par exemple).



(a) exemple de composant



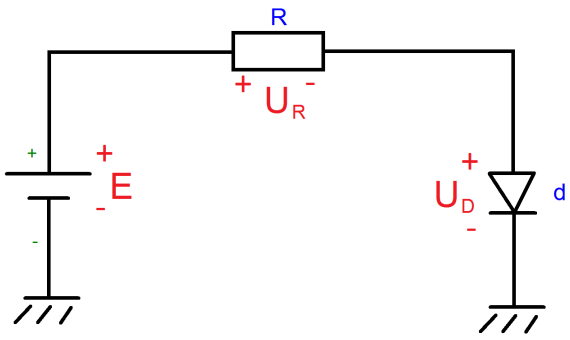
(b) symbole

NB :
Vous remarquerez qu'il n'y a pas d'exercice à faire pour la partie théorique de ce TP, juste lire, comprendre et retenir.

II. Partie expérimentale

A. Caractéristique de la diode

Simuler le montage de la Figure 7.6 sur Proteus avec une résistance $R = 100\Omega$.



E1. Relever sur votre compte-rendu la référence de votre diode (parmi les diodes 1N4148, 1N4007 et 1N5408) et faites varier la tension continue E afin d'effectuer les mesures suivantes :

E [V]	**-2**	**-1**	**0**	**0,2**	**0,4**	**0,6**	**0,8**	**1**	**1,2**	**1,4**	**1,6**	**1,8**	**2**	**3**	**4**	**5**
V_d [V]																
I_d [mA]																

E2. Tracer la **caractéristique courant-tension** $I_d = f(V_d)$ avec I_d en mA sur Excel.

E3. Sur la caractéristique courant-tension, identifier la zone de polarisation directe, la zone de polarisation inverse et la zone où la diode est bloquée/ passante.

E4. Est-ce que cette caractéristique change si l'on change la résistance ? Pourquoi ?

E5. Trouver un moyen de tracer directement la caractéristique de la diode sur Proteus (voir Cours 1, indice : DC Sweep).

B. Droite de charge et point de fonctionnement

On souhaite à présent déterminer graphiquement le point de fonctionnement de la diode.

E6. Qu'est-ce qu'un point de fonctionnement ?

E7. Renseigner la tension de générateur E et la valeur de la résistance avec laquelle vous travaillez, déterminer l'équation de la droite de charge statique et la tracer sur la caractéristique de votre diode.

E8. En déduire le **point de fonctionnement**. Comment varie t-il si l'on augmente E ? Justifier par le calcul.

E9. Déterminer la **résistance dynamique** R_d de la diode puis proposer les schémas électriques de diode réelle en polarisation inverse et directe.

E10. La **résistance statique** R_s est la résistance équivalente à la diode lorsqu'elle est insérée dans un circuit ayant la même différence de potentielle à sés bornes et traversée par le même courant. Combien peut valoir R_s dans votre circuit ?

TP n° 8

La chaîne d'acquisition

Depuis maintenant plusieurs décennies, les calculs se font sur des processeurs de plus en plus rapide et de moins en moins énergivores. Afin de pouvoir traiter un signal issu d'un capteur analogique, il est nécessaire de le convertir en une grandeur numérique.

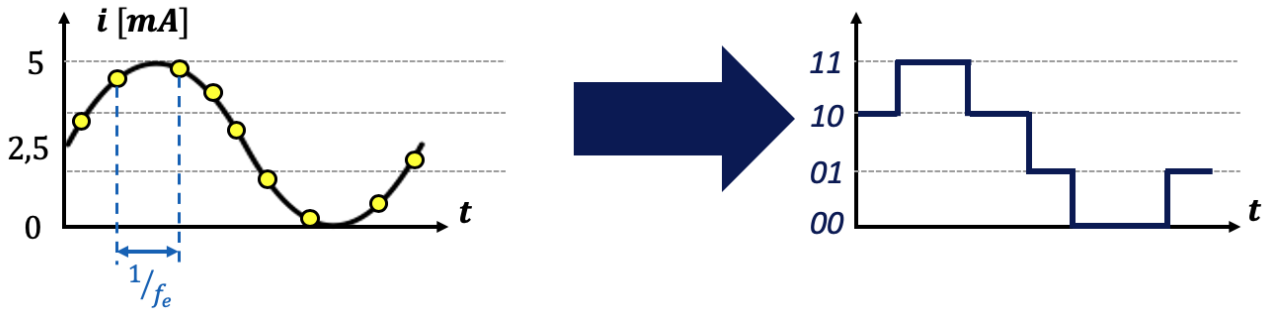


Figure 8.1 – Signal en entrée et en sortie de la chaîne d'acquisition

Dans sa forme la plus simple, une chaîne d'acquisition de données comprend :

- un **capteur** : conversion d'une grandeur physique en une grandeur électrique ;
- un **module de conditionnement** : amplification du signal issu du capteur ;
- un **échantillonneur-bloqueur** : discrétisation du signal analogique à convertir et maintien de la grandeur le temps de la conversion du CAN ;
- un **convertisseur analogique-numérique** : conversion de la grandeur analogique en une grandeur numérique.

Sa structure dans sa version la plus simple est présentée sur la Figure 8.2.



Figure 8.2 – Vue très simplifiée d'une chaîne d'acquisition de données

Le but de ce TP est d'étudier une chaîne d'acquisition simple aux travers de simulations sur Proteus.

Au cours de ce TP, nous allons concevoir une chaîne d'acquisition de données permettant de numériser un courant sinusoïdal d'amplitude 2,5 mA strictement positif (offset de 2,5 mA le faisant osciller avec une amplitude de 2,5 mA autour de 2,5 mA), et de fréquence 1 kHz, sur deux bits, avec une fréquence d'échantillonnage de 10 kHz. Les choix de la fréquence d'échantillonnage et de la valeur du condensateur du bloqueur seront justifiés en ING3.

Les parties I, II et III sont indépendantes. L'assemblage des trois modules ne peut se faire qu'après avoir terminé les parties I, II et III.

I. Module de conditionnement

Dans un premier temps, nous allons chercher à convertir le signal d'entrée (courant) en une tension et à l'amplifier comme illustré sur la Figure 8.3.

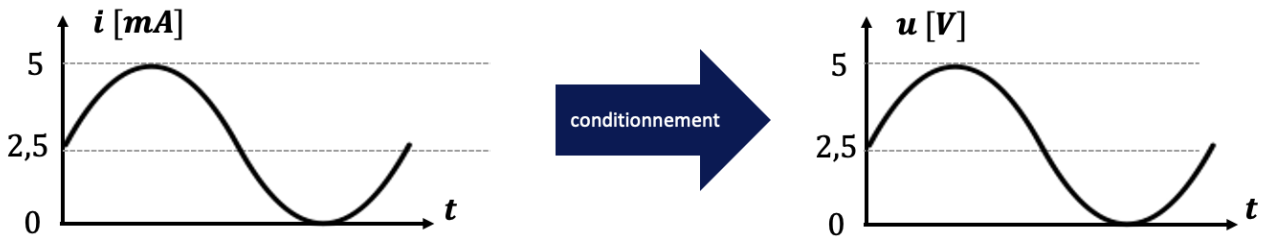


Figure 8.3 – Conversion courant-tension et amplification

Pour ce faire, nous allons utiliser le montage présenté sur la Figure 8.4.

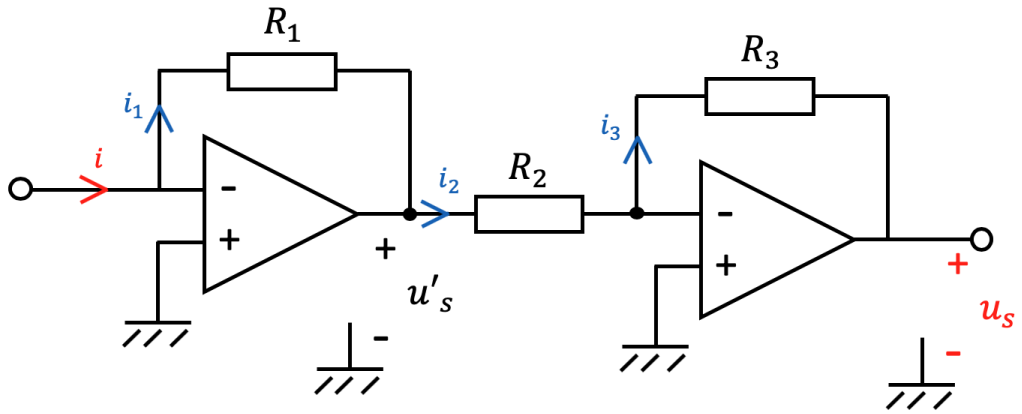


Figure 8.4 – Convertisseur courant-tension

T1. En considérant que les AOP sont idéaux, montrer que l'on a :

$$u_s = \frac{R_1 R_3}{R_2} i \tag{8.1}$$

T2. Quelle est l'équation de u_s en fonction du temps ? Trouver un triplet de valeurs $\{R_1, R_2, R_3\}$ de sorte à ce que le signal d'entrée qui a une étendue de 0 mA à 5 mA soit converti en une tension u_s d'étendue 0 V à 5 V.

S1. Simuler le circuit sur Proteus et mettre en évidence son bon fonctionnement sur une période. (ANALOGUE ANALYSIS).

II. Échantillonneur-bloqueur

Dans un second temps, nous allons discrétiser le signal afin de présenter en entrée du convertisseur analogique-numérique une tension constante lui laissant le temps de convertir, comme illustré sur la Figure 8.5.

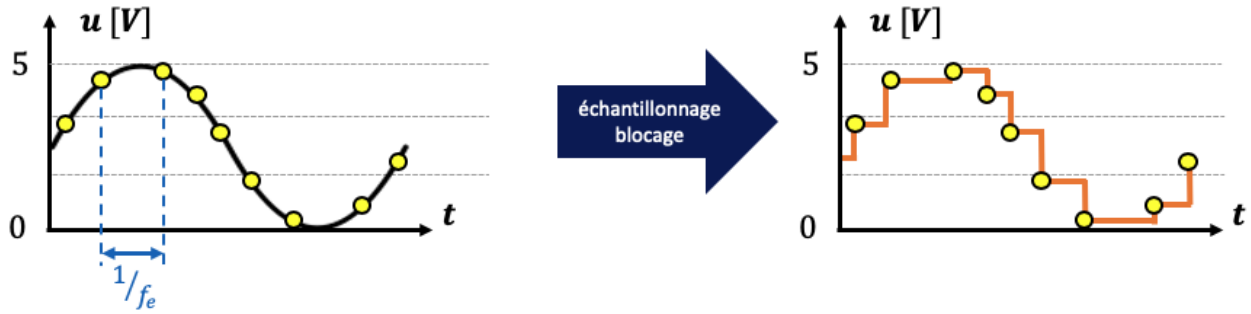


Figure 8.5 – Illustration de l'échantillonnage - blocage

On travaillera sur un signal d'entrée d'amplitude 2,5 V, de 2,5 V d'offset et de fréquence 1 kHz. Afin d'échantillonner le signal, on utilise un transistor FET à canal N dont la grille est pilotée par un signal rectangulaire de période d'échantillonnage $\frac{1}{f_e}$ avec $f_e = 10\,\text{kHz}$: on branche au drain le signal d'entrée et tous les $\frac{1}{f_e} = 100\,\mu\text{s}$, on prélève le signal de sortie en le connectant à la source du transistor. Le circuit est présenté sur la Figure 14.3.

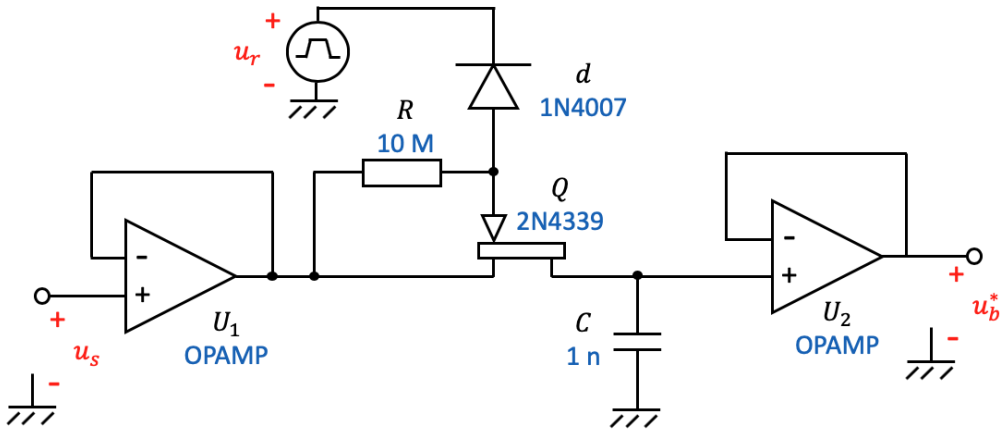


Figure 8.6 – Circuit de l'échantillonneur-bloqueur

T3. Consulter la datasheet du 2N4339 et justifier la valeur du condensateur de blocage en discutant la valeur de la constante de temps RC et la fréquence du signal rectangulaire u_r. En déduire le rapport cyclique minimal pour que le signal u_r permette une charge complète du condensateur.

S2. Simuler l'échantillonneur-bloqueur sur Proteus et mettre en évidence son bon fonctionnement (signaux u_s et échantillonné-bloqué u_b^* sur une période en ANALOGUE ANALYSIS).

NB : On prendra pour le signal rectangulaire $V_L = 0\,\text{V}$, $V_H = 5\,\text{V}$, $t_r = 1\,\text{ns}$, $t_f = 1\,\text{ns}$, $f = 10\,\text{kHz}$ et $\alpha = 50\,\%$.

S3. Que se passe t-il si l'on sous-échantillonne le signal, par exemple en choisissant $f = 1\,\text{kHz}$ pour u_r. Expliquer ce que vous observez sur Proteus (simulation ANALOGUE ANALYSIS) et ce que cela peut impliquer lors de la phase de traitement des données une fois converties (numériques).

Pensez à remettre une fréquence d'échantillonnage de 10 kHz pour la suite de l'étude.

III. Convertisseur analogique-numérique Flash

Pour cette dernière partie, on cherche à convertir une tension analogique discrète en un signal numérique ; ici, codé sur 2 bits pour alléger le circuit à l'aide d'un convertisseur analogique-numérique (CAN) Flash.

Le CAN Flash est composé de deux parties :

- un étage de comparaison à base d'AOP montés en comparateur qui comparent la tension analogique à des fractions croissantes de la tension de référence ;
- un étage d'encodage (logique combinatoire) qui prend les sorties des différents comparateurs et qui les interprète en un mot binaire.

La Figure 8.7 présente la structure d'un CAN Flash 2 bits.

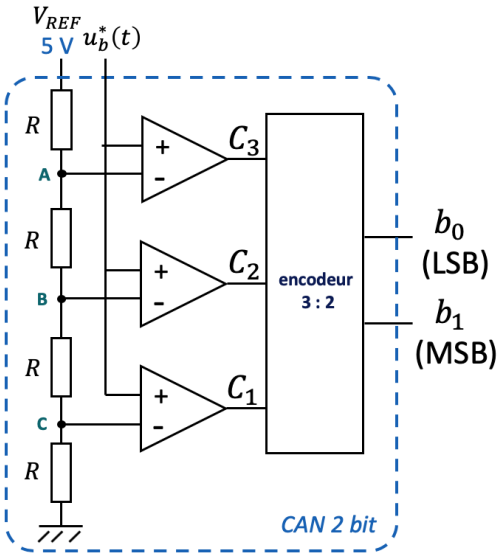


Figure 8.7 – Structure du CAN Flash 2 bit.

On fixe la tension de référence à 5 V et on prendra $R = 1\,\text{k}\Omega$.

T4. Déterminer les valeurs des tensions aux points A, B et C. En déduire le fonctionnement de l'étage de comparaison.

T5. Remplir le tableau suivant :

Tension analogique d'entrée	C_1	C_2	C_3	b_1	b_0
	0	0	0		
	1	0	0		
	1	1	0		
	1	1	1		

T6. Proposer une équation booléenne et *in fine* un circuit logique pour les sorties b_1 et b_0.

T7. Comment doit-on modifier le circuit pour qu'il soit d'une résolution de 4 bits ?

S4. Simuler votre CAN sur Proteus et mettre en évidence son bon fonctionnement.

S5. Simuler le circuit complet en intégrant les parties I, II et III sur une période (ANALOGUE ANALYSIS pour les signaux u_s et u_b^*) et DIGITAL ANALYSIS sur les sorties b_1 et b_0. Mettre en évidence le bon fonctionnement du circuit.

TP n° 9

Le transistor bipolaire en régime statique

Motivation

Le **transistor bipolaire** est un composant électronique actif à base de semi-conducteurs qui s'utilise principalement pour deux applications :

- l'amplification d'un signal ;
- la commutation : le transistor peut alors être vu comme un robinet qui laisse ou ne laisse pas passer le courant.

Afin de choisir le bon mode de fonctionnement, il est nécessaire de correctement polariser les jonctions en appliquant des tensions appropriées. La Figure 9.1 présente les caractéristiques courant-tension de l'amplificateur à émetteur commun (transistor NPN branché de telle sorte à ce que l'émetteur soit en commun aux tensions d'entrée et de sortie).

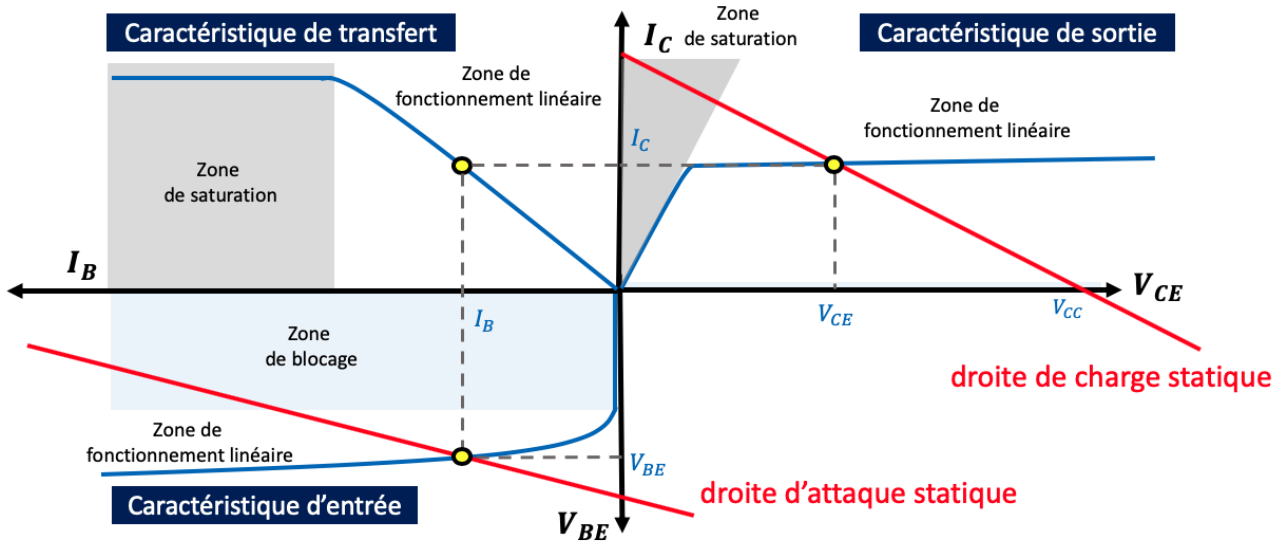


Figure 9.1 – Caractéristique courant-tension d'un amplificateur NPN à émetteur commun.

Le but de ce TP est d'apprendre à tracer les caractéristiques d'un montage à transistor en régime DC et d'en déduire son point de fonctionnement. La seconde partie de ce TP s'intéresse au fonctionnement en commutation.

I. Partie théorique

A. Introduction sur le transistor

« Transistor » vient de l'anglais *transfer resistor* qui signifie résistance de transfert. C'est un composant électronique composé de deux semi-conducteurs qui permet d'amplifier des courants électriques.

S'agissant du premier TP de cette seconde section sur les transistors, on se doit de rappeler le vocabulaire propre aux transistors bipolaires qui sont des composants constitués de deux jonctions PN.

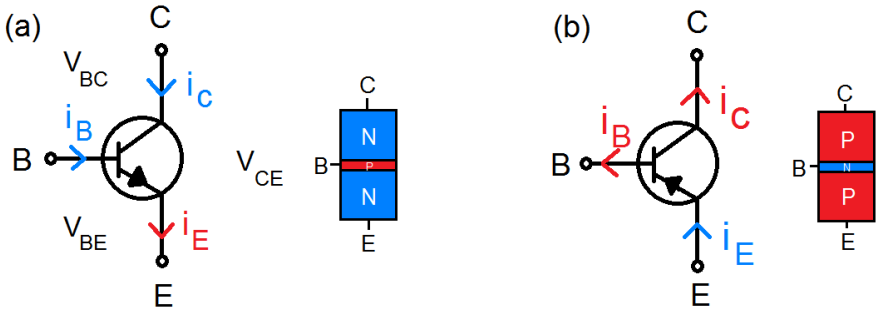


Figure 9.2 – Transistors **(a)** NPN et **(b)** PNP

Les trois bornes du transistor bipolaire sont la **base** (B) où se trouve le côté plat, l'**émetteur** (E) où se trouve la flèche, et le **collecteur** (C). Dans la mesure où c'est le transistor NPN qui est le plus communément utilisé, nous allons commencer par l'étudier en premier.

B. Les trois zones de fonctionnement

Afin de permettre le passage d'un courant à travers le transistor, il faut d'abord s'assurer de la conduction des jonctions. On distingue trois **zones de fonctionnement** :

- **zone de blocage** : le transistor équivaut à un interrupteur ouvert ;
- **zone active** : le transistor conduit correctement, le point de fonctionnement peut librement se déplacer dans cette zone ;
- **zone de saturation** : il peut être remplacé par un circuit équivalent comprenant deux générateurs de tension.

On appelle **caractéristique statique de sortie** la courbe $I_C = f(V_{CE})$ que voici :

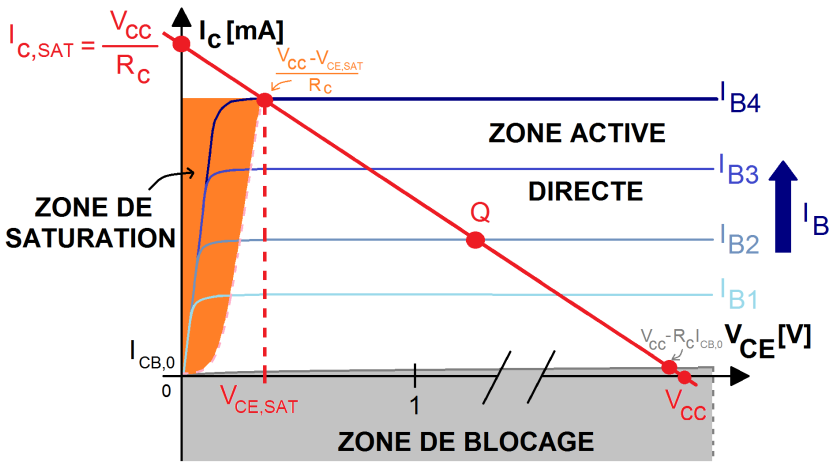


Figure 9.3 – Caractéristique statique de sortie

II. Partie expérimentale

A. Caractéristique de sortie

Dans un premier temps, on souhaite relever la **caractéristique de sortie** $I_C = f(V_{CE})$ du transistor 2N2222.

E1. Simuler sur Proteus la caractéristique de sortie du transistor en reproduisant le montage de la Figure 9.4.

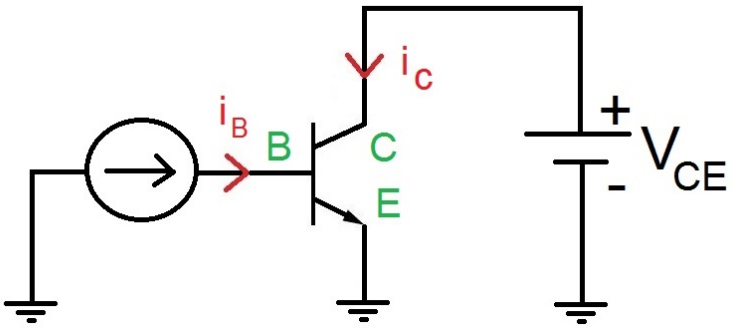


Figure 9.4 – Circuit permettant de relever la caractéristique de sortie

Vous tracerez sur le même graphe, les caractéristiques en DC Sweep pour $I_B = 10\ \mu A$, $I_B = 50\ \mu A$ et $I_B = 100\ \mu A$ et vous commenterez vos résultats. Vous choisirez plage de valeur raisonnable pour V_{CE} afin de pouvoir mettre en évidence la zone de saturation et la zone d'amplificaiton linéaire.

Remarque : Vous pouvez également utiliser le mode de simulation « DC Transfer curve analysis ».

E2. Isoler la courbe de $I_C = f(V_{CE})$ pour $I_B = 10\ \mu$ A et identifier la zone de saturation et la zone active directe puis donner une valeur approximative à deux chiffres significatifs pour $V_{CE,SAT}$. D'après le premier graphique, $V_{CE,SAT}$ est-il un paramètre constant propre au transistor ou un paramètre dépendant du circuit dans lequel il est ? Justifier.

B. Caractéristique d'entrée

Nous allons à présent relever la **caractéristique d'entrée** du transistor $I_B = f(V_{BE})$.

E3. Proposer un circuit permettant de relever en DC Sweep une telle caractéristique et lancer la simulation. À quel composant électronique peut-on comparer la jonction base-émetteur du transistor ? Proposer deux modèles dont un avec une résistance que vous mesurerez à l'aide d'une simulation.

E4. En déduire une condition pour que le transistor soit en conduction.

C. Caractéristique de transfert

Nous allons à présent vérifier expérimentalement que $I_C = \beta I_B$ et relever le gain en courant β à l'aide de la **caractéristique de transfert** $I_C = f(I_B)$ à V_{CE} constant.

E5. Pour $V_{CE} = 10$ V, proposer un circuit permettant de relever la caractéristique de transfert pour des valeurs de I_B allant de 0 à 1,2 mA (générateur I_{DC} (cocher la case courant si vous utilisez un VDC) ou encore un VDC en DC Sweep de 0 à 2 V en série avec une résistance de 1 kΩ).

E6. D'après la caractéristique de sortie, quelle est la condition pour que le transistor ne soit pas saturé ?

E7. Simuler le circuit pour une V_{CE} = 0,1 V, V_{CE} = 1 V et V_{CE} = 10 V et tracer les trois courbes un même graphique. Commenter les courbes.

E8. Relever les $\beta = \frac{I_C}{I_B}$ dans le domaine de linéarité de la courbe. D'après la datasheet du transistor, est-ce que ces valeurs vous semblent convenables ? Comment varie le gain en courant β avec V_{CE} ?

D. Exploitation des résultats

E9. Simuler un émetteur commun polarisé par pont avec V_{CC} = 15 V, R_1 = 780 Ω, R_2 = 220 Ω, R_C = 170 Ω et R_E = 100 Ω avec puis sans R_E afin de remplir le tableau suivant :

	i_{R1} [mA]	i_{R2} [mA]	i_B [mA]	i_C [mA]	i_E [mA]	V_B [V]	V_C [V]	V_E [V]	V_{BE} [V]	V_{CE} [V]	V_{BC} [V]
avec R_E											
sans R_E											

E10. Quel est l'état du transistor dans les deux cas de figure ? Dans les deux cas, placer le point de polarisation sur la caractéristique de sortie associée et donner le mode de fonctionnement du transistor.

TP n° 10

Analyse spectrale

Avant toute chose, nous allons nous assurer que l'on est capable d'observer le spectre en amplitude à l'oscilloscope de différents signaux vus en TD.

Qu'est-ce que le spectre en amplitude ?
Il s'agit de la représentation graphique du signal dans le domaine fréquentiel : pour chaque fréquence présente en abscisse est associé une raie d'une certaine amplitude (parfois nulle). Ainsi, un signal composité composé de deux sinusoides de fréquences et d'amplitudes différentes comme présenté sur la partie gauche de la Figure 10.1 a pour spectre pour les fréquences positives, deux raies correspondantes aux deux sinusoïdes.

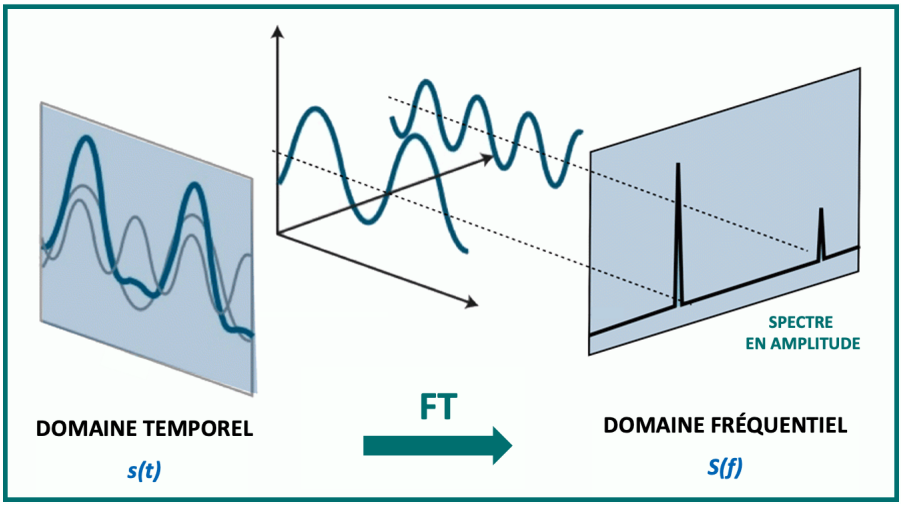


Figure 10.1 – Mise en évidence des deux raies d'un signal composé de deux sinusoïdes.

L'objectif de ce TP est d'observer le spectre de signaux périodique à l'aide d'un oscilloscope et de mettre en évidence la condition de Shannon pour un signal sous et correctement échantillonné.

I. Partie théorique

T1. En fonction de la periodicité des signaux, calculer la DSF ou la TF des fonctions sinus, porte, triangle et rectangle (centrés en 0).

Soit $x(t)$ à support spectral borné et $f_{\max}$ la fréquence maximale. On échantillonne $x(t)$ à $f_e = 2f_{\max}$ et on bloque chaque échantillon pendant une durée $T_e = 1/f_e$.

T2. Écrire le signal $y(t)$ échantillonné-bloqué puis calculer la transformée de Fourier.

II. Partie pratique

A. Mise en évidence de la décomposition de Fourier

Soit le filtre analogique présenté sur la Figure 10.2.

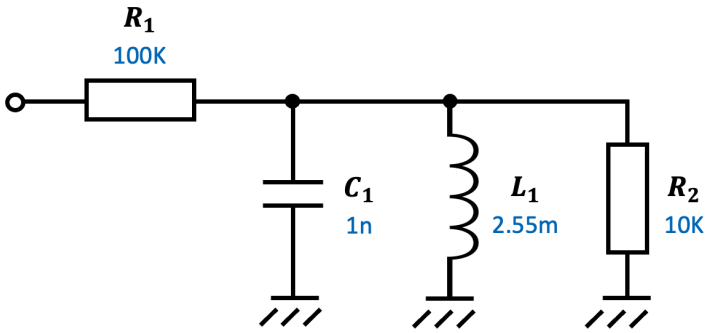


Figure 10.2 – Filtre à étudier

S1. À l'aide de Proteus, tracer le diagramme de Bode en gain et en phase. S'agit-il d'un filtre actif / passif ? Quel est l'ordre du filtre ? Quelle est la bande passante ? Le signal est-il amplifié ou atténué dans la bande passante ?

S2. Filtrer un signal rectangulaire de fréquence 100 kHz et d'amplitude crête à crête de 10 V (rapport cyclique de 0,5). Tracer la représentation temporelle du signal en sortie du filtre sur deux période. Tracer le spectre avant et après filtrage. Commentez vos résultats.

B. Spectres de signaux classiques

E1. À l'oscilloscope cette fois-ci, observer un signal rectangulaire de rapport cyclique 0,5, de fréquence 10 kHz et de 5 V d'amplitude.

E2. En utilisant la fonction FFT de l'osciloscope, visualiser le spectre de ce signal.

E3. Modifier le rapport cyclique afin qu'il vale à présente 1/4. Qu'observe-t'on ?

On règle à nouveau le rapport cyclique à 0,5. On souhaite comparer les spectres des signaux rectangulaire, sinusoïdal et triangulaire.

E4. Afficher les trois spectres à l'oscilloscope et les comparer.

C. Mise en évidence de la condition de Shannon

On souhaite observer le spectre d'un signal sinusoïdal d'amplitude 5 volts et de fréquence F=50 kHz.

E5. Observer le spectre de ce signal à l'oscilloscope.

E6. Augmenter la fréquence. Que se passe-t-il ? Interpréter le plus finement possible.

NB : Pour les laboratoires récents, n'hésitez pas à régler votre GBF sur des valeurs élevées en fréquence qui correspondent à la limite des oscilloscopes (ex : 30 voire 100 MHz pour certains) et observer le spectre sur l'oscilloscope (réglage à 50 MSa/s).

TP n° 11

Démodulation AM

I. Modulateur AM

Un signal AM (« Amplitude Modulation » ou « Modulation d'Amplitude » en français) est élaboré à l'aide d'un multiplieur selon le schéma ci-dessous :

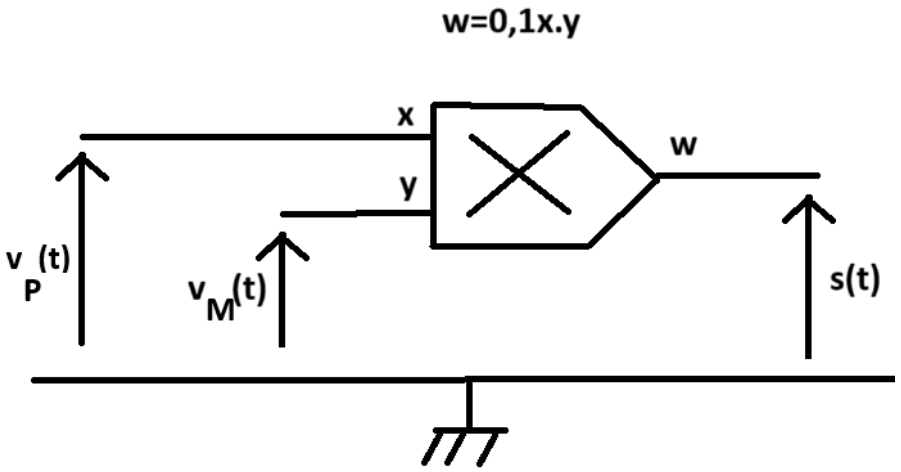


Figure 11.1 – Multiplieur

On donne les valeurs numériques suivantes :

- porteuse sinusoïdale : $V_P(t) = 5\cos(\omega_0 t)$ de fréquence $f_0 = \frac{\omega_0}{2\pi} = 250\,\text{kHz}$
- signal modulant : $V_M(t) = A + 5B\cos(\omega t)$ de fréquence $f = \frac{\omega}{2\pi} = 5\,\text{kHz}$

T1. Exprimer le signal modulé $s(t)$ sous la forme :

$$s(t) = A_P[1 + m_a\cos(\omega t)]\cos(\omega_0 t) \tag{11.1}$$

et préciser les expressions de l'amplitude de la porteuse A_P et de l'indice de modulation m_a.

T2. Pour un réglage des constantes A et B, on a obtenu le signal modulé AM de la Figure 11.2. Déterminer A_P et m_a.

T3. En déduire les valeurs des constantes A et B.

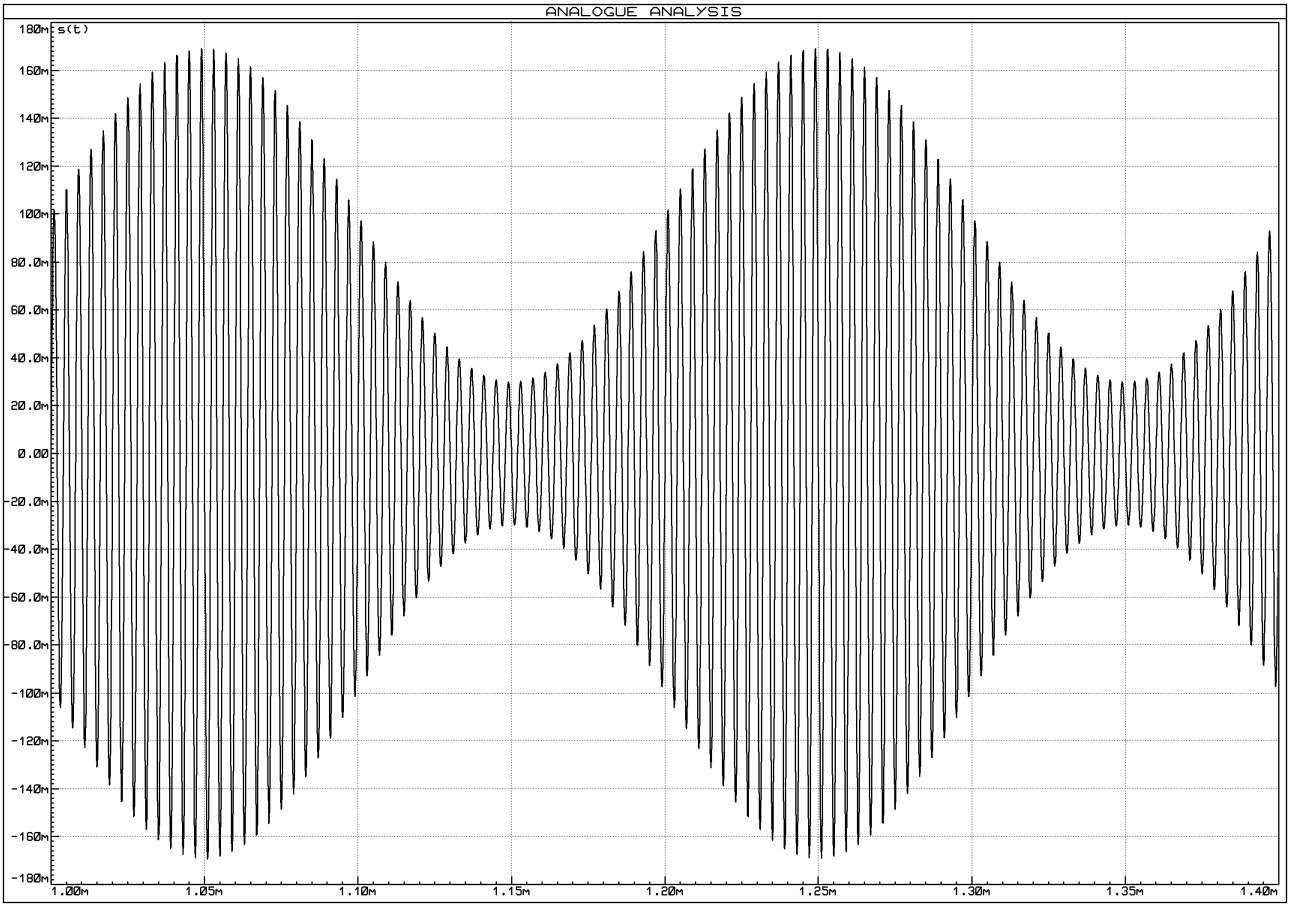


FIGURE 11.2 – Signal modulé AM

E1. Réaliser le montage sur Proteus qui permet à l'aide du multiplieur AD734 (qui peut monter jusqu'à une fréquence de 10 MHz dont la sortie est calculée selon :

$$W = A_0[\frac{(X_1 - X_2)(Y_1 - Y_2)}{U_1 - U_2} - (Z_1 - Z_2)] \tag{11.2}$$

(mettre VP à $+15\,\mathrm{V}$ et VN à $-15\,\mathrm{V}$; X_2 et Y_2 à la masse ; U_0, U_1 et U_2 à la masse ; la sortie W reliée à Z_1 ; et Z_2 un offset de 20 mV ; A_0 vaut typiquement 0,1) de retrouver la même expérience et le même graphique résultat que précédemment.

E2. A l'aide de Proteus, tracer le spectre du signal $s(t)$.

T4. Retrouver ce résultat à l'aide de la formule :

$$2\cos(a)\cos(b) = \cos(a-b) + \cos(a+b) \tag{11.3}$$

T5. Choisir vous-même des valeurs différentes du signal modulant et modulé, et tracer l'allure du signal obtenu.

E3. Retrouver ce résultat à l'aide de Proteus.

II. Démodulateur AM à détection synchrone

Un signal AM d'expression :

$$s(t) = A_p[1 + m_a \cos(\omega_m t)] \cos(\omega_p t) \tag{11.4}$$

est appliqué à l'entrée du multiplieur analogique de la Figure 11.3.

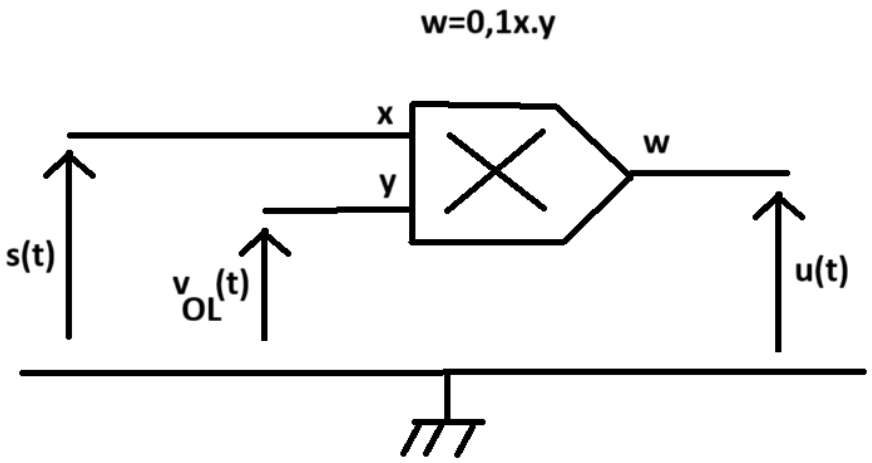


Figure 11.3 – Multiplieur analogique

Les caractéristiques du signal AM-P sont les suivantes :

- $A_p = 5\,\mathrm{V}$
- $f_p = 60\,\mathrm{kHz}$
- $f_m = 1\,\mathrm{kHz}$
- $m_a = 70\,\%$

On applique alors à la seconde entrée du multiplieur une tension sinusoïdale de même fréquence que la porteuse :

$$v_{OL}(t) = \cos(\omega_p t) \tag{11.5}$$

Le multiplieur fournit en sortie la tension :

$$u(t) = 0,1 s(t) v_{OL}(t) \tag{11.6}$$

On utilisera, étant données les basses fréquence ici considérées, le composant simple et bon marché multiplieur AD633. Mettre le signal modulant sur X1, X2 à la masse, le signal porteur sur Y1 et Y2 à la masse. En sortie, mettre Z (offset) à la masse et VS+ à $+15\,\mathrm{V}$ et VS- à $-15\,\mathrm{V}$. On a de façon générale :

$$W = (X1 - X2)(Y1 - Y2)/10 + Z \tag{11.7}$$

T6. Montrer que le spectre du signal $u(t)$ en sortie du multiplieur comporte cinq composantes dont vous déterminerez l'amplitude et la fréquence.

Rappel :

$$2\cos(a)\cos(b) = \cos(a - b) + \cos(a + b) \tag{11.8}$$

E4. Réaliser le montage sur Proteus permettant de créer le signal $s(t)$. Faire des copies d'écran du signal modulant, porteur, et la sortie $s(t)$ dans un premier temps ; ainsi que leurs spectres respectifs.

E5. En plaçant un second multiplieur pour finir le montage de la figure précédente, afficher $u(t)$ et son spectre. Confirmer qualitativement et numériquement le résultat de la question T1.

Le signal $u(t)$ traverse ensuite un filtre

1. Passe Bas

   ou

2. Passe Haut

selon le traitement souhaité.

T7. Lequel des deux filtres faut-il utiliser pour récupérer le signal modulant de fréquence $f_m = 1\,\text{kHz}$?

T8. Proposer une fréquence de coupure $f_{-3\,\text{dB}}$ pour le filtre retenu.

E6. Réaliser alors le montage sur Proteus (on pourra concevoir de façon très simple un filtre à partir de résistance et condensateur, dont on précisera les valeurs). Faire une copie d'écran du montage et du signal final (temporel et spectre). Obtient-on le résultat attendu d'une démodulation ?

T9. Comment faire pour supprimer la composante continue du signal final ?

E7. Conclure.

III. Démodulateur AM à détection d'enveloppe

Un signal audio fréquence est transmis par onde hertzienne avec une modulation d'amplitude d'une porteuse de fréquence $f_P = 60\,\text{kHz}$. On utilise un AD633 (vu à l'exercice précédent) pour créer la modulation, et le signal modulant est une somme de $V_{m1} = 5{+}3,5\cos(2\pi f_{m1}t)$ et de $V_{m2} = \cos(2\pi f_{m2}t)$ où $f_{m1} = 1\,\text{kHz}$ et $f_{m2} = 3\,\text{kHz}$ (on a utilisé un sommateur à AOP pour créer le signal modulant $V_m = V_{m1} + V_{m2}$). A la réception, on obtient la tension modulée $s(t)$ représentée sur l'enregistrement de la Figure 11.4

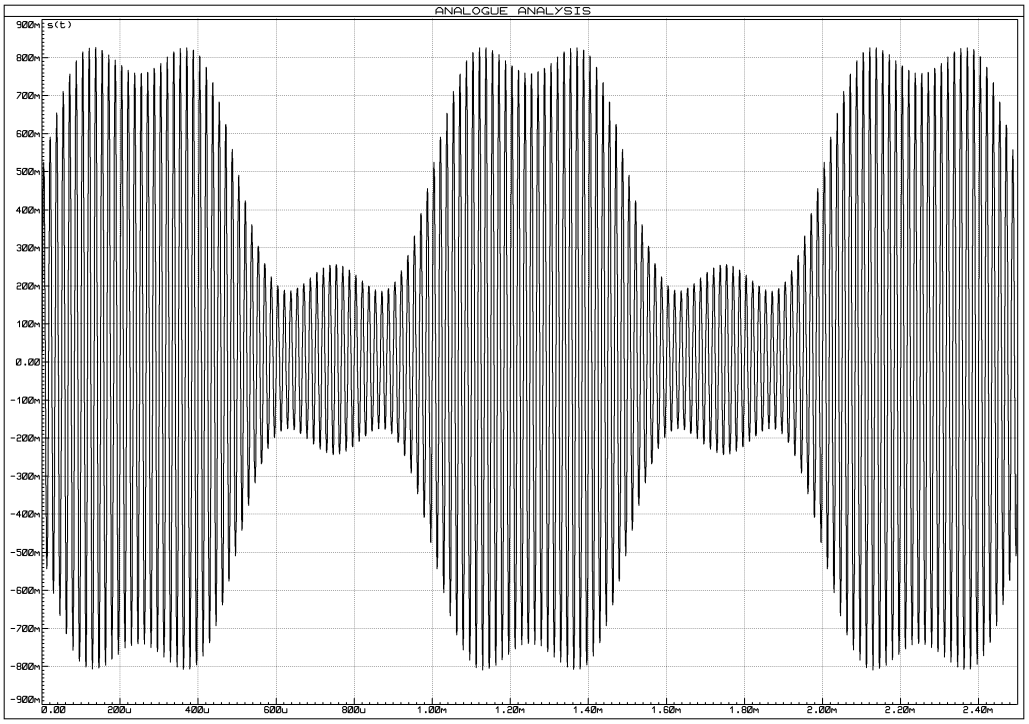


Figure 11.4 – Tension modulée reçue

La démodulation d'amplitude est réalisée à l'aide d'un détecteur à diode représenté sur la figure ci-dessous :

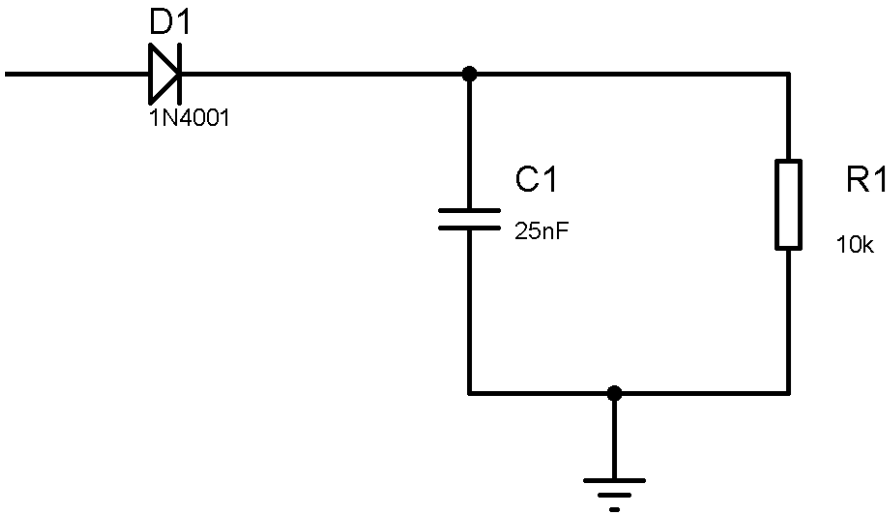


Figure 11.5 – Détecteur à diode

T10. De façon générale, quelle est la condition à respecter sur l'indice de modulation m_a pour pouvoir utiliser un tel démodulateur à diode ?

T11. Quelle condition doit satisfaire la constante de temps (qui vaut ici $\tau = R_1 C_1$) par rapport à la période T_P de la porteuse ? Par rapport à la période T_m du signal audio modulant ?

On a enregistré successivement la tension $s_d(t)$ pour deux diodes différentes, une de qualité « redressement » 1N4001 et une de qualité « signal » 1N4148, sur les deux Figures 11.6 et 11.7.

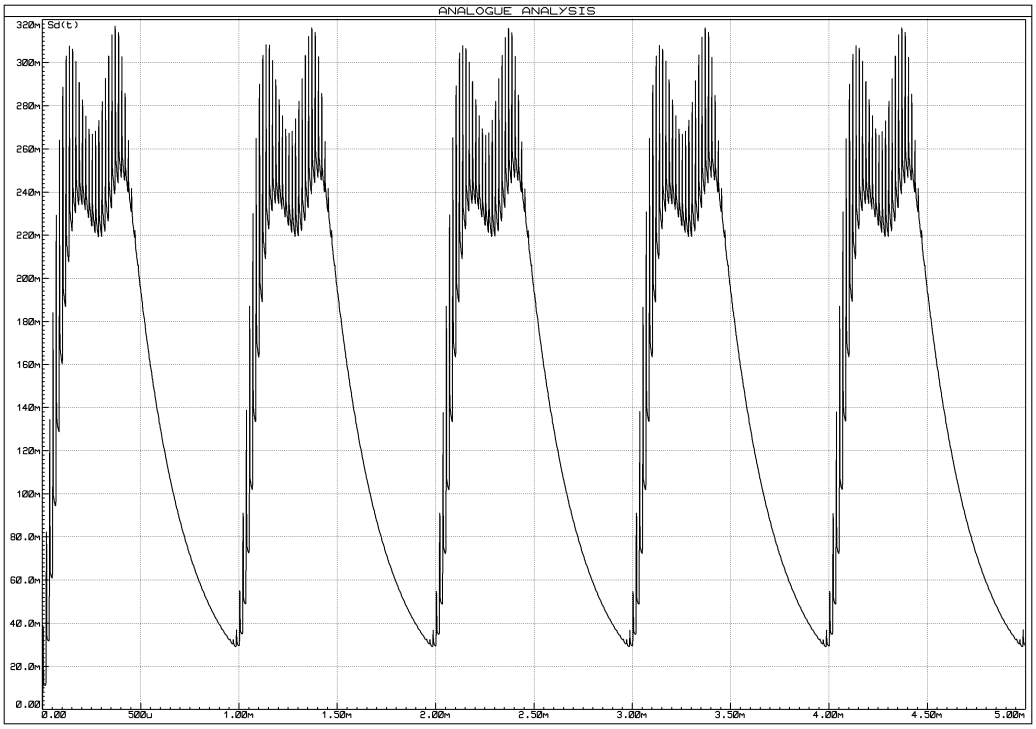


Figure 11.6 – Enregistrement A

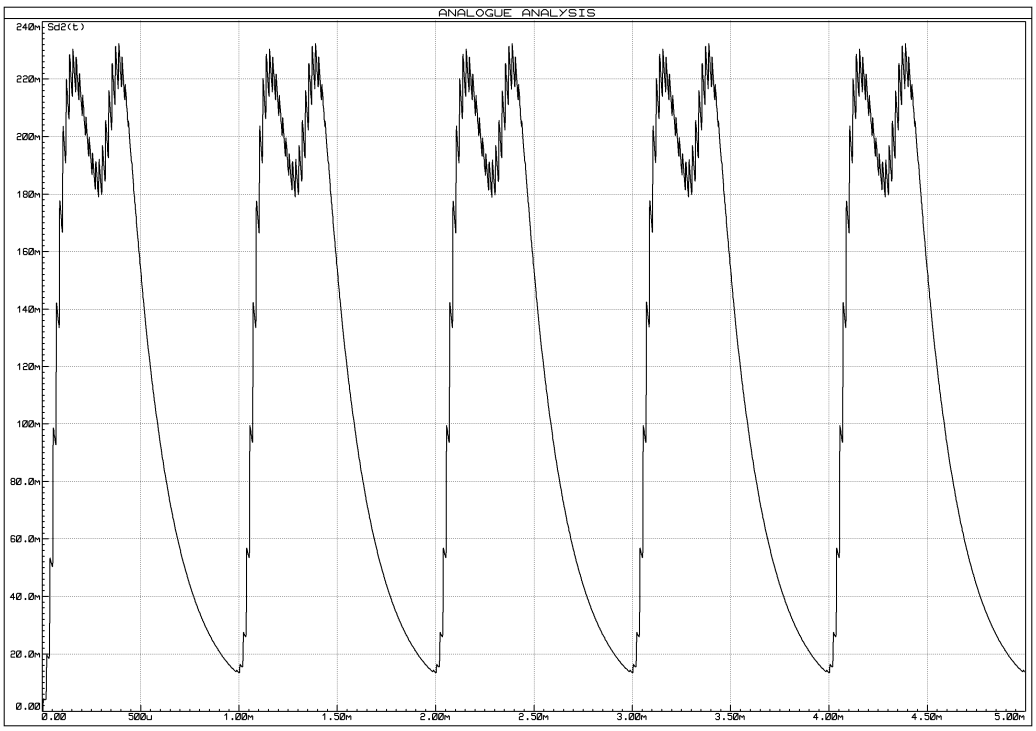


Figure 11.7 – Enregistrement B

T12. Par observation de ces deux courbes, attribuer à chacune d'elles la diode qui lui correspond.

E8. Retrouver ces résultats à l'aide de Proteus.

Pour supprimer l'ondulation résiduelle qui peut apparaitre, la structure réelle du démodulateur comporte un étage de détection d'enveloppe à diode suivi d'un filtre actif passe-bas du premier ordre de fréquence de coupure à $-3\,\text{dB}$ $f_c = 5\,\text{kHz}$

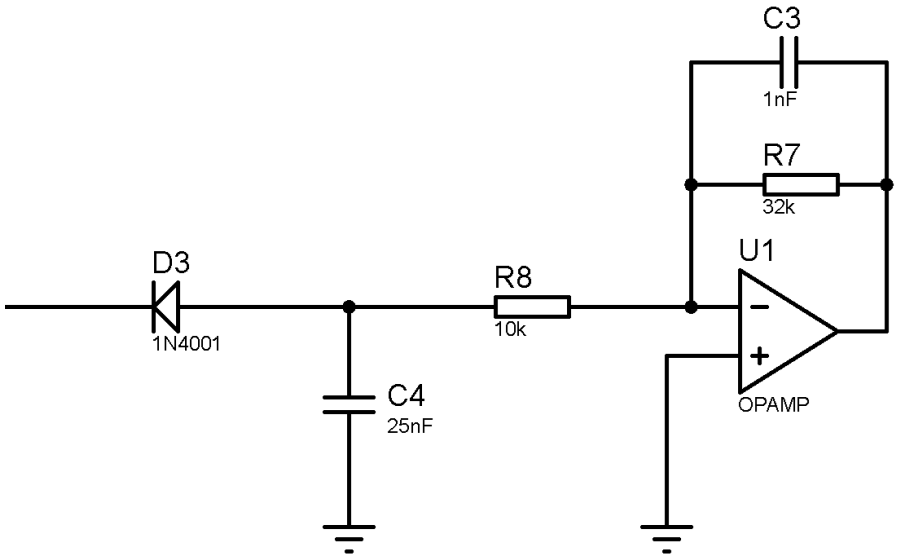


Figure 11.8 – Étage de détection d'enveloppe + filtre

T13. Justifier les valeurs de C_3 et R_7 choisies.

E9. Montrer que l'on obtient pour $s_F(t)$ le signal de sortie suivant :

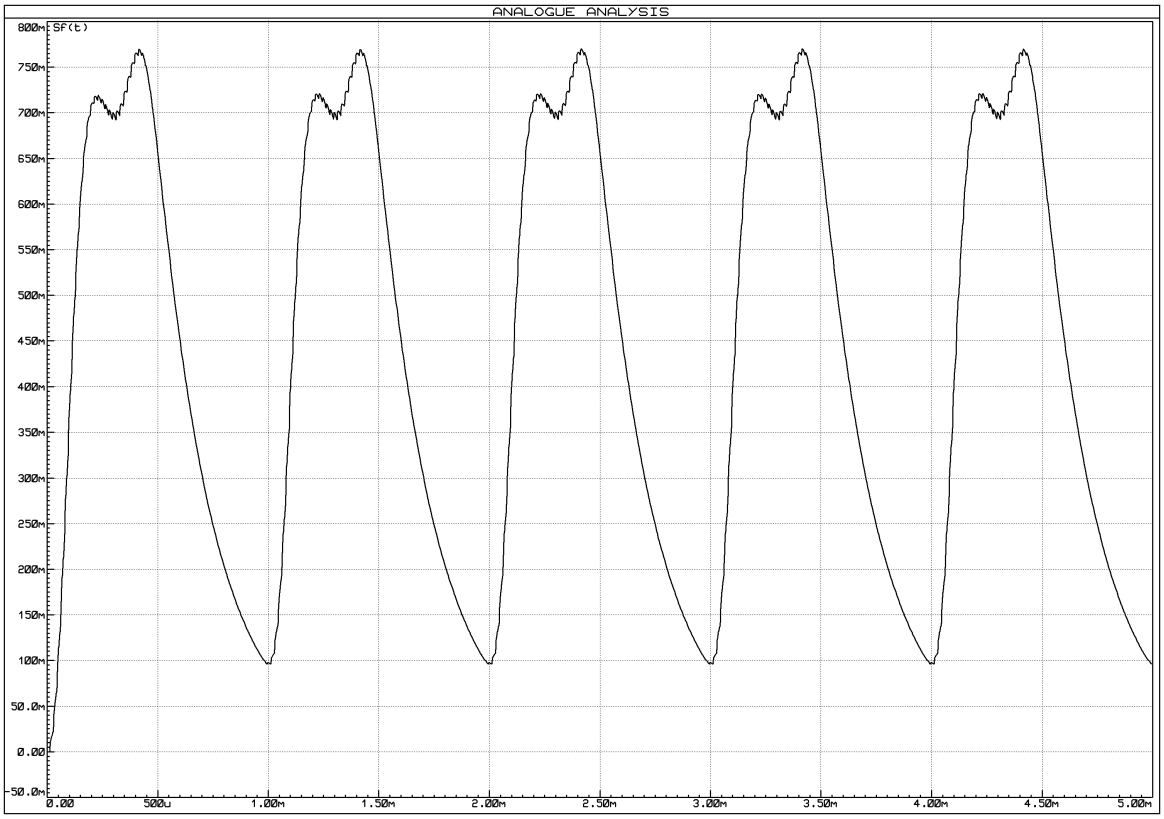


Figure 11.9 – Signal de sortie

TP n° 12

L'additionneur numérique et le multiplexeur

Motivation

Au CP, nous avons vu que l'addition de deux nombres pouvait se faire en « posant l'addition » où l'on note en exposant du chiffre de puissance $n+1$ une retenue s'il y en a une. En électronique, c'est la même chose : on additionne des nombres en suivant le même ordre (le dernier bit A_0 du premier nombre avec le dernier bit B_0 du deuxième nombre puis, l'avant dernier bit A_1 du premier nombre avec l'avant-dernier B_1 bit du deuxième nombre, etc.) mais en étant limité par des chiffres binaires : 0 ou 1, comme suit :

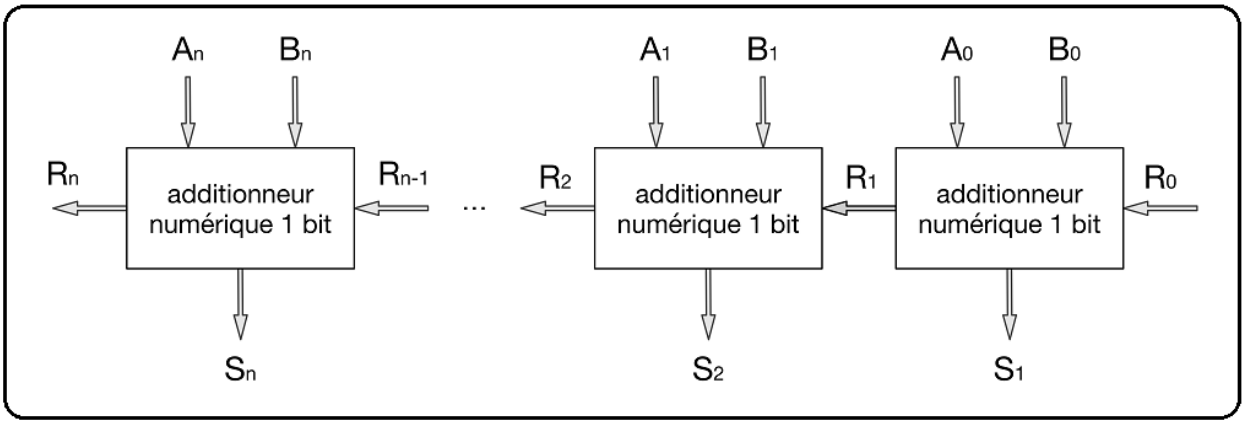


Figure 12.1 – Schéma électrique présentant l'addition de n bits par n additioneurs de 1 bit

La brique essentielle d'un additionneur numérique est donc le bloc « additionneur numérique 1 bit » dans la mesure où il suffit de le dupliquer autant de fois que l'on a de bits à additionner !

Mais comment implémente-t-on cet additionneur 1 bit ?

I. Partie théorique

On souhaite synthétiser un additionneur dont les entrées sont les bits A_n, B_n et la retenue de rang inférieur R_{n-1}. Les sorties sont S_n (Somme du $n^{ième}$ bit) et R_n (Retenue du $n^{ième}$ bit).

T1. Reproduire et remplir la table de vérité de cet additionneur.

A_n	B_n	R_{n-1}	R_n	S_n
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

T2. Établir les expressions de S_n et de R_n en fonction des entrées.

T3. Établir le schéma structurel de S_n et R_n sur un même circuit.

II. Simulations

A. Additionneur numérique

E1. Sur Proteus, simuler un additionneur numérique 1-bit et relever sa table de vérité en utilisant des générateurs PULSE et un graphique DIGITAL ANALYSIS. Commentez vos résultats en vérifiant que vous avez bien ce que vous souhaitez pour au moins deux combinaisons.

Remarque : Vous pouvez utiliser directement le circuit suivant :

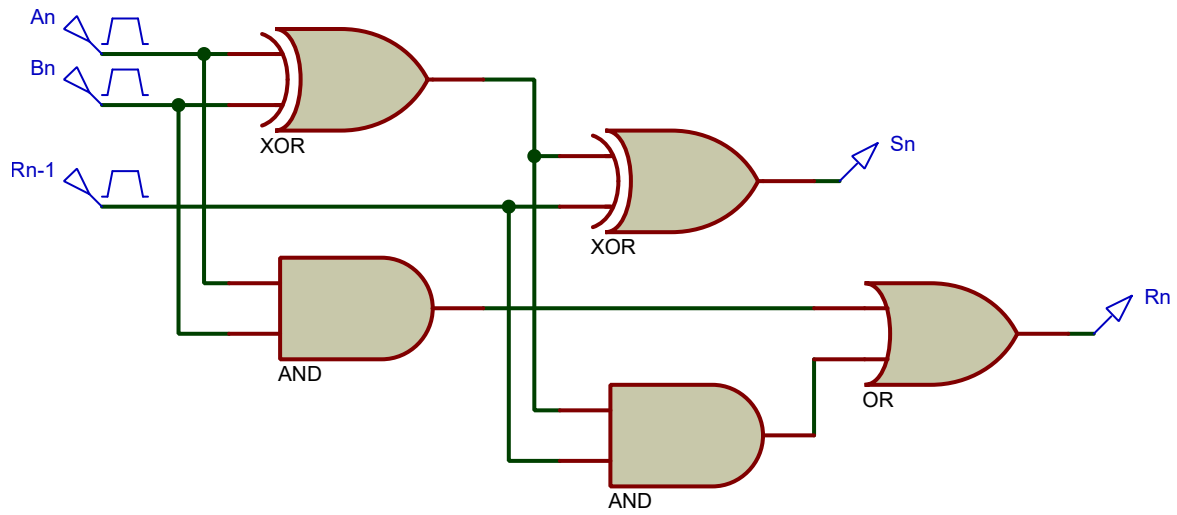


Figure 12.2 – Schéma de l'additionneur 1 bit

E2. En utilisant la fonctionnalité de copie (clic droit « block copy ») de Proteus, mettre en évidence l'addition de deux nombres codés sur 3 bits. Vous expliquerez votre raisonnement en utilisant des logicstate et logicprobe, et vous vérifierez également que le calcul est correct en faisant le calcul en écriture décimale.

B. Simulation d'un multiplexeur 2 → 1

Nous allons nous intéresser dans cette partie aux multiplexeurs.

Le multiplexeur 2 → 1 permet de sélectionner une information parmi deux. Il a donc un seul bit de sélection SEL, une sortie OUT et deux entrées : IN_0 et IN_1. Son équation s'écrit :

$$OUT = IN_0 \cdot \overline{SEL} + SEL \cdot \overline{IN_1}$$

Sa table de vérité est :

SEL	OUT
0	IN_0
1	IN_1

E3. Simuler le MUX 2 → 1 présenté sur la figure et vérifier son bon fonctionnement.

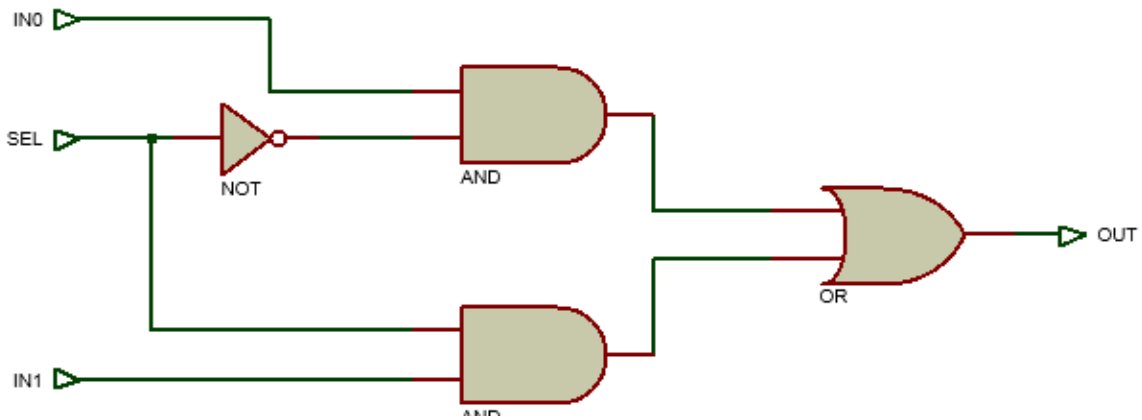


Figure 12.3 – Schéma électrique du multiplexeur 2 → 1

C. Simulation d'un multiplexeur 4 → 1

E4. Etablir la table de vérité d'un multiplexeur 4 → 1 comme suit :

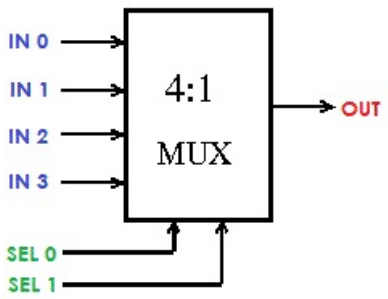


Figure 12.4 – schéma électrique du multiplexeur 4 → 1

E5. Déduisez-en son équation booléenne et simuler votre circuit.

E6. Tester le fonctionnement du double multiplexeur 74153. Peut-on réaliser un multiplexeur 8 → 1 avec un unique double multiplexeur 4 → 1 ? Si oui, proposer une simulation et commenter vos résultats.

E7. Si l'on désire établir un multiplexeur pour 100 entrées, combien de bits doit on prévoir pour recevoir le mot binaire contenu dans le SEL (SE_L0, SEL_1, ... SEL_n) ?



L'essentiel

- Simulation DIGITAL ANALYSIS
- Différence entre la logique combinatoire et la logique séquentielle, savoir donner des exemples
- Conception d'un circuit numérique
  - Fonctionnalité : table de vérité
  - Équation booléenne et simplification de cette dernière
  - Circuit logique : identification des entrées et des sorties puis connecter des portes logiques en accord avec l'équation
- Simulation
  - LOGIC STATE (entrées statiques) et LOGIC PROBE (sorties statiques)
  - DIGITAL ANALYSIS pour afficher un chronogramme
- Multiplexeur : rôle et équation logique

TP nº 13

Le diviseur d'horloge

Lors du TP « Le compteur asynchrone », nous avons vu comment concevoir un compteur numérique. Pour peu que l'entrée de la première bascule soit cadencée par un signal périodique, un compteur peut permettre de le ralentir.

Typiquement, pour fonctionner, un microcontrôleur a besoin d'une alimentation, d'un code dans sa mémoire de programme (Flash) et d'un signal d'horloge, permettant ainsi de cadencer l'exécution des instructions contenues dans le programme.

En regardant la NeuroBoard S3 de plus près, on remarquera que l'horloge est donnée par un crystal de quartz logé juste à côté du microcontrôleur. Il est cadencé à 40 MHz, soit un temps caractéristique de l'ordre de la dizaine de nanoseconde ! Afin d'effectuer des instructions d'un ordre de grandeur plus adapté pour l'humain, il est donc nécessaire de recopier et de ralentir cette horloge. C'est justement le but des diviseurs d'horloge.

Au cours de ce TP, nous allons prendre un compteur 4-bit et l'utiliser comme un diviseur d'horloge. Le signal rectangulaire à ralentir émanera d'une pin PWM de la NeuroBoard S3 (GPIO 05 à 980 Hz). En sortie des différentes bascules du compteur, on obtiendra ce même signal ralenti d'un facteur 2, 4, 8 ou 16 et un multiplexeur nous permettra de choisir le signal voulu, qui lui-même alimentera un buzzer afin de jouer des sons différents, comme illustré sur la Figure 13.1

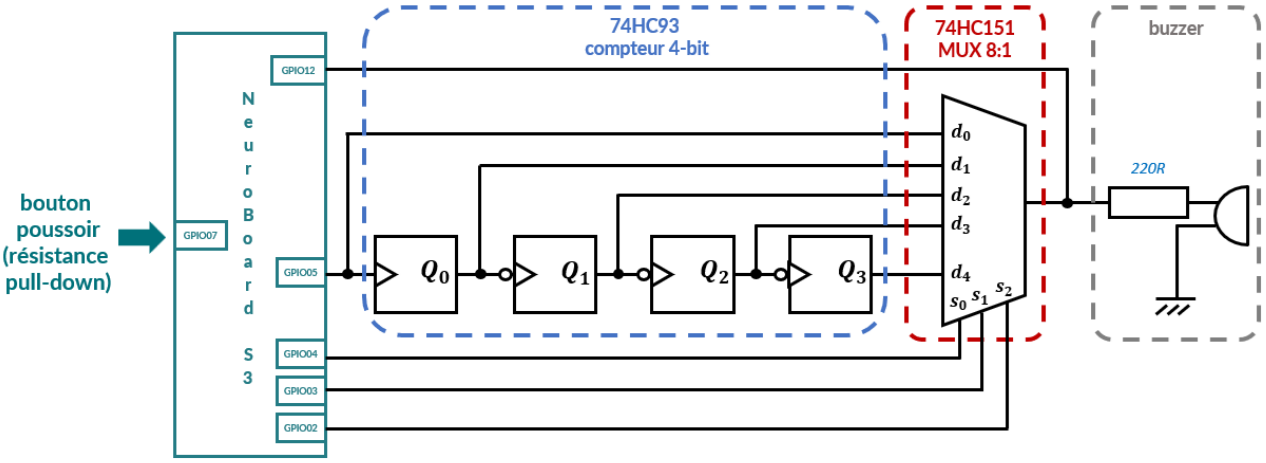


Figure 13.1 – Circuit permettant de ralentir un signal PWM d'une NeuroBoard S3 par des multiples de 2, et d'alimenter un buzzer.

Le but de ce TP est de comprendre le fonctionnement d'un diviseur d'horloge et de s'entrainer à câbler deux circuits intégrés : le compteur 4-bit 74HC93 et le multiplexeur 8 vers 1 74HC151.

I. Étude de datasheets

A. Le compteur binaire 74HC93

Le circuit logique interne du 74HC93 est présenté sur la Figure 13.2.

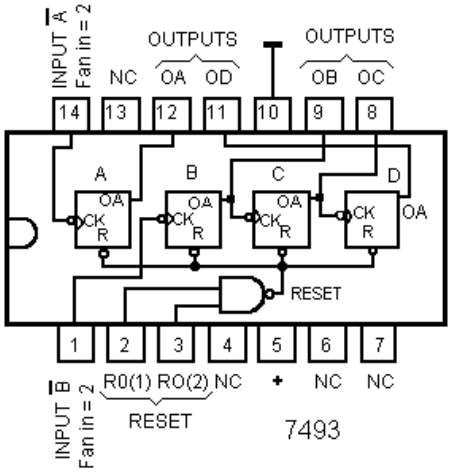


Figure 13.2 – Extrait de la datasheet du 74HC93 : circuit logique interne

T1. Expliquer pourquoi tel quel ce circuit ne permet pas de compter sur 4 bits. Discuter le fait d'avoir deux entrées complémentées INPUT $\overline{A}$ et INPUT $\overline{B}$. Comment peut-on obtenir un compteur 4-bits tel que présenté sur la Figure 13.1 ?

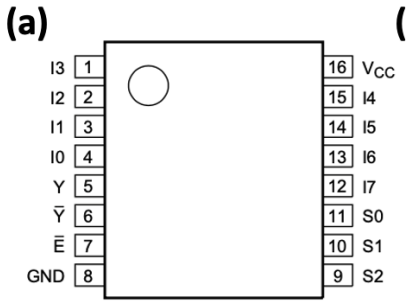
T2. Rappeler la table de vérité de la porte NAND et expliquer comment on peut réinitialiser le compteur.

Consulter la *datasheet* du 74HC93.
T3. Comment l'alimente t-on ? Quelle est la plage de valeur en tension recommandée ?

T4. Proposer un schéma de branchement (en prenant en compte l'alimentation du circuit intégré 74HC93) de sorte à obtenir le circuit de la Figure 13.1 et vous préparer à la partie pratique du TP. Vous présenterez votre travail sous forme de tableau qui indiquera quelle pin doit être connectée à quel élément du circuit.

B. Le multiplexeur 74HC151

La Figure 13.3 présente le brochage du multiplexeur 8 vers 1 74HC151.



(b)

Symbol	Pin	Description
I0 to I7	4, 3, 2, 1, 15, 14, 13, 12	data inputs
Y	5	multiplexer output
$\overline{Y}$	6	complementary multiplexer output
$\overline{E}$	7	enable input (active LOW)
GND	8	ground (0 V)
S0, S1, S2	11, 10, 9	common data select inputs
V_{CC}	16	supply voltage

Figure 13.3 – Extrait de la datasheet du 74HC151 : **(a)** brochage et **(b)** description des pins

T5. Proposer un schéma de branchement (en prenant en compte l'alimentation du circuit intégré 74HC151) de sorte à obtenir le circuit de la Figure 13.1 et vous préparer à la partie pratique du TP. Vous présenterez votre travail sous forme de tableau qui indiquera quelle pin doit être connectée à quel élément du circuit.

La Figure 13.4 présente le circuit interne du 74HC151.

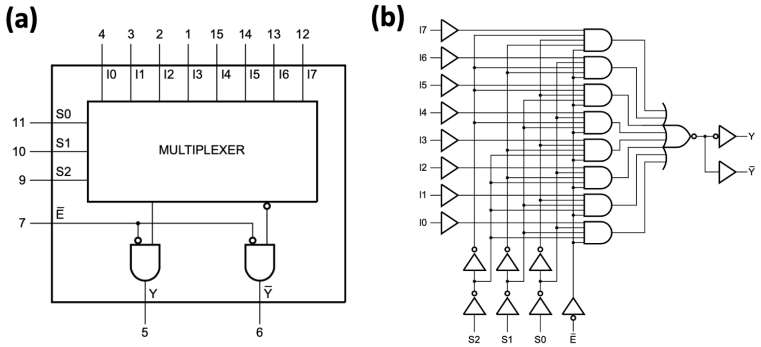


Figure 13.4 – Extrait de la datasheet du 74HC151 : **(a)** diagramme fonctionnel et **(b)** Logigramme.

T6. À quoi peut bien servir la sortie complémentée $\overline{Y}$?

T7. À quoi peut bien servir l'entrée complémentée $\overline{E}$?

II. Câblage et test du circuit

E1. Câbler et tester le circuit de la Figure 13.1.

Quelques précisions :

- à chaque appui sur le bouton, on incrémente la combinaison envoyée au select du multiplexeur ;
- après avoir appuyé sur le bouton, on exécute une instruction `delay` de 100 ms de sorte à éviter les rebonds ;
- pensez à bien câbler les deux premières bascules du compteur (elle ne sont pas reliées dans le 74HC93) ;

- la PWM émanant de la broche GPIO 05 de NeuroBoard S3 a une fréquence d'environ 980 Hz. Pour générer ce signal de 980 Hz sur la broche GPIO 05, il est nécessaire de configurer le périphérique PWM (LEDC) de l'ESP32-S3 via le code suivant 13.1 :

```
const int PWMPin = 5;
const int canalPWM = 0;
const int freq = 980;
const int resolution = 8; // Résolution 8 bits (0-255)
    void setup() {
  ledcSetup(canalPWM, freq, resolution);
  ledcAttachPin(PWMPin, canalPWM);
  ledcWrite(canalPWM, 128); // Signal carré (rapport cyclique 50%, i.e. valeur 128
de 255)
}
void loop() {// Le signal est géré par le matériel}}
```

Code 13.1 – Code Arduino IDE ou VSCode Platformio pour Configurer signal PWM à 980 Hz sur le GPIO 05

E2. Commencer par coder le signal PWM de la pin 5, et relever les signaux à l'aide de l'oscilloscope pour vérifier chaque sortie des bascules : Q0, Q1, Q2 et Q3 du diviseur d'horloge. Commenter chaque relevé.

E3. Expliquer la fonctionnalité d'incrément de SELECT par appui sur le bouton. (on utilisera un `switch` pour le coder).

E4. Proposer un code pour faire fonctionner le montage complet.

E5. Mettre en évidence le bon fonctionnement à la sortie du multiplexeur. (Utiliser l'oscilloscope pour la sortie du multiplexeur, et le moniteur série pour l'appui sur le bouton).

Le bouton permet de changer la fréquence de la PWM envoyée au buzzer et donc *in fine* la fréquence du son émis. Afin de changer le volume sonore, on peut changer le rapport cyclique de la PWM, de sorte à transmettre plus ou moins de puissance.

E6. Ajouter un potentiomètre permettant de sélectionner un rapport cyclique entre 0 et 1 et donc de modifier le volume sonore en sortie du buzzer. Quelles modifications faut il apporter au code ? Relever les mesures à l'oscilloscope pour deux signaux de rapport cyclique différent.

L'essentiel

- Utilisation d'un compteur pour ralentir un signal rectangulaire
- Lecture d'une datasheet : alimentation et câblage de circuits intégrés
- Notion de Timers dans la NeuroBoard S3 : des registres particuliers qui s'incrémentent à un multiple de deux de l'horloge du microcontrôleur indépendamment du processeur.

TP n° 14

Interruptions externes

Motivation

Jusqu'à présent, nous avons vu que la carte NeuroBoard S3 travaille avec des mots binaires et qu'elle possède des entrées/sorties numériques dont plusieurs sont connectées à ses convertisseurs analogique-numérique internes (ADC1 et ADC2).

Aujourd'hui, nous allons piloter un servomoteur et choisir l'angle d'orientation de la pale à l'aide d'un potentiomètre. Le circuit étudié correspondra donc à celui présenté sur la Figure 8.1. Pour ce faire, nous allons utiliser la fonction `analogWrite()` qui permet de générer des signaux PWM (via le périphérique matériel LEDC de l'ESP32-S3).

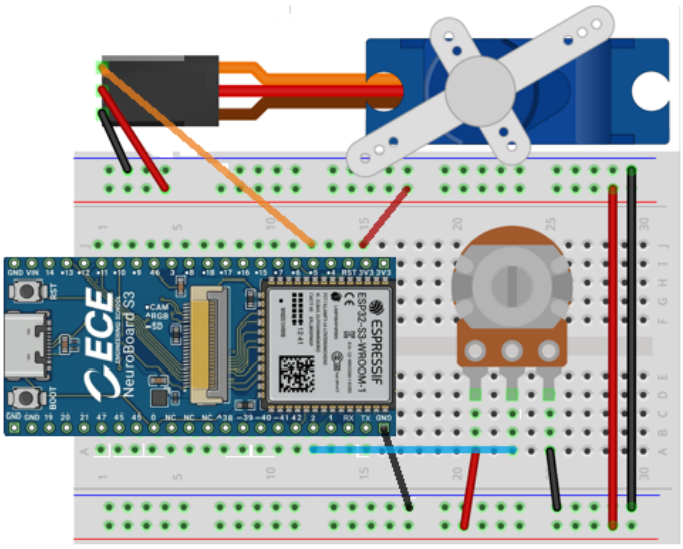


Figure 14.1 – Circuit permettant de contrôler un servomoteur à l'aide d'un potentiomètre et d'une NeuroBoards S3

Dans un second temps, nous allons utiliser une fonction d'**interruption** qui empêchera le contrôle du servomoteur tant qu'elle sera appelée. Le bon fonctionnement de l'appel de cette fonction d'interruption sera mis en évidence à l'aide d'un tracé « en temps réel » de la valeur de la consigne d'angle en fonction du temps.

Le but de ce TP est d'apprendre à piloter un servomoteur à l'aide d'un potentiomètre et de comprendre l'intérêt d'utiliser des fonctions d'interruption.

I. Partie théorique

A. Pilotage d'un servomoteur

Le servomoteur se branche en utilisant trois fils :

- une ligne d'alimentation et une ligne de masse permettant d'alimenter le moteur ;
- une ligne d'ordre permettant d'envoyer l'angle souhaité : la **consigne**, sous forme de signal rectangulaire de telle sorte à ce que le temps à l'état haut soit proportionnel à l'angle souhaité, comme illustré sur la Figure 14.2.

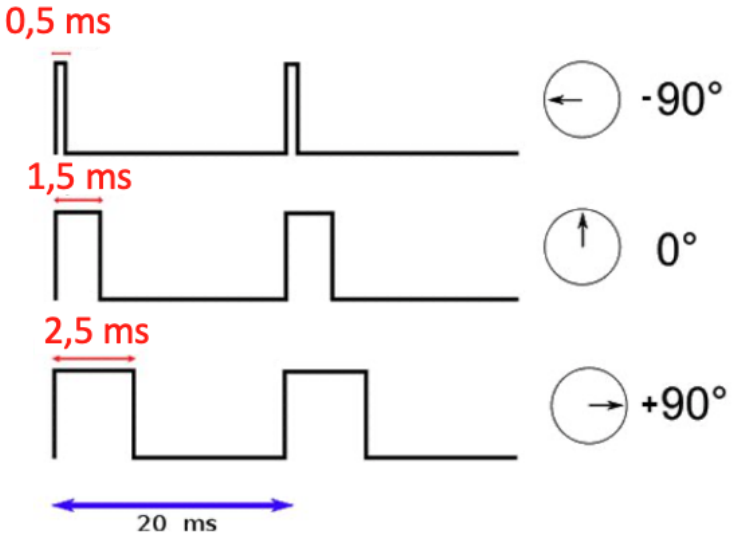


Figure 14.2 – L'angle de la pale du moteur est proportionnel au temps à l'état haut du signal de commande.

T1. Connaissant la période des signaux PWM représentés sur la Figure 14.2, en déduire la fréquence du signal PWM.

T2. Dire depuis quelles pins on peut les générer. Donner le numéro des pins concernées.

Le fonctionnement du servomoteur SG90 est basé sur l'**asservissement** d'un moteur à courant continu : la consigne est envoyée au moteur afin qu'il se règle sur l'angle souhaité mais dans un même temps, un capteur détecte sa position réelle afin d'ajuster cette consigne et être sûr que le moteur se positionne bien à l'angle souhaité. Ce système asservi peut être modélisé par le **schéma bloc** présenté sur la Figure 14.3.

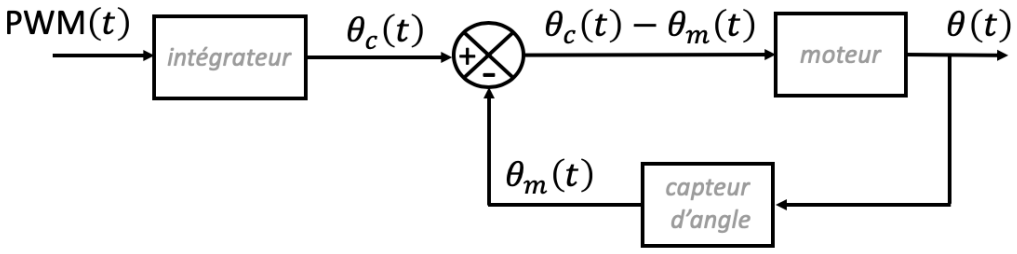


Figure 14.3 – Schéma bloc simplifié de l'asservissement d'un moteur (servomoteur).

Pour ce TP, nous allons nous contenter d'utiliser la bibliothèque `ESP32Servo.h` (adaptée à la Neuro-Board S3 et disponible via le gestionnaire de bibliothèques) dont la documentation et les méthodes

sont similaires à celles de la bibliothèque officielle Arduino.

B. Les fonctions d'interruption

Les fonctions d'**interruption** permettent la suspension temporaire de l'exécution d'un programme afin d'exécuter une tâche prioritaire.

Dans un code « classique », le déclenchement d'une fonction se fait lorsqu'un test (`if(...)`) renvoit une valeur positive. Cependant, avant d'arriver à ce test `if(...)`, il faut parfois attendre plusieurs lignes de code, ce qui implique un retard de traitement, comme illustré sur la Figure 14.4.

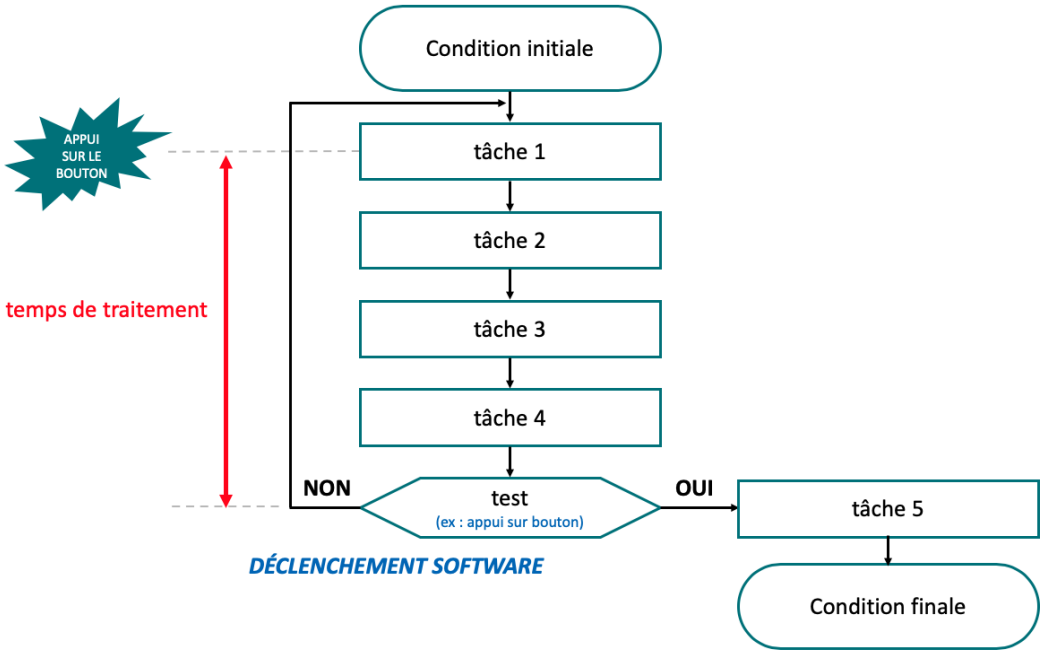


Figure 14.4 – Exemple d'algorigramme de test d'appui sur bouton classique. Le traitement ne se fait qu'une fois la ligne de code de test atteinte.

Le recours à des fonctions d'interruption permet alors de répondre à cette problématique. De façon automatique (test avec des circuits logiques internes au microcontrôleur), des fonctions dites d'interruption peuvent être déclenchées en réponse à un signal électrique (hardware), comme illustré sur la Figure 14.5.

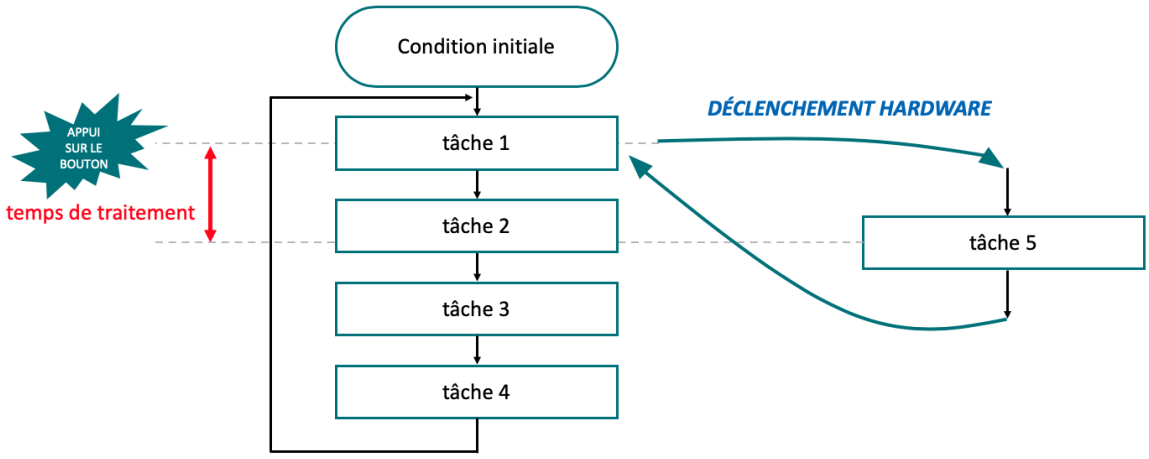


Figure 14.5 – Exemple d'algorigramme de test d'appui sur bouton avec fonction d'interruption. Une fois le bouton appuyé, la tâche en cours est interrompue et le traitement est lancé. Le temps de traitement est donc optimisé.

Ce type de fonction est implémentable sur les microcontrôleurs, dont l'ESP32-S3 de la NeuroBoard S3 (sur laquelle quasiment l'ensemble des broches GPIO peuvent servir d'interruption externe). Un exemple de code permettant d'inverser l'état logique d'une pin est présenté sur le Code 14.1. Le code commence classiquement par paramétrer le microcontrôleur (procédure `setup()`) avant d'entrer dans une boucle infinie : la `void loop()`. Lorsqu'il y a un changement d'état sur la broche configurée (par exemple le `GPIO 06`), la fonction d'interruption `blink()` est appelée et l'état logique de la variable `state` est inversé. Une fois la fonction d'interruption exécutée, le programme revient dans la `void loop()`.

```
#include <NeuroBoardS3.h>
const int pinLed = 3;        // GPIO3 connecté à la LED
const int pinBouton = 6;     // GPIO6 connecté au bouton-poussoir
volatile int state = LOW;    // Déclaration d'une variable volatile pour l'ISR

// Fonction d'interruption externe (ISR) exécutée en IRAM pour l'ESP32-S3
void IRAM_ATTR blink() {
  state = !state;                 // Inverse l'état de la variable
  digitalWrite(pinLed, state);    // La LED reflète l'état de la variable
}

void setup() {
  pinMode(pinLed, OUTPUT);
  pinMode(pinBouton, INPUT);      // Entrée configurée (pull-down externe dans le circuit)
  // Attache l'interruption au GPIO6 avec la fonction digitalPinToInterrupt
  attachInterrupt(digitalPinToInterrupt(pinBouton), blink, CHANGE);
}
void loop() {
  // Le processeur reste libre pour exécuter d'autres tâches
}
```

Code 14.1 – Code permettant d'inverser l'état logique d'une pin à l'aide d'une routine d'interruption ici en détectant un changement d'état sur la GPIO 06 pour allumer une LED sur la GPIO 03

Remarque : Sur la NeuroBoard S3 (ESP32-S3), quasiment toutes les broches GPIO peuvent être configurées comme interruptions externes. Il n'est donc pas nécessaire de se limiter aux pins 06 et 03 comme sur l'Arduino Nano.

II. Partie expérimentale

A. Pilotage d'un servomoteur

Soit le code suivant :

```
#include <NeuroBoardS3.h>
const int analogInPin = 2; // GPIO 2 sur la NeuroBoard S3
int sensorValue = 0;
int percent = 0;

void setup() {
  pinMode(analogInPin, INPUT);
  Serial.begin(115200); // Vitesse standard pour l'ESP32
}
void loop() {
  sensorValue = analogRead(analogInPin);
    // Mappage de la résolution 12 bits (0-4095) vers le pourcentage (0-100)
  percent = map(sensorValue, 0, 4095, 0, 100);
    Serial.println(percent);
  delay(1000);
}
```

Code 14.2 – Code mystère

T3. Pour chaque signal PWM représenté sur la Figure 14.2, donner, en utilisant la bibliothèque Servo, la ligne de code permettant de le générer.

E1. Expliquer de façon claire et précise l'utilité de ce code (maximum 2 lignes).

E2. À l'aide d'une NeuroBoard S3 et d'un potentiomètre, tester le Code 14.2. Afficher l'angle du potentiomètre sous forme de pourcentage et l'afficher dans le moniteur série. Ensuite, utiliser Téléplot pour l'afficher graphiquement.

E3. Nous allons à présent contrôler un servomoteur à l'aide de la NeuroBoard S3. Se référer à la datasheet du SG90 et associer à chaque fil de couleur un nom de signal et une description succinte puis brancher le servomoteur au GPIO 5 (Sur la NeuroBoard S3, grâce au périphérique matériel LEDC de l'ESP32-S3, la totalité des broches GPIO peuvent générer des signaux rectangulaires PWM - Modulation de largeur d'impulsion / *Pulse Width Modulation*).

E4. Tester le Code 14.3 puis le modifier afin de pouvoir piloter l'angle du servomoteur à l'aide du potentiomètre : position de 0° pour une valeur lue sur A0 de 0, position de 180° pour une valeur lue sur A0 de 1023 (utiliser la fonction `map`).

```
#include <NeuroBoardS3.h>
#include <ESP32Servo.h> // Bibliothèque Servo adaptée pour ESP32

Servo monServomoteur;
const int pinServo = 5; // Définition du GPIO 05 pour le signal PWM

void setup() {
  // Association du servomoteur au GPIO 5
  monServomoteur.attach(pinServo);
}

void loop() {
  // Fait bouger le bras de 0° à 180°
  for (int position = 0; position <= 180; position++) {
```

```
    monServomoteur.write(position);
    delay(15);
  }

  // Fait bouger le bras de 180° à 0°
  for (int position = 180; position >= 0; position--) {
    monServomoteur.write(position);
    delay(15);
  }
}
```

Code 14.3 – Code permettant de contrôler un servomoteur

E5. Relever à l'osciloscope le signal envoyé au servomoteur pour deux angles différents (utiliser les curseurs de l'oscilloscope).

Pour différents angles (une dizaine), tracer la durée du temps haut t_H du signal en fonction de l'angle désiré.

Commenter les résultats (valeurs minimale et maximale notamment) et trouver une relation entre ces deux grandeurs.

E6. Vérifier votre relation entre l'angle et le temps à l'état haut du signal rectangulaire pour deux angles vus en question E5.

B. Fonctions d'interruption

E7. Câbler un bouton en pull-down sur une des pins numériques configurée en entrée et implémenter une fonction d'interruption permettant d'interrompre le programme **Sweep** tant que le bouton est appuyé. Mettre en évidence le bon fonctionnement de la fonction d'interruption en allumant une LED dans la fonction d'interruption. Utiliser Téléplot pour montrer le bon fonctionnement. Proposer un code.

E8. Proposer et tester un code qui permet de compter le temps écoulé entre deux appuis sur le bouton, et en utilisant une fonction d'interruption. Quel est l'avantage d'utiliser une fonction d'interruption ici ?

L'essentiel

- Le servomoteur est un moteur asservi en angle. On le commande à l'aide d'un signal rectangulaire et son angle, compris entre 0° et 180° pour le SG90, est proportionnel au temps à l'état haut du signal de commande.
- L'interruption est une fonction pouvant être appelée automatiquement par le microcontrôleur et permettant d'interrompre la boucle principale pour traiter des tâches critiques. En particulier, le mécanisme de fonction d'**interruption externe** regarde le changement d'état d'un des ports du microcontrôleur pour déclencher une fonction dite d'interruption.

TP nº 15

Timers et GPIO de l'ESP32-S3

Motivation

La carte NeuroBoard S3 est une carte de développement basée sur un module ESP32-S3-WROOM-1, intégrant le microcontrôleur (SoC) ESP32-S3 32 bits. Celui-ci possède un processeur double cœur Xtensa® LX7 32 bits. À la différence d'un microprocesseur seul, un microcontrôleur intègre sur une même puce le processeur ainsi que de nombreux périphériques. L'ESP32-S3 comprend notamment de la mémoire interne (ROM et SRAM), de la mémoire Flash externe intégrée au module, des convertisseurs analogique-numérique (ADC), des modules de communication sans fil (Wi-Fi et Bluetooth Low Energy), des interfaces de communication série (SPI, UART, I^2C, I^2S, USB), ainsi que des timers matériels 54 bits et des périphériques dédiés à la génération de signaux PWM, notamment le LEDC.

Dans ce TP, nous allons en particulier voir comment sont gérés les **registres 32 bits** pilotant les GPIO (entrées/sorties) du SoC ESP32-S3, comment modifier plus rapidement l'état de ces GPIO via les registres de type `W1TS` et `W1TC`, puis étudier plus en détail les **timers matériels 54 bits** afin d'implémenter un autre type de fonction d'interruption : **les interruptions sur timer**. Ces dernières permettent d'exécuter des fonctions à intervalles de temps réguliers en interrompant temporairement l'exécution « principale » du programme.

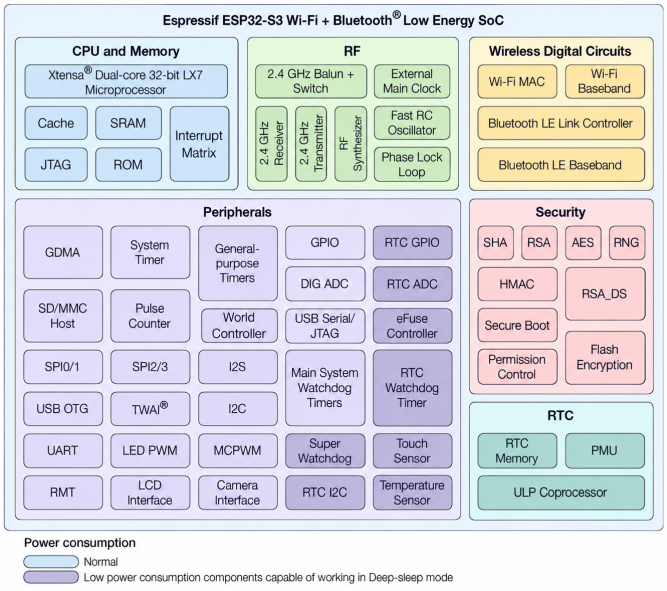


Figure 15.1 – Architecture du microcontrôleur ESP32-S3

Avant de lire la suite, prenez le temps de repérer les différents blocs énumérés plus haut. Notez que les instructions du programme sont stockées dans la mémoire Flash. Le processeur Xtensa® LX7 utilise un registre appelé **compteur ordinal** (*Program Counter*, PC), qui contient l'adresse de l'instruction à exécuter.

L'ESP32-S3 possède une **architecture 32 bits** : les registres principaux du processeur sont sur 32 bits et celui-ci manipule naturellement des données de 32 bits. La mémoire SRAM est, quant à elle, utilisée pour stocker les données nécessaires à l'exécution du programme.

Il est temps d'appliquer ce que vous avez vu en cours sur le microcontrôleur afin de mieux comprendre son fonctionnement et d'optimiser le temps d'exécution de votre code.

I. Timers et interruptions sur timer

A. Partie théorique

Un des types de module très utilisé sur les microcontrôleurs est le timer. Sur l'ESP32-S3 de la Neuro-Board S3, il existe 2 groupes de timers généralistes (Group 0 et Group 1) comprenant chacun 2 timers, soit un total de **quatre timers matériels** :

- **Timer 0 (Groupe 0)** : sur 54 bits ;
- **Timer 1 (Groupe 0)** : sur 54 bits ;
- **Timer 0 (Groupe 1)** : sur 54 bits ;
- **Timer 1 (Groupe 1)** : sur 54 bits.

Ces timers étant sur 54 bits ($n = 54$), ils peuvent prendre des valeurs comprises entre **0 et** $2^{54} - 1$, soit environ $1{,}8 \times 10^{16}$ valeurs. Cette grande capacité permet d'obtenir des durées de comptage très importantes avant débordement, comparativement aux timers 8 ou 16 bits de nombreux microcontrôleurs plus simples.

Afin de les utiliser comme compteurs, comptes à rebours ou pour mesurer des durées, il est nécessaire de maîtriser la vitesse à laquelle le timer s'incrémente ou se décrémente. Pour cela, chaque timer peut sélectionner comme source d'horloge `APB_CLK` **(80 MHz par défaut)** ou `XTAL_CLK` **(40 MHz)**. Cette horloge traverse ensuite un **diviseur d'horloge (prescaler) 16 bits**, configurable afin d'ajuster la fréquence d'incrémentation du compteur, comme présenté sur la Figure 15.2.

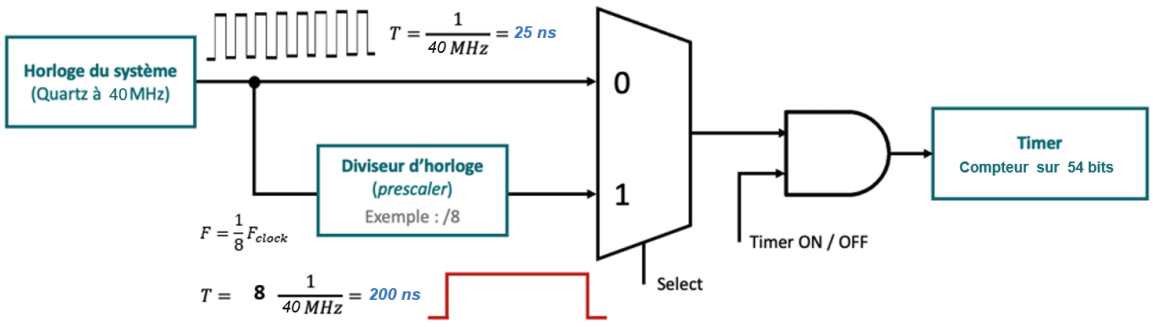


Figure 15.2 – Schéma de principe de la division d'horloge de l'ESP32-S3 (sélection de la source d'horloge à 40 MHz).

Ces divisions d'horloge permettent de **réduire la fréquence de l'horloge sélectionnée**. Par exemple, pour une horloge à 40 MHz, la période vaut :

$$T = \frac{1}{f} = \frac{1}{40\ \text{MHz}} = 25\ \text{ns}$$

. L'utilisation d'un préscaler augmente la période entre deux incrémentations du compteur et permet ainsi de **mesurer des durées plus longues avant que le compteur n'atteigne sa valeur limite**.

Remarque : Il faut veiller à bien choisir les timers utilisés lorsqu'une application nécessite plusieurs bases de temps indépendantes. Un même timer matériel ne possède qu'une seule base de temps à un instant donné ; il ne peut donc pas fonctionner simultanément avec deux fréquences de comptage différentes.Un timer généraliste de l'ESP32-S3 est constitué notamment d'un **compteur matériel 54 bits**, cadencé par une horloge dont la fréquence peut être ajustée à l'aide d'un préscaler.Contrairement à l'ATmega328P, pour lequel certains timers sont également utilisés pour la génération PWM ou certaines fonctions de temporisation, l'ESP32-S3 dispose de **périphériques dédiés**, notamment le module **LEDC pour la génération de signaux PWM**. La répartition des ressources est résumée dans le Tableau 15.3.

	taille	utilité
TIMER0 (Groupe 0)	**54 bits (compte de 0 à 2^{54}-1)**	Timer matériel général : mesure de durée, interruptions périodiques ou ponctuelles, temporisation précise
TIMER1 (Groupe 0)	**54 bits (compte de 0 à 2^{54}-1)**	Même utilisation : alarmes, interruptions, mesure de temps, tâches périodiques
TIMER0 (Groupe 1)	**54 bits (compte de 0 à 2^{54}-1)**	Même utilisation, indépendant des timers du groupe 0
TIMER1 (Groupe 0)	**54 bits (compte de 0 à 2^{54}-1)**	Même utilisation, indépendant des autres timers

Figure 15.3 – Tableau présentant les principales utilisations de Timers dans des applications usuelles.

Note : Contrairement à l'ATmega328P, où plusieurs fonctions Arduino s'appuient sur les mêmes timers matériels et peuvent donc entrer en conflit avec certaines utilisations, l'ESP32-S3 dispose de plusieurs périphériques matériels dédiés :

- Les fonctions `millis()` et `micros()` reposent sur le mécanisme `esp_timer`, qui utilise sur l'ESP32-S3 le **timer système SYSTIMER de 52 bits**. La fonction `delay()`, quant à elle, s'appuie sur **FreeRTOS** (`vTaskDelay()`).
- La génération de signaux PWM avec `analogWrite()` est assurée par le périphérique matériel **LEDC**, indépendamment des timers généralistes. Le pilotage des servomoteurs peut également utiliser le LEDC selon la bibliothèque employée.

Résultat : Les **quatre timers généralistes de 54 bits** sont des ressources distinctes du SYSTIMER et du périphérique LEDC. Les fonctions Arduino précédentes ne les utilisent donc pas directement. Ils peuvent être exploités pour créer des temporisations ou des interruptions matérielles personnalisées, sous réserve qu'une autre bibliothèque ne les utilise pas déjà.

Le fonctionnement du timer étant rappelé, nous allons pouvoir nous en servir. En particulier, nous allons combiner le concept de fonction d'interruption et l'utilisation de timer, afin d'implémenter des **interruptions sur timer**. Ces dernières permettent de déclencher des fonctions à intervalles de temps bien définis, par exemple toutes les secondes : dans le `setup()`, on règle la fréquence du prescaler, la valeur cible de l'alarme et la fonction d'interruption (ISR) associée au timer choisi. Par la suite, ce dernier va compter tout seul **indépendamment de l'exécution du code principal** et, lorsqu'il aura atteint sa valeur cible, il va déclencher une fonction d'interruption.

Nous allons chercher à faire clignoter une LED avec une période de 2 secondes (1s allumée, 1s éteinte) en utilisant deux méthodes.

(a) Méthode 1 : fonction delay()

```
#include <NeuroBoardS3.h>
const int led = 3; // GPIO3 de la NeuroBoard S3
void setup() { pinMode(led, OUTPUT);}
void loop() {
  digitalWrite(led, HIGH);
  delay(1000);
  digitalWrite(led, LOW);
  delay(1000);}
```

Code 15.1 – Code C classique pour faire clignoter une LED

Ce code fonctionne parfaitement mais il a comme inconvénient de « bloquer » le processeur : on doit exécuter la procédure `delay()` pour temporiser le code et, pendant ce temps là, aucune autre instruction ne peut être exécutée. De plus, pour rester en phase avec des spécifications techniques demandant un clignotement avec une période de 2 secondes, on ne doit pas ajouter de code dans la procédure `loop()`.

(b) Méthode 2 : interruptions sur timer

Pour cette seconde approche, nous allons nous appuyer sur la bibliothèque `Ticker.h` (intégrée au cœur ESP32). Elle permet d'implémenter aisément des interruptions matérielles basées sur un temporisateur, en associant directement une fonction de rappel (*callback*) à un intervalle de temps défini.

```
#include <NeuroBoardS3.h>
#include <Ticker.h>
const int led = 3; // Broche GPIO3 de la NeuroBoard S3
volatile bool output = HIGH;
Ticker timerLED;
void flash() {
  digitalWrite(led, output);
  output = !output;
}
void setup() {
  pinMode(led, OUTPUT);
  // Exécute la fonction flash() toutes les 1.0 secondes (1000 ms)
  timerLED.attach(1.0, flash);
  delay(6000); // Maintient le temporisateur actif pendant 6 secondes
  timerLED.detach(); // Équivalent à MsTimer2::stop()
  digitalWrite(led, LOW); // Force l'extinction de la LED à la fin
}
void loop() {
  // Boucle principale libre
}
```

Code 15.2 – Code C pour faire clignoter une LED avec le timer

La configuration du timer s'appuie sur deux méthodes principales de l'objet `Ticker` :

- `timerLED.attach(1.0, flash)` : associe la fonction de rappel (callback) flash au temporisateur et définit sa période d'exécution répétitive en secondes (ici 1,0 s = 1000 ms) ;
- `timerLED.detach()` : arrête le temporisateur et désactive l'appel périodique de la fonction d'interruption. ;

Le gros avantage de ce code est, comme pour les interruptions externes, qu'il ne surcharge pas la procédure `loop()`, et n'y appelle pas la fonction à exécuter : c'est le périphérique timer de l'ESP32-

S3 qui gère le décompte en arrière-plan et déclenche automatiquement l'interruption (ici la fonction `flash()`) à intervalles réguliers.

B. Partie expérimentale

(a) Méthode 1

E1. Dans le premier code, comment peut-on faire varier la durée du clignotement de la LED ?

E2. Réaliser l'expérience en choisissant une durée de clignotement de 1 seconde puis de 0,5 seconde. Combien de fois la LED clignote-t-elle ?

Rajouter l'instruction `delay(2000);` à la fin du code de la procédure `loop()`.

E3. Qu'observe-t-on ?

(b) Méthode 2

E4. Dans le second code, comment peut-on faire varier la durée du clignotement de la LED ?

E5. Réaliser l'expérience en choisissant une durée de clignotement de 1 seconde puis de 0,5 seconde. Combien de fois la LED clignote-t-elle ?

E6. Que faire pour qu'elle ne clignote que 7 fois ?

Rajouter l'instruction `delay(2000);` à la fin du code de la procédure `loop()`.

E7. Qu'observe-t-on ?

II. GPIOs de l'ESP32-S3

A. Les registres des GPIOs

Le sigle GPIO (General Purpose Input/Output) signifie entrée/sortie à usage général. Ils sont très utilisés dans les microcontrôleurs et permettent la communication avec des composants électroniques et des circuits externes.

Comme on l'a vu dans un précédent TP, les GPIOs (broches) peuvent être configurés pour être reconnus comme une entrée ou bien comme une sortie :

- dans le cas d'une **entrée**, on peut lire l'état logique de l'élément branché sur la broche en question en lisant le contenu du registre interne correspondant ;
- dans le cas d'une **sortie**, on écrit directement dans le registre interne afin de modifier l'état de la sortie correspondante.

Jusqu'à présent, nous avons configuré les broches à l'aide de la fonction `pinMode()`, puis lu ou modifié leur état logique à l'aide des fonctions `digitalRead()` et `digitalWrite()`. Mais que se passe-t-il réellement à l'intérieur du microcontrôleur 32 bits ?

La Figure 15.4 présente les broches et les principales fonctionnalités des GPIO de l'ESP32-S3 (boîtier QFN56). Contrairement à certains microcontrôleurs 8 bits utilisant des registres de ports de 8 bits,

tels que `PORTx`, l'ESP32-S3 possède **45 GPIO physiques** dont les fonctions peuvent être configurées et routées vers différents périphériques internes grâce à **l'IO MUX** et à la **GPIO Matrix**.

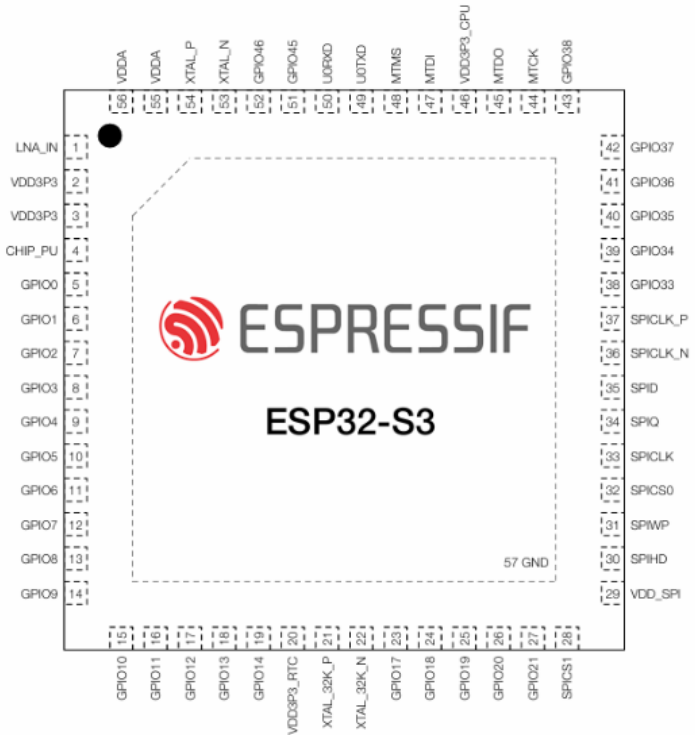


Figure 15.4 – Brochage (Pinout) du boîtier QFN56 de l'ESP32-S3 et la disposition de ses GPIOs (GPIO0 à GPIO48).

Comme présenté sur la Figure 15.5, la gestion des GPIO de l'ESP32-S3 repose sur plusieurs registres matériels. Pour les GPIO 0 à 31, on trouve notamment :

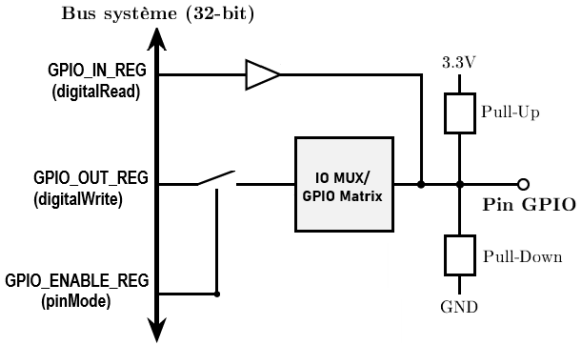


Figure 15.5 – Schéma simplifié de la structure interne d'une broche GPIO de l'ESP32-S3 et son équivalence avec les registres 32 bits .

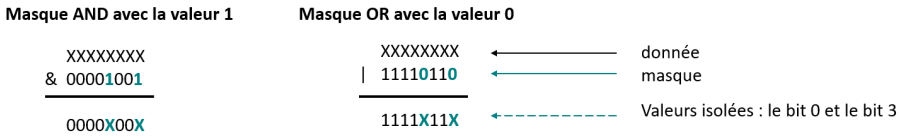
- `GPIO_IN_REG` : permet de lire l'état logique des GPIO 0 à 31 ;
- `GPIO_OUT_REG` : contient les valeurs de sortie des GPIO 0 à 31 ;
- `GPIO_ENABLE_REG` : permet d'activer ou de désactiver le **pilote de sortie** des GPIO 0 à 31 ;
- **GPIO Matrix / IO MUX** : permet de configurer et de router les broches vers les différents périphériques internes (UART, SPI, LEDC, etc.).

Pour les GPIO de numéro supérieur à 31, des registres complémentaires tels que `GPIO_IN1_REG`, `GPIO_OUT1_REG` et `GPIO_ENABLE1_REG` sont utilisés.

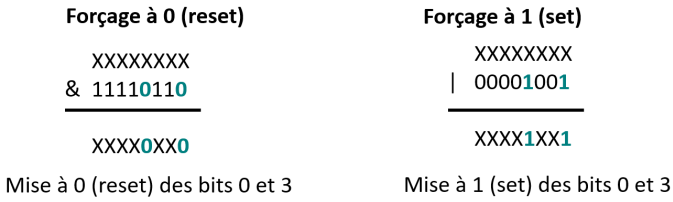
B. Programmation bas niveau : les opérations logiques

Les opérations logiques permettent de générer du code plus compacte et plus efficace dans la mesure où ils seront traduit en langage machine prenant bien souvent moins de lignes de code assembleur, et donc moins d'opérations à exécuter. Passons en revue les principales opérations courament utilisées.

(a) Isoler / lire des bits



(b) Forcer à 1 (set) ou à 0 (reset) des bits

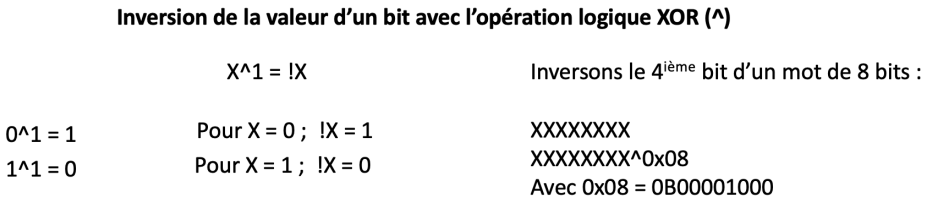


(c) Décalage de bit



Remarque : Cela permet de multiplier ou de diviser un nombre binaire par 2^n, soit n crans de décalage. Les concepts mathématiques et logiques ne changent pas (les masques AND, OR et les décalages vers la gauche/droite fonctionnent exactement de la même manière dans la logique booléenne de n'importe quel processeur. Les registres de l'ESP32-S3 sont sur 32 bits. Les masques s'étendent donc jusqu'à 32 bits (exemple : 0b00000000000000000000000000000001).

(d) Inversion de bit



(e) Exemple 1 : Faire clignoter une LED

On cherche à faire clignoter une LED avec une période de 1 seconde. Ci-dessous, deux approches possibles.

```ino
#include <NeuroBoardS3.h>
const int led = 3; // GPIO3 sur NeuroBoard S3
void setup() {
  pinMode(led, OUTPUT);
}
void loop() {
  digitalWrite(led, HIGH);
  delay(1000);
  digitalWrite(led, LOW);
  delay(1000);
}
```

Code (15.3) Programme en haut niveau (fonction API Arduino)

```ino
#include <NeuroBoardS3.h>
#include "soc/gpio_reg.h"
#define LED_PIN 3

void setup() {
  // Active le GPIO3 en sortie (remplace DDRD)
  REG_WRITE(GPIO_ENABLE_REG, REG_READ(GPIO_ENABLE_REG) | (1 << LED_PIN));
}
void loop() {
  // Met le GPIO3 à l'état HAUT (Set)
  REG_WRITE(GPIO_OUT_W1TS_REG, (1 << LED_PIN));
  delay(1000);
  // Met le GPIO3 à l'état BAS (Clear)
  REG_WRITE(GPIO_OUT_W1TC_REG, (1 << LED_PIN));
  delay(1000);
}
```

Code (15.4) Programme en bas niveau (ESP32-S3)

Figure 15.6 – Clignotement de LED : Comparaison entre la programmation haut niveau et bas niveau sur ESP32-S3.

(f) Exemple 2 : Appuyer sur un bouton

```
#include <NeuroBoardS3.h>
const int bouton = 4; // GPIO04 sur NeuroBoard S3
void setup() {
  pinMode(bouton, INPUT);
  Serial.begin(115200);
}
void loop() {
  bool etat_bouton = digitalRead(bouton);
  Serial.println(etat_bouton);
  delay(100);
}
```

Code (15.5) Programme en haut niveau (fonction API Arduino)

```
#include <NeuroBoardS3.h>
#include "soc/gpio_reg.h"
#include "soc/io_mux_reg.h"
#define bouton 4
void setup() {
  // 1. Sélectionne la fonction GPIO sur la matrice IO_MUX
  PIN_FUNC_SELECT(GPIO_PIN_MUX_REG[bouton], PIN_FUNC_GPIO);
  // 2. Active le tampon d'entrée numérique (Input Enable)
  SET_PERI_REG_MASK(GPIO_PIN_MUX_REG[bouton], FUN_IE);
  // 3. Configure en entrée (Remplace le DDRD de l'AVR)
  REG_WRITE(GPIO_ENABLE_REG, REG_READ(GPIO_ENABLE_REG) & ~(1 << bouton));
  Serial.begin(115200);
}
void loop() {
  byte etat_bouton;
  // Lecture directe du registre 32 bits (Remplace le PIND de l'AVR)
  etat_bouton = (REG_READ(GPIO_IN_REG) & (1 << bouton)) >> bouton;
  Serial.println(etat_bouton);
  delay(100);
}
```

Code (15.6) Programme en bas niveau

Figure 15.7 – Lecture de l'appui sur un bouton : Comparaison entre la programmation haut niveau et bas niveau sur ESP32-S3.

C. Partie expérimentale

E8. Proposer et tester un code utilisant des **opérateurs bit à bit** permettant d'inverser l'état d'une LED branchée sur la broche **GPIO 5** lorsqu'un bouton, branché sur la broche **GPIO 4** et configuré en **pull-up**, est appuyé.

On cherche ensuite à comparer le temps d'exécution de la fonction `digitalWrite()` à celui d'une écriture directe dans les registres GPIO à l'aide d'opérations bit à bit. Cette fonction pourra, dans la suite, être appelée `fastDigitalWrite()`.

E9. Proposer un protocole expérimental permettant de mesurer et de comparer les temps d'exécution de `digitalWrite()` et de `fastDigitalWrite()`. Mettre en œuvre ce protocole et discuter les résultats obtenus.



L'essentiel

- L'ESP32-S3 intègre 4 timers généraux de 64 bits, ainsi que des modules dédiés comme le LEDC (PWM) et le MCPWM, capables de fonctionner indépendamment de l'exécution du cod.
- Les **interruptions sur Timer** sont des fonctions appelées par le processeur lorsque le dit timer arrive à une valeur pré-définie. Cela permet d'exécuter une fonction à une fréquence bien précise.
- Le recours à des opérateurs logiques permet d'optimiser le temps d'exécution d'un code.
  - **Lecture** : & ou | accompagné d'un masque
  - **Forçage** : & 0 (reset) et | 1 (set)
  - **Décalage** : >> (à droite) et << (à gauche)
  - **Inversion** : ^